6.896
2/4/04
L1.1

6.896 Theory of Parallel Hardware

Administrivia «Mention webpage bug»
Overview «Discuss models»

Systolic computation

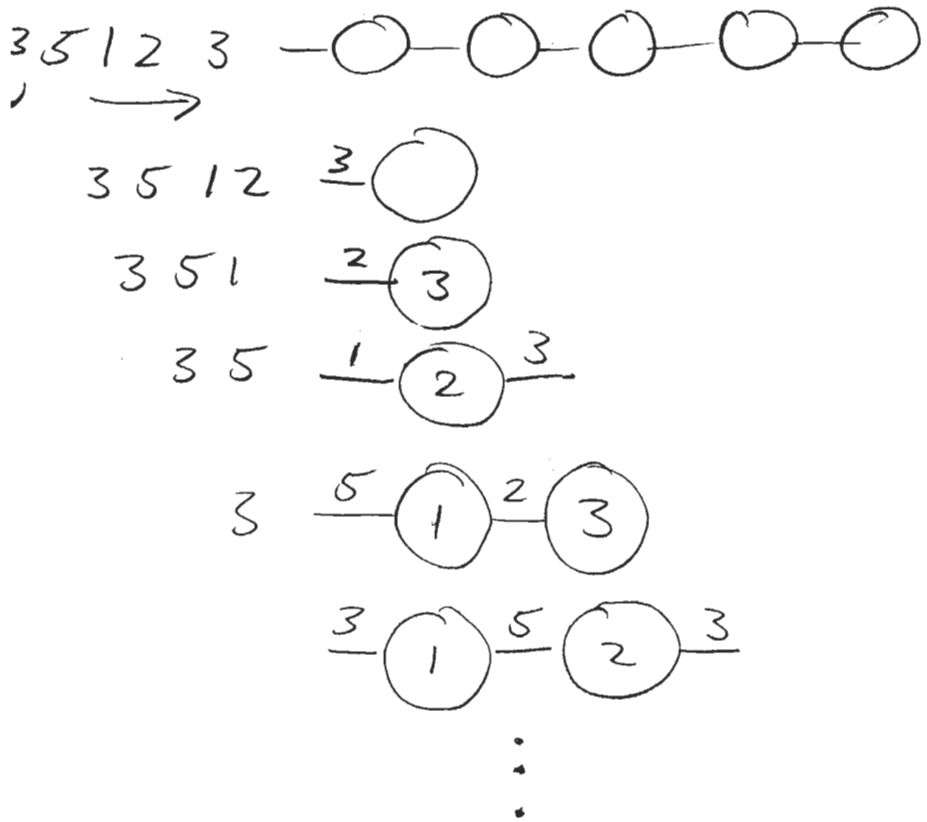
E.g. linear array I/O —O—O—O—O— ... —O

"Fixed-connection" network
1. Underlying graph fixed.
2. Local communication only
3. I/O location restricted.

At each step of a globally synchronous clock, each processor
1. receives inputs from neighbors (or I/O)
2. inspects local memory
3. performs local computation
4. updates local memory
5. generates outputs for neighbors

Example: Sorting

- Accept left input
- Compare input to stored value
- Store smaller value
- Output bigger # to right.



Correctness: induction

N inputs. How many steps? $2N = \Theta(N)$

<<Discuss outputting of values.>>

Total time = $3N$

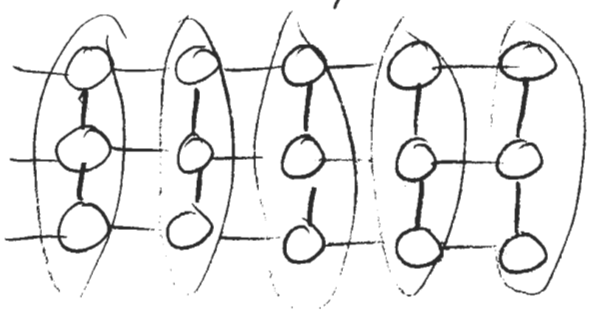
Sorting in the bit model (vs. word model)

- One processor per bit.

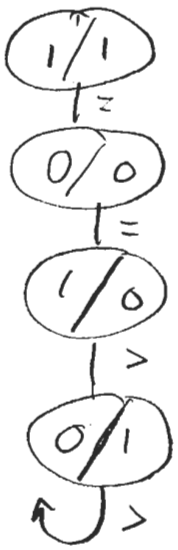
N k-bit #'s

3	5	1	2	3
0	1	0	0	0
1	0	0	1	1
1	1	1	0	1

$N \times k$ array

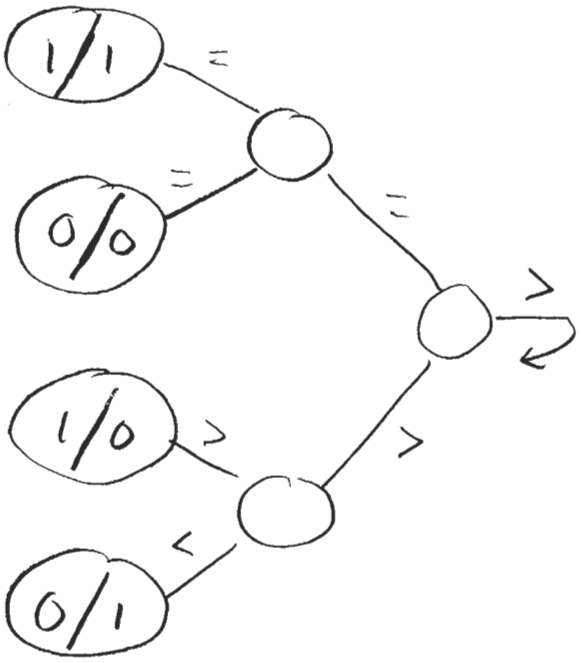


Comparing 2 k-bit words



$\Theta(k)$ steps to compute
Sort in $\Theta(Nk)$ time

Faster comparison – binary tree



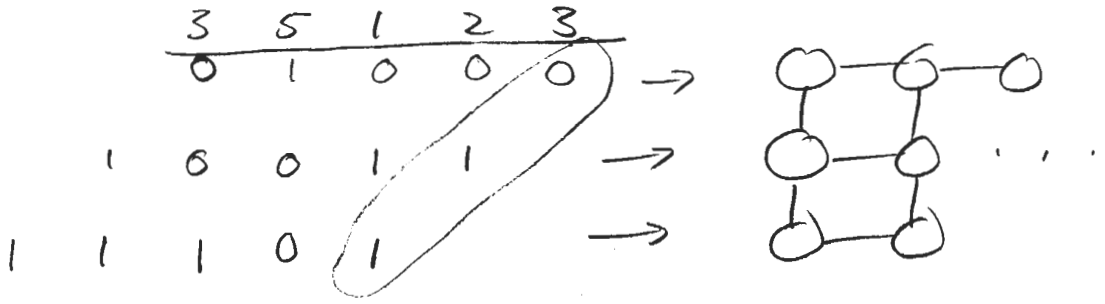
$\Theta(\lg k)$ compare
Sort in $\Theta(N \lg k)$

$\lg = \log_2$

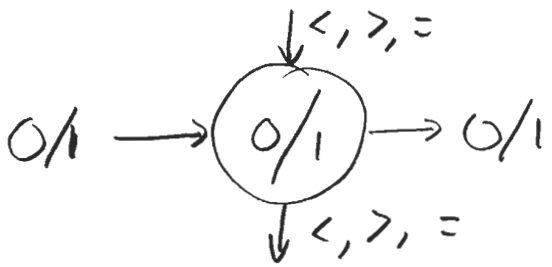
6.896
2/4/04
L1.4

Pipelining

- compare while sorting
- stagger bits of input.



Each processor:



Time = $\Theta(N+k)$ bit steps.
Can we do better on $N \times k$ array?

Lower bounds

1. I/O bandwidth
   Nk bits to input at k places $\Rightarrow \Omega(N)$ steps.

2. Network diameter
   $\Omega(N+k)$

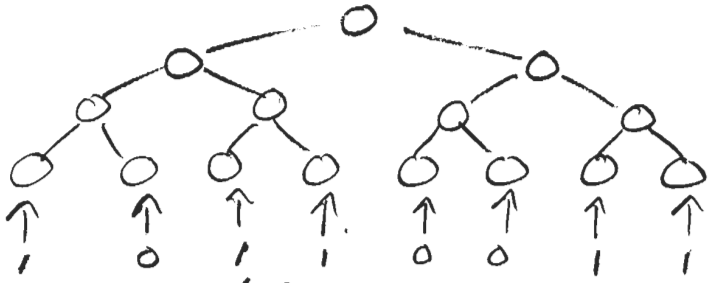
3. Communication bandwidth (bisection width).

$$T \geq \frac{\#\text{ bits crossing cut}}{\text{size of cut}}$$



$$T \geq \frac{\Theta(Nk)}{\Theta(k)} = \Theta(N).$$

Problem. N 1-bit #'s input at N leaves of complete binary tree. Time to sort?

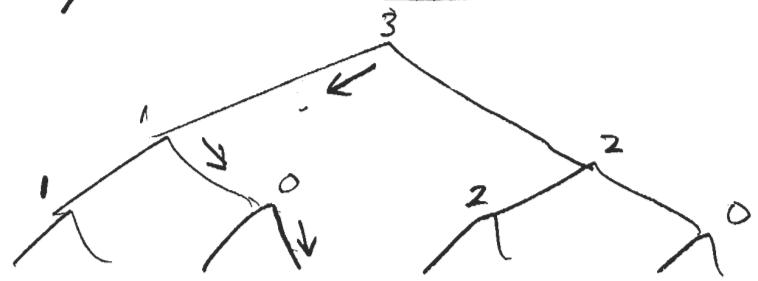


I/O: $T \geq N/N = 1$.
Diam: $T \geq 2 \lg N$
BW: $T \geq \Theta(N)/1 = \Theta(N)$

$\therefore$ Sorting N 1-bit #'s takes $\Omega(N)$ time on CBT.

Wrong! Can sort in $O(\lg N)$ time!

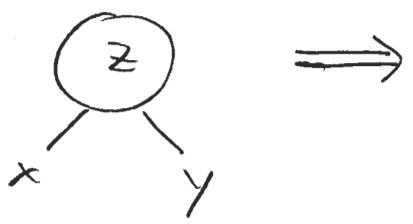
Idea: Only need to count # 0's.

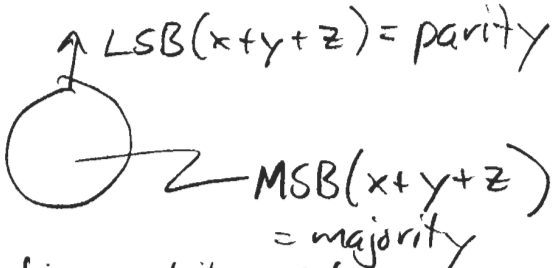


Input	1	0	1	1	0	0	1	1
Output	0	0	0	1	1	1	1	1

Sum 0's upward, select downward.

1-bit summer





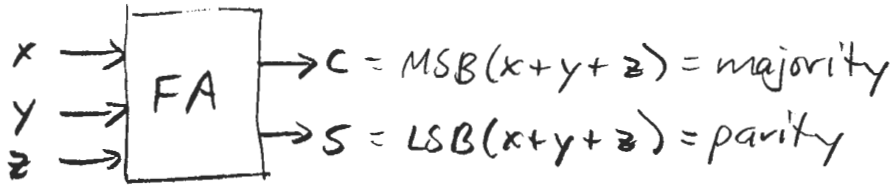
Pipeline in bit model.

Q. Why doesn't BW lower bound hold?
A. Didn't really need to ship $\Theta(N)$ bits across bisection. Could encode info more compactly.
Careful!

<<Sort with $\Theta(\lg N)$ procs in linear array>>
<<Firing squad - $\Theta(1)$ size state>>

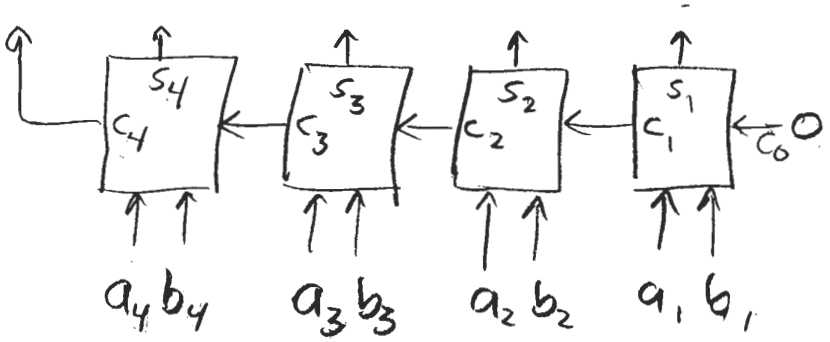
Addition

Basic component: **Full adder** - combinational



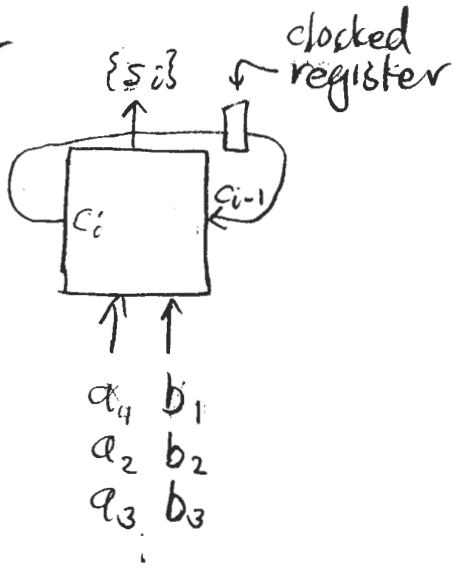
Problem:
Add 2 N-bit numbers

Ripple-carry adder



N-bit #'s $\Rightarrow$ $\Theta(N)$ time, $\Theta(N)$ HW, combinational

Serial adder



$\Theta(N)$ time, $\Theta(1)$ hardware, sequential (clocked)

6.896
2/9/04
L 2.2

Fast addition

Idea: carries are the hard part.
Know carries ⇒ compute sum in $\Theta(1)$ time
How? Array of full adders. «Show on ripple-carry adder»

	1	0	1	0	1	0	1
	1	1	1	0	0	1	1
1	1	0	0	1	0	0	0
	g	p	g	k	p	p	g (k)

Classify stages:

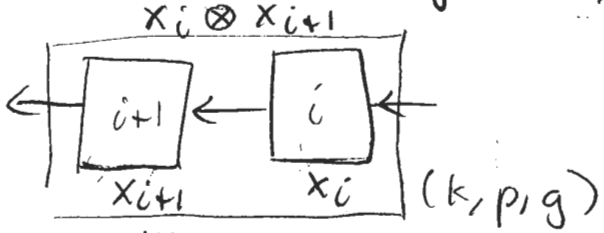
kill: $\begin{smallmatrix}0\\0\end{smallmatrix}$ ⇒ carry-out = 0

propagate: $\begin{smallmatrix}0\\1\end{smallmatrix}$ or $\begin{smallmatrix}1\\0\end{smallmatrix}$ ⇒ carry-out = carry-in

generate: $\begin{smallmatrix}1\\1\end{smallmatrix}$ ⇒ carry-out = 1

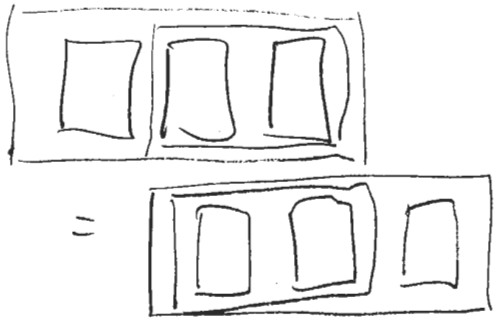
Carry into stage = { 1 if most recent non-p is k
0 otherwise

When do 2 consecutive stages kill, prop, gen?



x_i \ x_{i+1}	k	p	g
k	k	k	g
p	k	p	g
g	k	g	g

Associative!



Theorem. Let x_i = carry status of stage i, where $x_0 = k$. Define $y_i = x_0 \otimes x_1 \otimes \cdots \otimes x_i$.

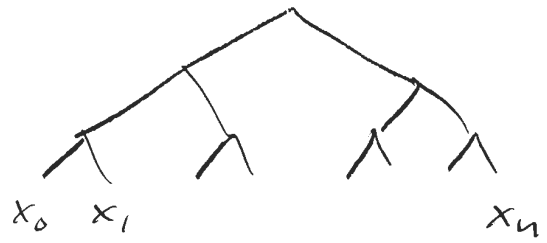
Then $y_i = k \Rightarrow c_i = 0$
$y_i = g \Rightarrow c_i = 1$
$y_i = p$ does not occur.

Proof. Induction on i. ⊠

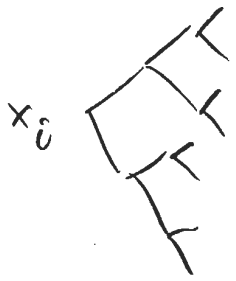
Log-time circuit:

$$y_0 = x_0$$
$$y_1 = x_0 \otimes x_1$$
$$y_2 = x_0 \otimes x_1 \otimes x_2$$
$$y_N = x_0 \otimes x_1 \otimes \cdots \otimes x_N$$

Use tree for each calculation:



Use tree to broadcast inputs (bounded-degree network):



Time = $\Theta(\lg N)$, HW = $\Theta(N^2)$.

Carry-lookahead addition

$\Theta(\lg N)$ time, $\Theta(N)$ HW.

"Parallel prefix"

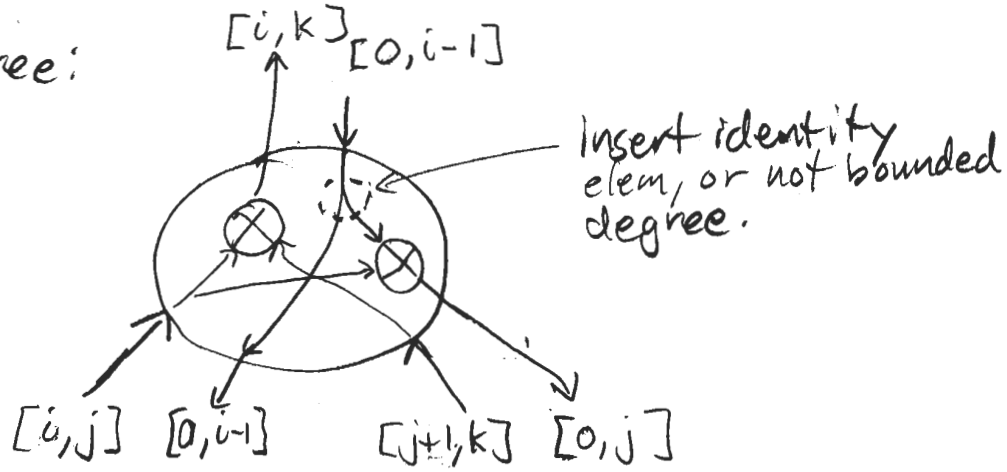
Let $[i, j]$ denote $x_i \otimes x_{i+1} \otimes \cdots \otimes x_j$

Lemma. $[i, j] \otimes [j+1, k] = [i, k]$ ⊠

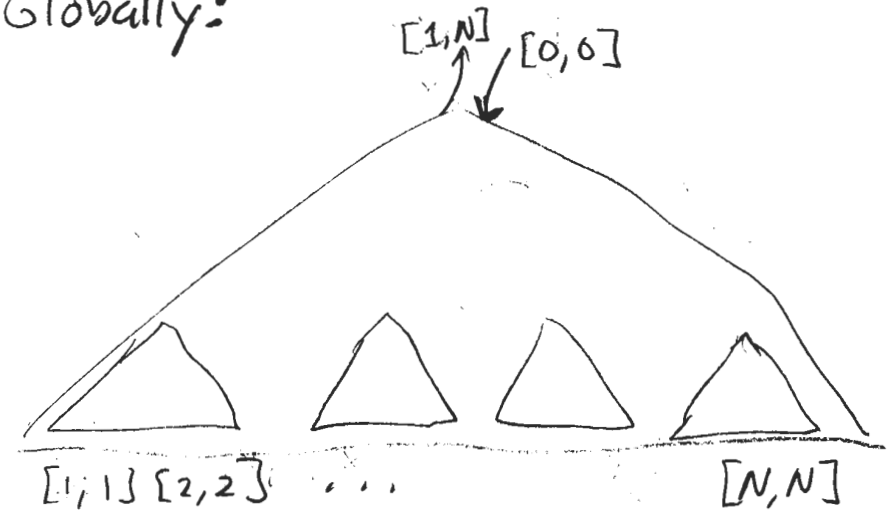
$x_i = [i, i]$

$y_i = [0, i]$

Build tree:



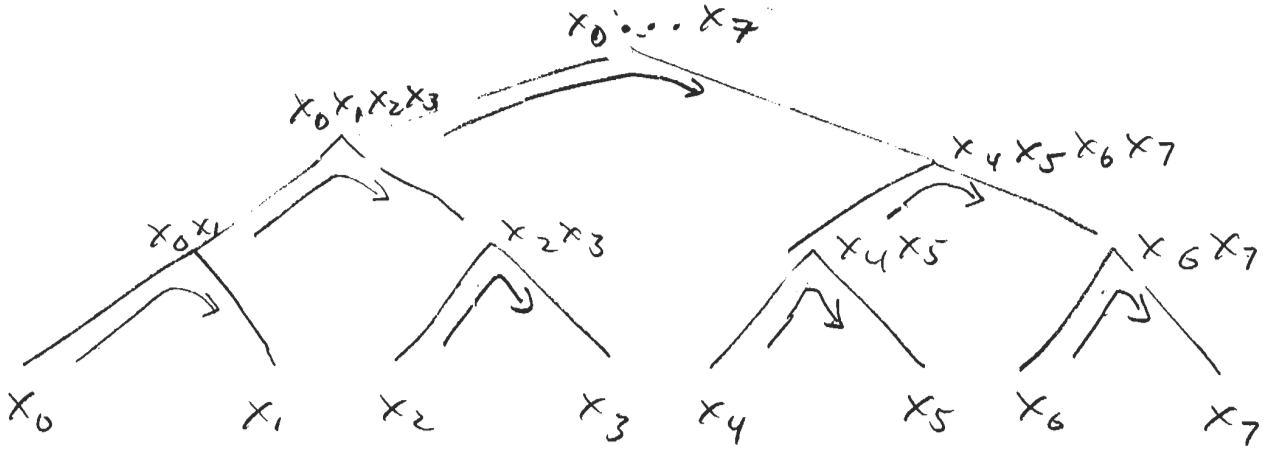
Globally:



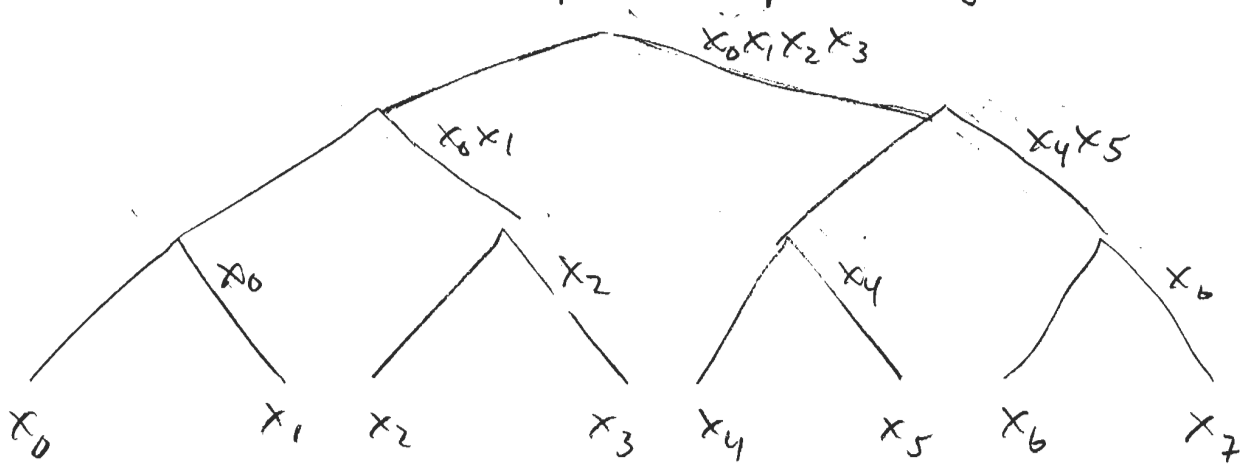
Left child values are passed up.

6.896
2/9/04
L 2.5

Similar method:



Left child values are passed up and right



Postscript kill, propagate, generate first used in Harvard relay calculator circa mid-1940's.

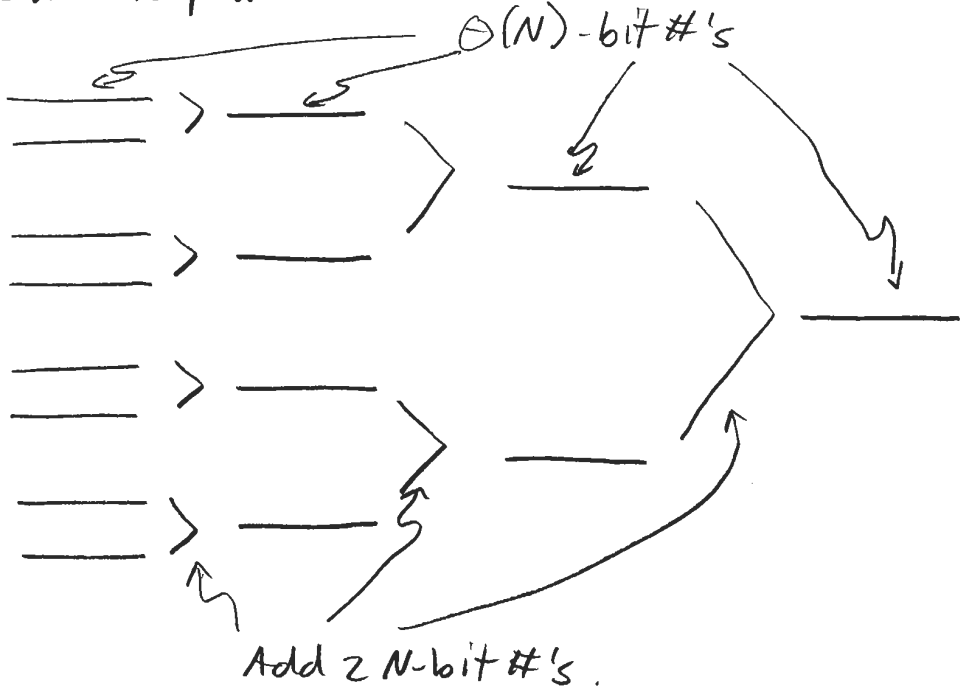
O(1)-time addition (in their model).

Adding N N-bit numbers

Add 2 N-bit #'s in $\Theta(\lg N)$ steps, $\Theta(N)$ HW
Add N 1-bit #'s in $\Theta(\lg N)$ steps, $\Theta(N)$ HW.

N N-bit #'s.

Break into pairs:



$\Theta(N^2)$ HW
$\Theta(\lg^2 N)$ steps
$\uparrow (\lg N)^2$

Can do better!

Carry-save addition

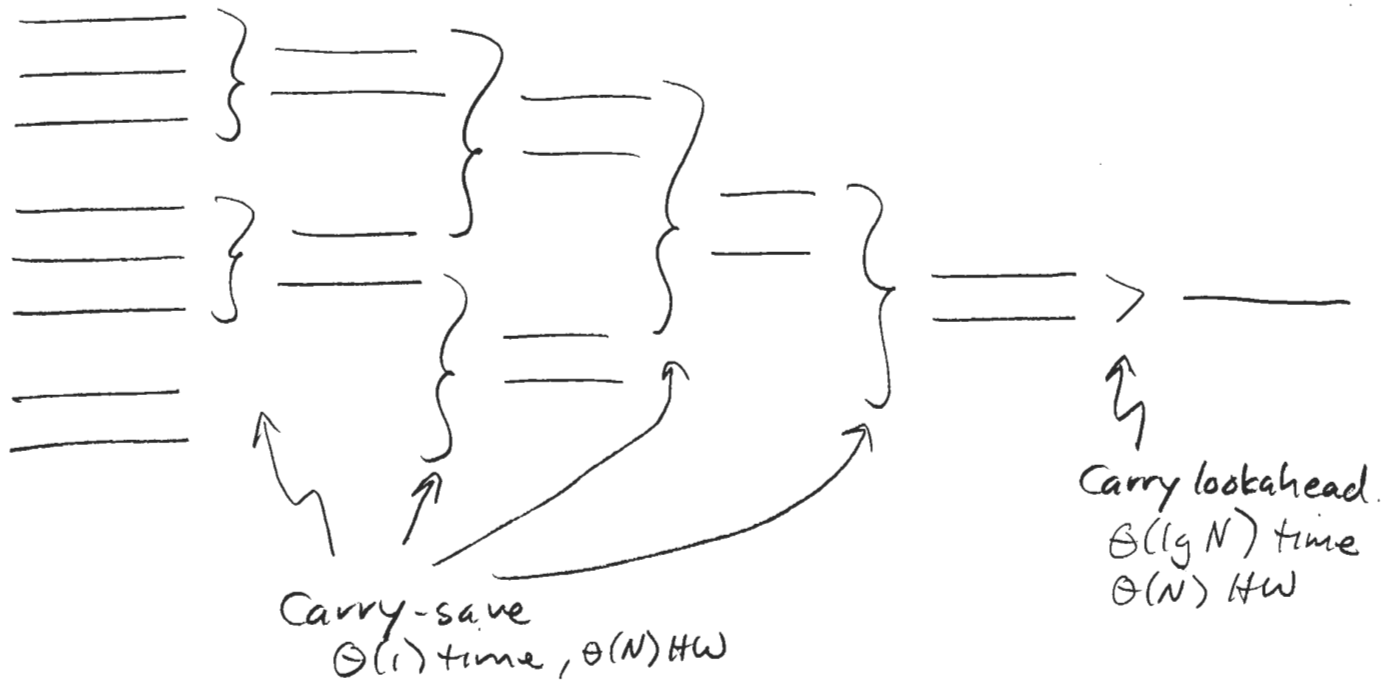
3 N-bit #'s → 2 #'s in 1 step!

Ex.

```
  1 0 1 1 0 1 0
  1 0 1 0 1 1 1
  0 1 1 1 0 1 1
-----------------
  0 1 1 0 1 1 0   ← parity
1 0 1 1 0 1 1     ← majority.
```

Array of full adders!

Wallace tree



Time $T(N) = T(\lceil 2N/3 \rceil) + \Theta(1)$ $(T(2) = 0)$

Master theorem: $T(N) = \Theta(\lg N)$

$T(N) \approx \log_{3/2} N$

HW $H^{\#}(N) = H^{\#}(\lceil 2N/3 \rceil) + \Theta(N)$ // Number of Hardware Components

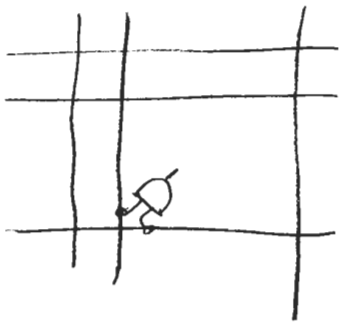
$= \Theta(N)$

$\Rightarrow H(N) = \Theta(N) \cdot H^{\#}(N) = \Theta(N^2)$

Integer multiplication

```
    0 1 1
    1 0 1
  -------
    0 1 1
  0 0 0
0 1 1
-------
```

- N #'s with $\leq 2N$ bits each
- Form partial products with matrix of AND gates:



← Avoid broadcast with tree.

- $\Theta(N^2)$ HW to form partial products in $\Theta(\lg N)$ time.

Wallace-tree add: $\Theta(\lg N)$ time, $\Theta(N^2)$ HW.

Convolution

$a = (a_1, a_2, \ldots, a_N)$
$b = (b_1, b_2, \ldots, b_N)$

Compute $c = (c_1, c_2, \ldots, c_{2N-1})$, where

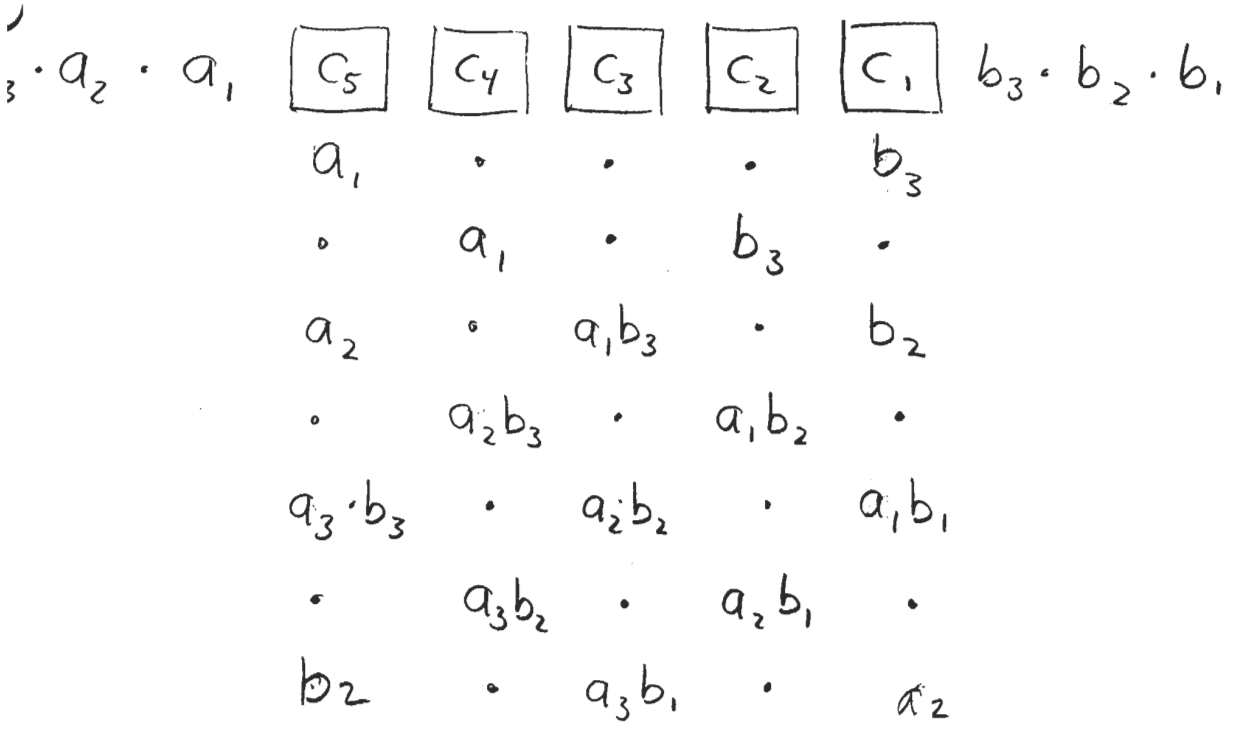
$$c_1 = a_1 b_1$$
$$c_2 = a_1 b_2 + a_2 b_1$$
$$\vdots$$
$$c_k = a_1 b_k + \cdots + a_k b_1$$
$$\vdots$$
$$c_{2N-1} = a_N b_N$$

Polynomial multiplication

$$\left(a_1 + a_2 x + a_3 x^2 + \cdots + a_N x^{N-1}\right)\left(b_1 + b_2 x + \cdots + b_N x^{N-1}\right)$$
$$= c_1 + c_2 x + \cdots + c_{2N-1} x^{2N-2}$$

6.896
2/11/04
L3.4

Linear array:



$\Theta(N)$ time, $\Theta(N)$ HW.

50% utilization
1. Solve 2 problems
2. Coalesce
3. Interlace
4. Adjust timing
5. Make multiplier slower

Integer mult

$$\begin{array}{r} a_3\ a_2\ a_1 \\ b_3\ b_2\ b_1 \\ \hline a_3b_1\ a_2b_1\ a_1b_1 \\ a_3b_2\ a_2b_2\ a_1b_2 \\ a_3b_3\ a_2b_3\ a_1b_3 \\ \hline \end{array}$$

Idea:
- Similar to convolution
- Send carry to left.

Coarsening

Def. Sup. alg runs in T time on P procs.
The **work** is $P \cdot T$.

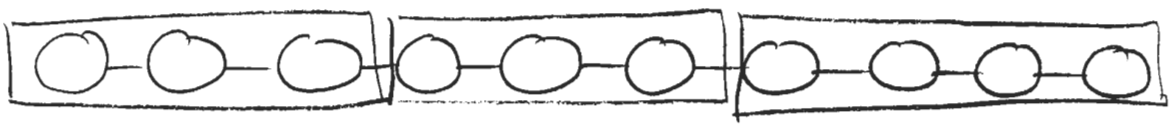
$$\text{Efficiency} = \frac{\text{Work} = \text{Time of (best) serial alg.}}{\text{Work of parallel alg.}}$$

Theorem A time-T, P-proc. alg can be simulated on an m-proc machine, where $m < P$, in $T \cdot \lceil \frac{P}{m} \rceil$ steps (assuming free multiplexing).

Proof. Simulate 1 P-proc step with $\lceil P/m \rceil$ m-proc steps. ☒

Note: For fixed-connection networks, must also embed "guest" network in "host" network.

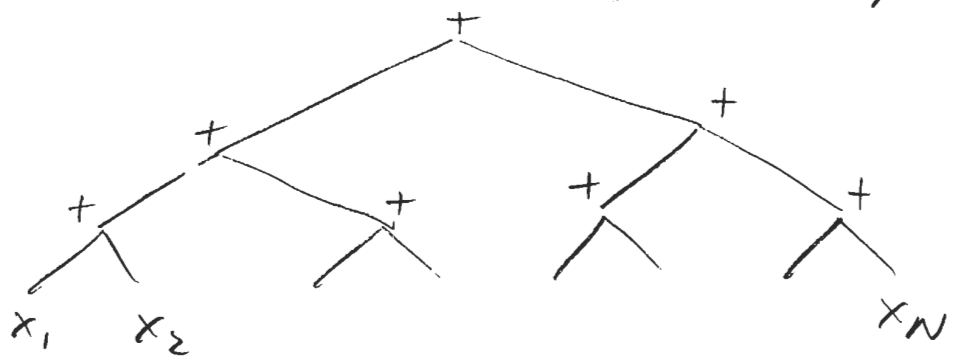
Ex. $P = 10$, $m = 3$



$$\begin{aligned}\text{Work of } m\text{-proc alg.} &= m \cdot T\lceil P/m \rceil \\ &\le m \cdot T\left(\tfrac{P}{m} + 1\right) \\ &= T \cdot (P + m) \\ &= \Theta(PT), \text{ since } m < P.\end{aligned}$$

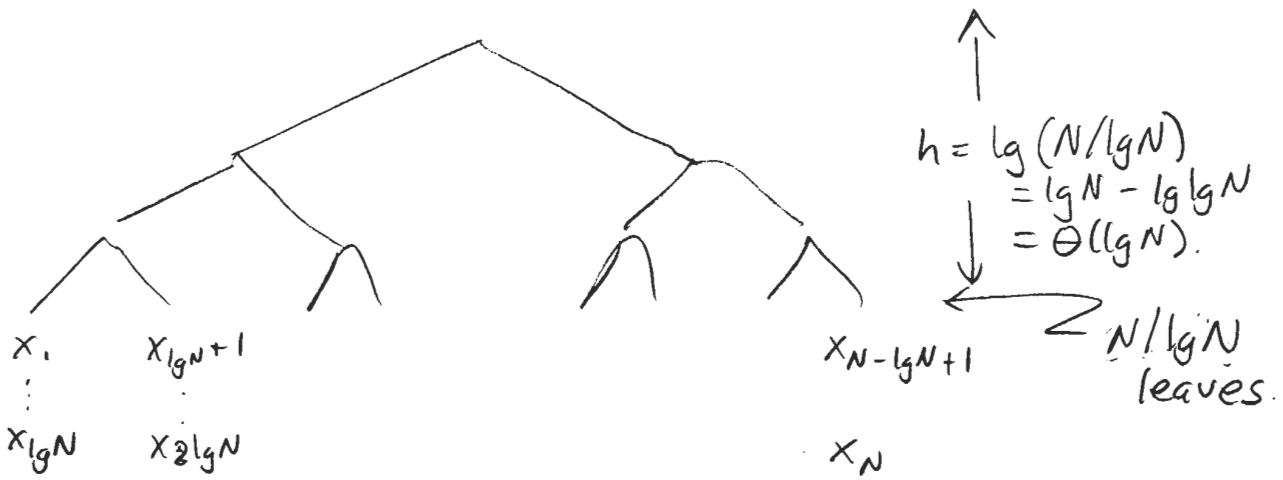
Thus, no asymptotic loss in efficiency.
Motivates study of fine-grained algs, since can always slow down for coarse grained.

Ex. Sum N numbers on complete binary tree.



Time = $\Theta(\lg N)$.
HW = $\Theta(N)$
Work = $\Theta(N \lg N)$.

More efficient:



Time = $\Theta(\lg N)$
HW = $\Theta(N/\lg N)$
Work = $\Theta(N)$.

Michael Bender lecturing

6.896
2/17/04
L4

Today: Division — compute n leading bits of u/y.

Elementary-school approach: $1/3$

$$
\begin{array}{r}
.010101 \\
11\overline{)1.0000} \\
11 \\
\hline
100 \\
11 \\
\hline
100
\end{array}
$$

Simplifications

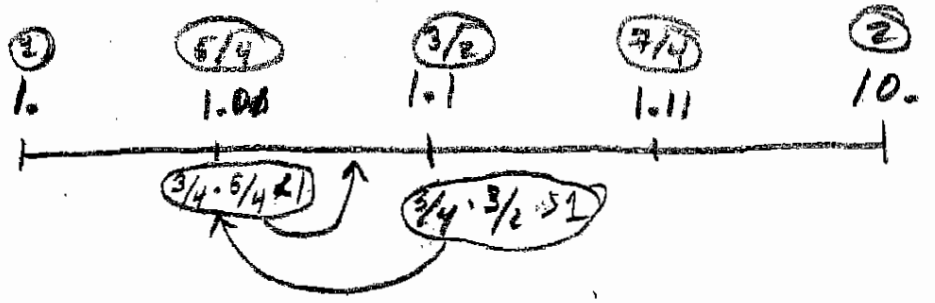
1) Focus on computing $1/y$ because can mult by u

2) Rescale y so that $1/2 \le y < 1 \Rightarrow 1 < 1/y \le 2$.

First Approach: Binary Search

Let x_i = i^{th} guess for $1/y$

$x_0 = 3/2$.



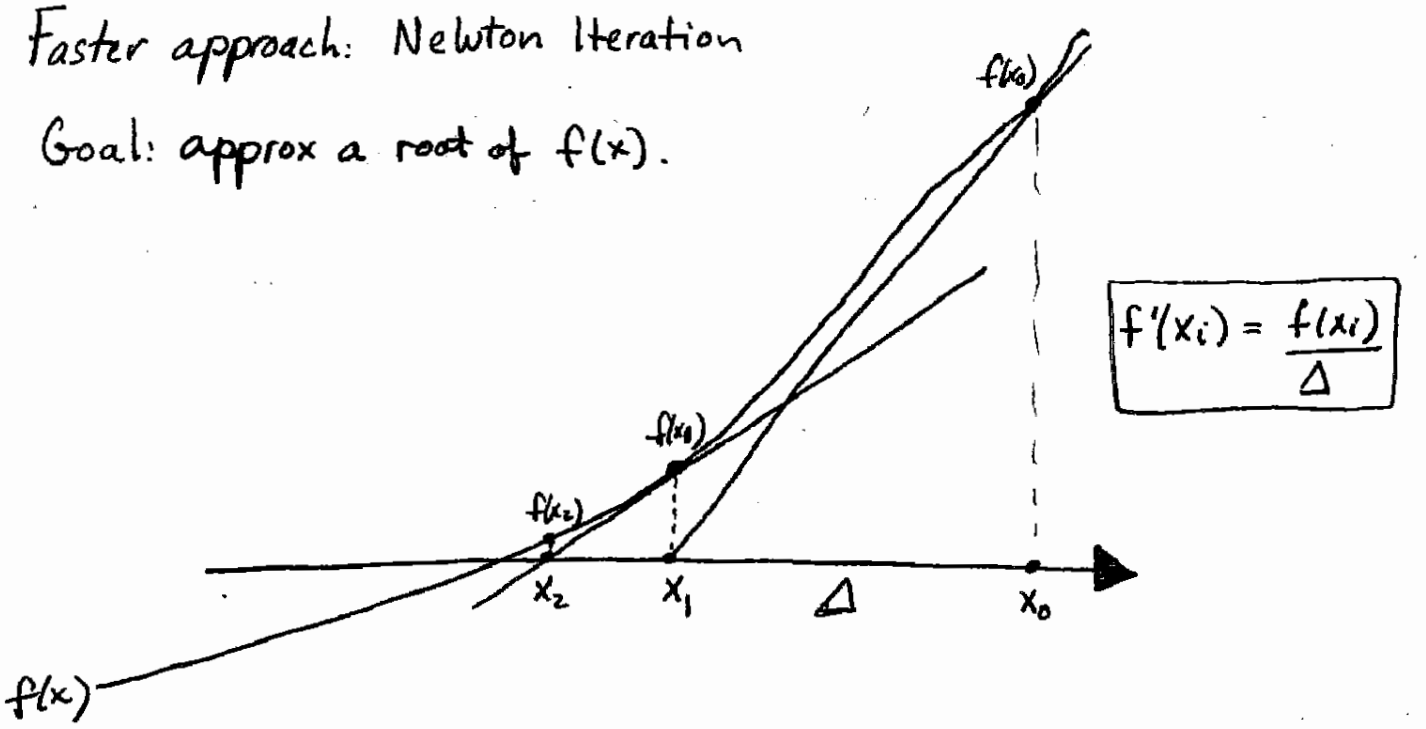
Ex: $y = 13 \Rightarrow y = 3/4$

Performance: One bit of accuracy per iteration

$O(N)$ rounds $\Rightarrow O(N \log N)$ time.

Faster approach: Newton Iteration

Goal: approx a root of $f(x)$.



$$f'(x_i) = \frac{f(x_i)}{\Delta}$$

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

To compute $1/y$, find root of

$$f(x) = 1 - xy$$

$$\Rightarrow \quad f'(x) = -y.$$

$$x_{i+1} = x_i - \frac{1 - x_i y}{-y}$$

$$= x_i + \frac{1}{y}(1 - x_i y)$$

<< Uh oh. To compute $\frac{1}{y}$, all we need is $1/y$. >>

Ex: $y = .11$

$x_0 = 1.1$

$x_1 = 1.0101$

$x_2 = 1.0101010001$

etc

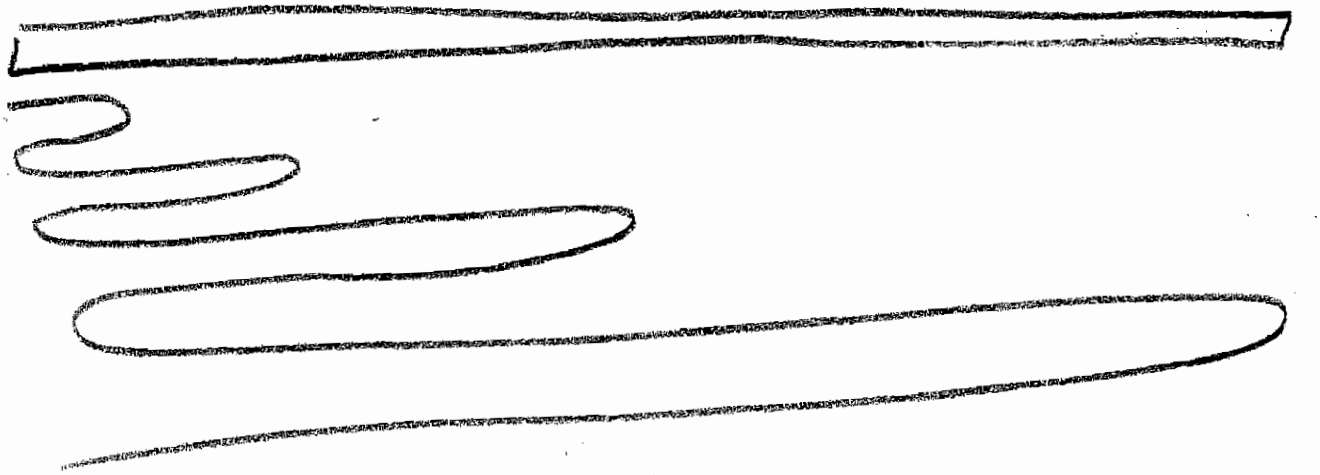
Division on N-cell Linear Array

Lg N iterations each composed of $O(N)$ steps.

$\Rightarrow O(N \lg N)$ steps on N-cell linear array

Better idea:

Precision of x_i only kept to $2^{i+1}+1$ bits.



Cost: $T(N) = T(N/2) + \Theta(N)$

$= \Theta(N)$.

Computing x/y in $O(\lg x)$ steps

Easy case: y fixed $\Rightarrow$ precompute $1/y$.

Simplifications:

1) focus on $1/y$ (as before)

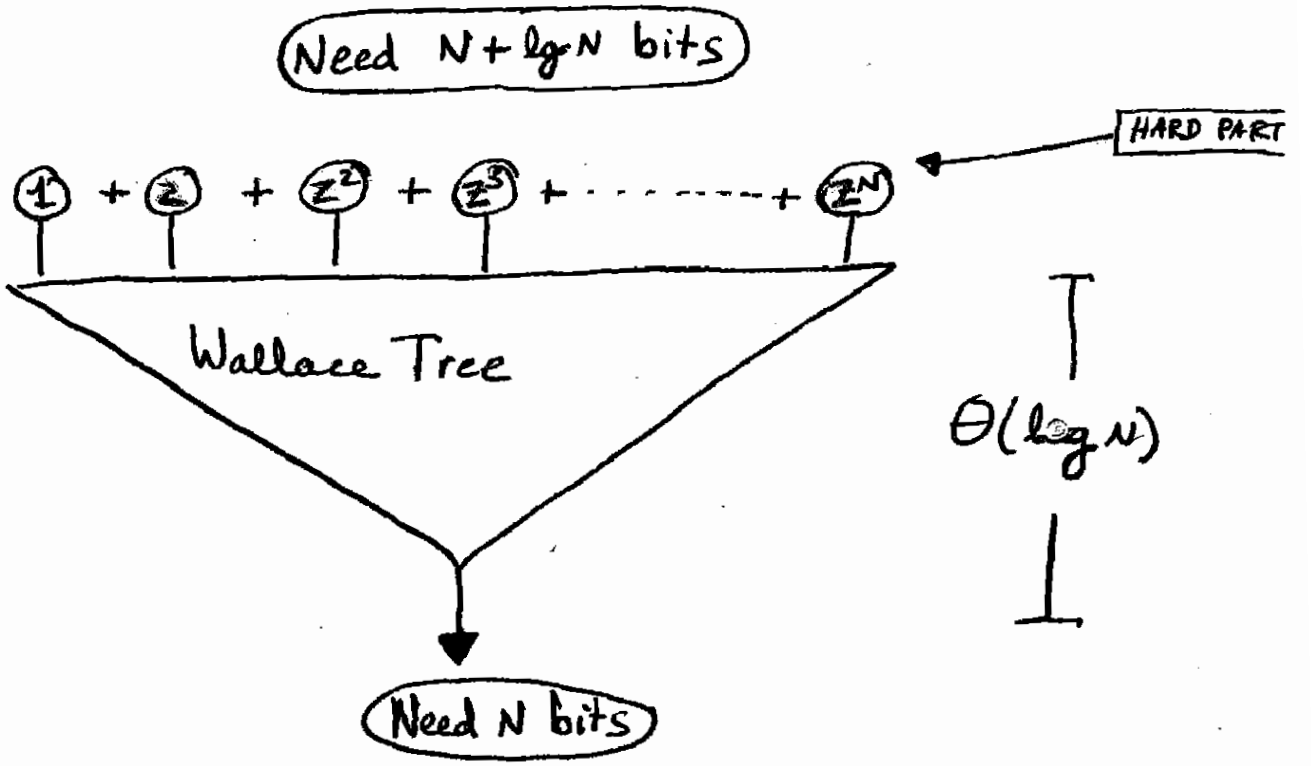
2) rescale so $y = 1 - z$, $0 \le z \le 1/2$ (as before)

3) $$\frac{1}{y} = \frac{1}{1-z}$$
$$= 1 + z + z^2 + z^3 + \cdots$$

Let
$$x_i = 1 + z + z^2 + \cdots + z^i$$

$$|1/y - x_i| = z^{i+1} + z^{i+2} + \cdots$$
$$\le \frac{1}{2^{i+1}} + \frac{1}{2^{i+2}} + \cdots$$
$$\le \frac{1}{2^i}$$

$\Rightarrow$ Sufficient to compute x_N



Reduce to calculating z^i, $i = 0 \cdots N$.

Naive: Repeated squaring $\Rightarrow \Theta(\lg^2 N)$.

Chinese Remainder Theorem

Let $p_1, p_2, \ldots, p_s$ be prime numbers.

Let $P = p_1 p_2 \cdots p_s$.

For any number Z, define the vector of residues to be

$(z_1, z_2, \ldots, z_s)$, where $0 \le i < p_i$ and $z_i = Z \bmod p_i$ $(i = 1 \sim s)$.

For each Z, $0 \le Z < P$, the vector of residues is unique.

Moreover the value of Z can be calculated from its residues

by setting

(precomputed for all Z)

$$Z = \sum_{i=1}^{s} \beta_i z_i \bmod P,$$

where

$$\beta_i = \left(\frac{P}{p_i}\right) \alpha_i$$

and

$$\alpha_i = \left(\frac{P}{p_i}\right)^{-1} \bmod p_i.$$

Represent numbers with CRT encoding

$$Z \longleftrightarrow (z_1, \ldots, z_s)$$

Example of CRT:

$p_1 = 2$
$p_2 = 3$
$p_3 = 5$
$p_4 = 7$

$P = p_1 p_2 p_3 p_4 = 210.$

$$\alpha_1 \equiv \left(\frac{210}{2}\right)^{-1} \equiv (105)^{-1} \equiv 1 \pmod 2$$

$$\alpha_2 \equiv \left(\frac{210}{3}\right)^{-1} \equiv (70)^{-1} \equiv 1 \pmod 3$$

$$\alpha_3 \equiv \left(\frac{210}{5}\right)^{-1} \equiv (42)^{-1} \equiv 3 \pmod 5$$

$$\alpha_4 \equiv \left(\frac{210}{7}\right)^{-1} \equiv (30)^{-1} \equiv 4 \pmod 7$$

$$\beta_1 = \left(\frac{210}{2}\right) \cdot 1 = 105$$

$$\beta_2 = \left(\frac{210}{3}\right) \cdot 1 = 70$$

$$\beta_3 = \left(\frac{210}{5}\right) \cdot 3 = 126$$

$$\beta_4 = \left(\frac{210}{7}\right) \cdot 4 = 120$$

Ex For any $P = 2 \cdot 3 \cdot 5 \cdot 7 = 210$, can represent any $Z < 210$.

$\beta_1 = 105$

$\beta_2 = 70$

$\beta_3 = 126$

$\beta_4 = 120$

If $Z = 132$, $(\gamma_1, \gamma_2, \gamma_3, \gamma_4) = (0, 0, 2, 6)$

$132 = 0 \cdot 105 + 0 \cdot 70 + \underbrace{2 \cdot 126}_{42} + \underbrace{6 \cdot 120}_{90} \bmod 210$

If $Z = 70$, $(\gamma_1, \gamma_2, \gamma_3, \gamma_4) = (0, 1, 0, 0)$

$70 = 1 \cdot 70$

Computing z from $(\beta_1, \beta_2, \ldots \beta_s)$.

$$z = \sum_{i=1}^{s} \beta_i z_i \bmod p$$

Taking mods:

$$\text{blah} \bmod p = \text{blah} - \left\lfloor \frac{\text{blah}}{p} \right\rfloor \cdot p$$

$\frac{1}{p}$ precomputed

Computing Z^N in CRT Notation

<< Need P big enough to represent Z^N >>

Need $P > Z^N$

$> (Z^N)^N$

$> Z^{N^2}$

Sufficient that $P = p_1 p_2 \cdots p_{N^2}$.

$$Z^N = \sum_{i=1}^{N^2} \beta_i \left(Z^N \bmod p_i \right) \bmod P$$

Calculated by computing

$z_i^N \quad (1 \le i \le s)$

$=$

$(Z \bmod p_i)^N$

Lemma: Each z_i $(1 \le i \le s)$ represented with $\Theta(\lg N)$ bits.

<< In contrast, Z represented with $\Theta(N)$ bits. >>

Pf: By Prime Number Theorem, which says

#primes $< N$ is $\Theta(N / \lg N)$.

$\Rightarrow$ our largest prime only $\Theta(N^2 / \lg N)$.

$$Z^N = \sum_{i=1}^{N^2} \beta_i \left(Z^N \bmod p_i \right) \bmod P$$

β_i: precomputed

$1/p_i$ precomputed

$1/P$ precomputed

Big Question: How to compute $Z^N \bmod p_i$??

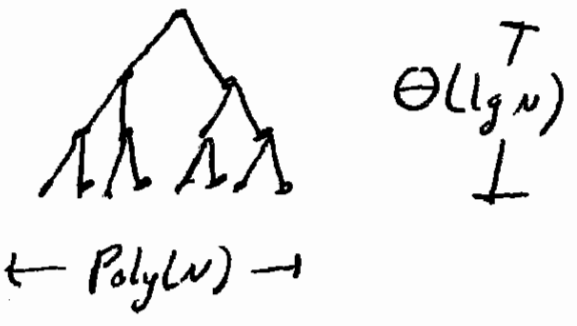
Good Answer: Since only $\Theta(\lg N)$ bits $\Rightarrow$ $\Theta(\lg N \lg\lg N)$ time.
<<But can do better!>>

Better: Lookup Tables!!!

$\forall$ p_i, $Z \bmod p_i$, precompute $(Z^N \bmod p_i)$.

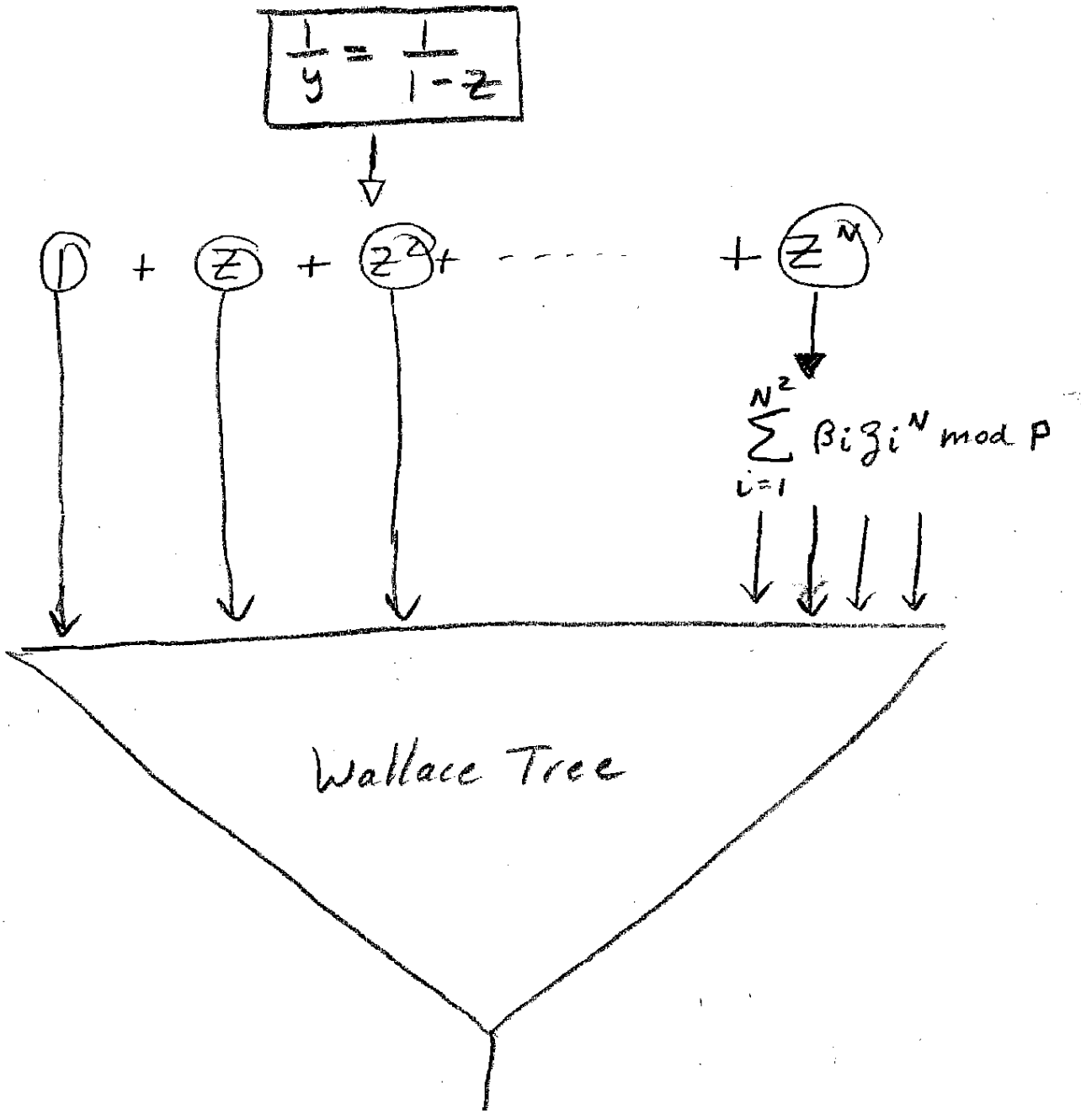
p_i: N^2 choices

$Z \bmod p_i$: $2^{\Theta(\lg N)}$ choices



$\Rightarrow O(\lg N)$ time per lookup.

Summary


$\frac{1}{y} = \frac{1}{1-z}$
$1 + z + z^2 + \cdots\cdots + z^N$
$\sum_{i=1}^{N^2} \beta_i z_i^N \bmod P$
Wallace Tree

6.896
2/18/04
L 5.1

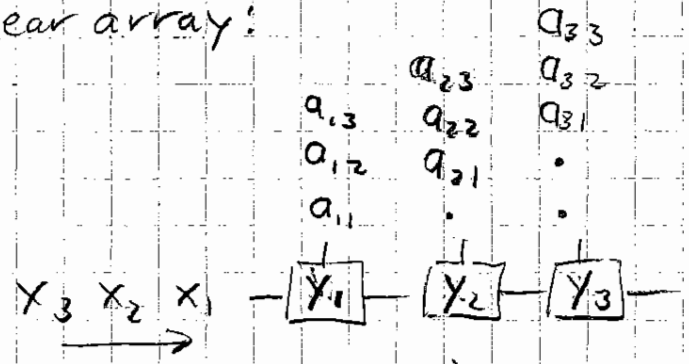
Matrix computations

- dense matrices
- mesh networks (1D & 2D)
- word model

Matrix-vector mult.

$N \times N$ matrix $A = (a_{ij})$
N-vector $x = (x_j)$
Compute N-vector $y = (y_i)$, where $y_i = \sum_{j=1}^{N} a_{ij} x_j$

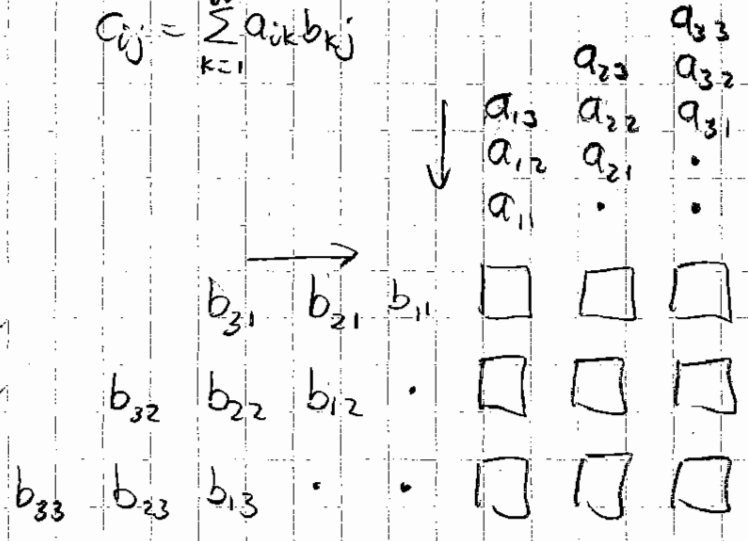
Linear array:



$\Theta(N)$ time ($2N$ steps)
$\Theta(N)$ HW

Matrix mult.

$N \times N$ matrices.
Compute $C = AB$
$c_{ij} = \sum_{k=1}^{N} a_{ik} b_{kj}$



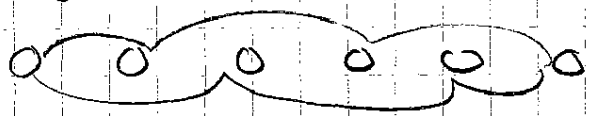
$\Theta(N)$ time ($3N$ steps)
$\Theta(N^2)$ HW

Torus: mesh + "end around"
N steps:

$a_{11}b_{11}$ $a_{23}b_{31}$ $a_{32}b_{21}$
$a_{13}b_{32}$ $a_{22}b_{22}$ $a_{31}b_{12}$
$a_{12}b_{23}$ $a_{21}b_{13}$ $a_{33}b_{33}$

6.896
2/18/04
L5.2

Simulating torus on mesh:



Each step of torus simulated by 2 mesh steps.

Gaussian elimination

$Ax = b$, solve for x (A symmetric, positive definite or irreducible, diag dominant $\Rightarrow$ pivot on diagonal)

$$\begin{pmatrix} 2 & -3 & 1 \\ 1 & -1 & -2 \\ 3 & 1 & -1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 7 \\ -2 \\ 0 \end{pmatrix}$$

Tabular method:

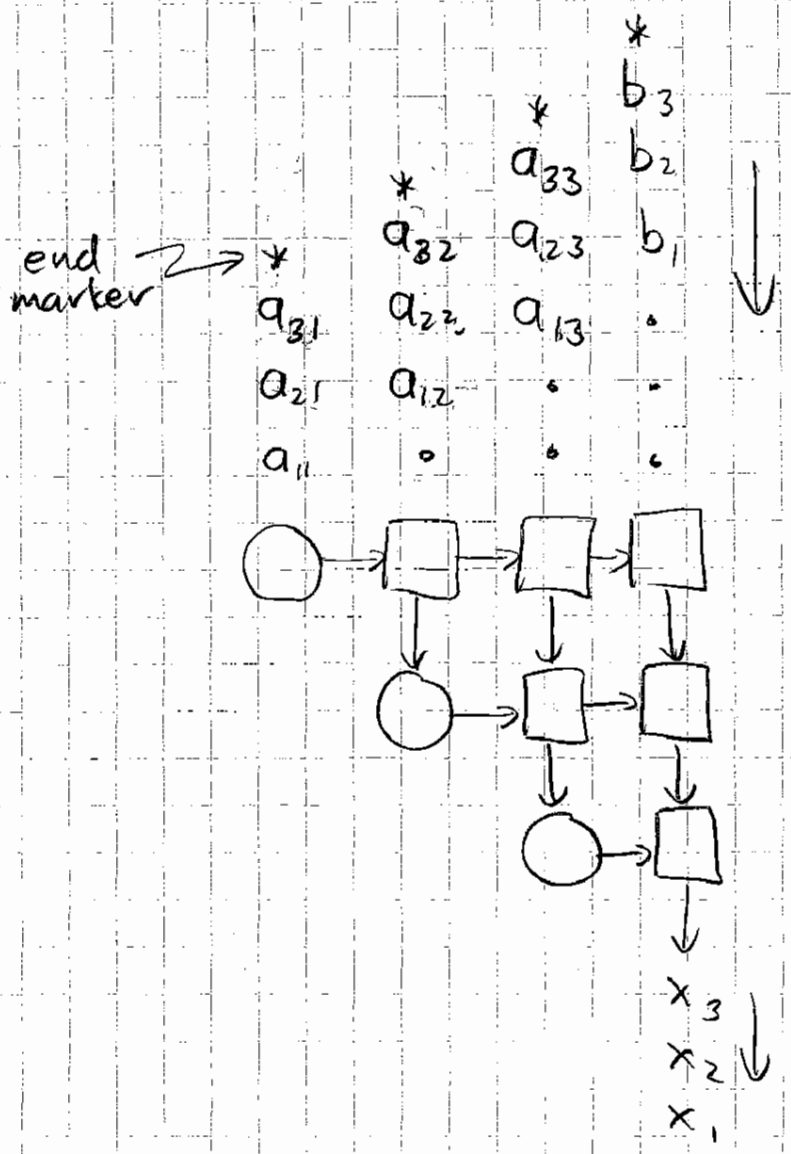
$$\left(\begin{array}{ccc|c} 2 & -3 & 1 & 7 \\ 1 & -1 & -2 & -2 \\ 3 & 1 & -1 & 0 \end{array}\right) \Rightarrow \left(\begin{array}{ccc|c} \textcircled{1} & -3/2 & 1/2 & 7/2 \\ 1 & -1 & -2 & -2 \\ 3 & 1 & -1 & 0 \end{array}\right) \Rightarrow \left(\begin{array}{ccc|c} 1 & -3/2 & 1/2 & 7/2 \\ 0 & 1/2 & -5/2 & -11/2 \\ 0 & 11/2 & -5/2 & -21/2 \end{array}\right)$$

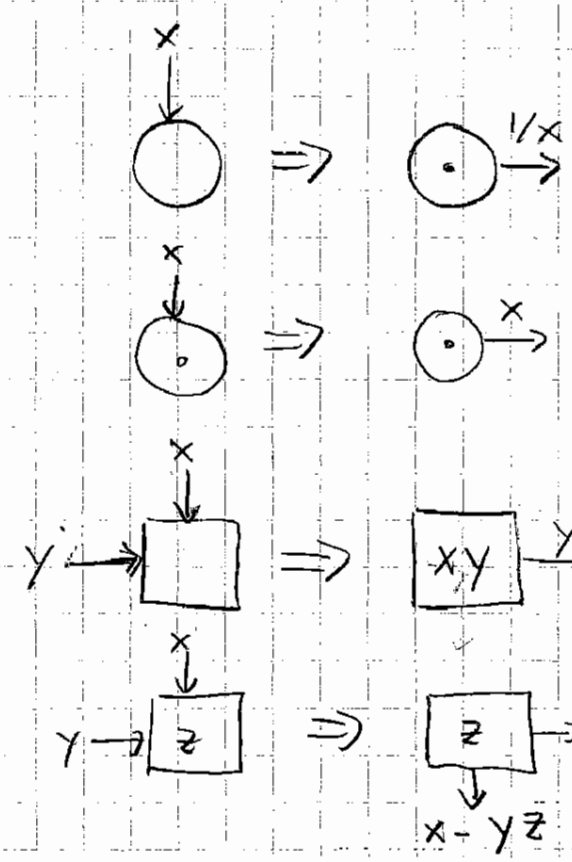
$$\Rightarrow \left(\begin{array}{ccc|c} 1 & -3/2 & 1/2 & 7/2 \\ 0 & \textcircled{1} & -5 & -11 \\ 0 & 11/2 & -5/2 & -21/2 \end{array}\right) \Rightarrow \left(\begin{array}{ccc|c} 1 & 0 & -7 & -13 \\ 0 & 1 & -5 & -11 \\ 0 & 0 & 25 & 50 \end{array}\right) \Rightarrow \left(\begin{array}{ccc|c} 1 & 0 & -7 & -13 \\ 0 & 1 & -5 & -11 \\ 0 & 0 & \textcircled{1} & 2 \end{array}\right)$$

$$\Rightarrow \left(\begin{array}{ccc|c} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 2 \end{array}\right) \qquad x = \begin{pmatrix} 1 \\ -1 \\ 2 \end{pmatrix}$$

$$a_{ij}^{(k)} = a_{ij}^{(k-1)} - \frac{a_{ik}^{(k-1)} a_{kj}^{(k-1)}}{a_{kk}^{(k-1)}}$$

6.896
2/18/04
L5.3

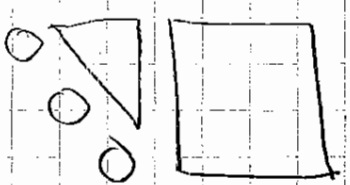




$\Theta(N)$ time
$\Theta(N^2)$ HW

Similar alg with pivoting (Leighton pp. 82-92)

Matrix inverse:

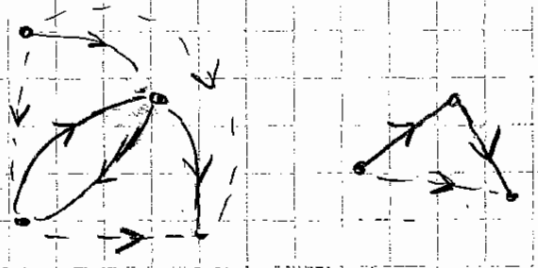


«Inverse bad numerically»

6.896
2/18/04
L5.4

Transitive closure of a digraph

Problem: Given a directed graph, for all pairs (i,j), determine if $\exists$ directed path from i to j.



Adjacency matrix

$A = (a_{ij})$ $\quad a_{ij} = \begin{cases} 1 & \text{if } i \to j \text{ is an edge} \\ 0 & \text{otherwise} \end{cases}$

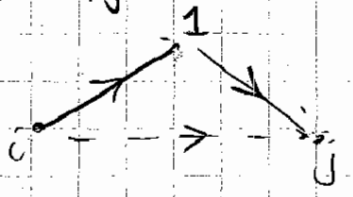
(Assume $a_{ii} = 1$ for simplicity)

Trans. closure $A^* = (a^*_{ij})$, $a^*_{ij} = 1$ iff $\exists$ path $i \overset{*}{\to} j$.

Step 1: $\forall i,j$ s.t. $a_{i1}^{(0)} = 1$ and $a_{1j}^{(0)} = 1$, set $a_{ij}^{(1)} = 1$.

I.e., $a_{ij}^{(1)} \leftarrow a_{ij}^{(0)} \vee a_{i1}^{(0)} a_{1j}^{(0)}$

Shortcut node 1:



$A = A^{(0)} \to A^{(1)}$

Step 2: $a_{ij}^{(2)} \leftarrow a_{ij}^{(1)} \vee a_{i2}^{(1)} a_{2j}^{(1)}$ $\quad A^{(1)} \to A^{(2)}$

⋮

Step k: $\boxed{a_{ij}^{(k)} \leftarrow a_{ij}^{(k-1)} \vee a_{ik}^{(k-1)} a_{kj}^{(k-1)}}$ $\quad A^{(k-1)} \to A^{(k)}$

6.896
2/18/04
L5.5

Theorem $A^{(N)} = A^*$

Proof. (Induction on k)

Claim: $a_{ij}^{(k)} = 1$ iff $\exists$ path $i \overset{*}{\Rightarrow} j$ in orig graph only going through nodes $1, 2, \ldots, k$.

($\Rightarrow$) Easy.

($\Leftarrow$) Case 1. $i \overset{*}{\Rightarrow} j$ through $1, \ldots, k-1$ $a_{ij}^{(k-1)} = 1$.
Case 2. $i \overset{*}{\Rightarrow} k$ through $1, \ldots, k-1$ $a_{ik}^{(k-1)} = 1$.
$k \overset{*}{\Rightarrow} j$ through $1, \ldots, k-1$ $a_{kj}^{(k-1)} = 1$. ☒

$\Theta(N)$ time on $N \times N$ mesh.
Idea: Same computation as G.E.:

$$a_{ij}^{(k)} = a_{ij}^{(k-1)} - \frac{a_{ik}^{(k-1)} a_{kj}^{(k-1)}}{a_{kk}^{(k-1)}}$$

Also, shortest paths:

a_{ij} = {weight of edge from i to j. / ∞ if no edge.

a_{ij}^* = weight of min-weight path from i to j. (sum edge weights.)

$$a_{ij}^{(k)} = \min\left(a_{ij}^{(k-1)}, a_{ik}^{(k-1)} + a_{kj}^{(k-1)}\right)$$

Homework: min spanning tree.

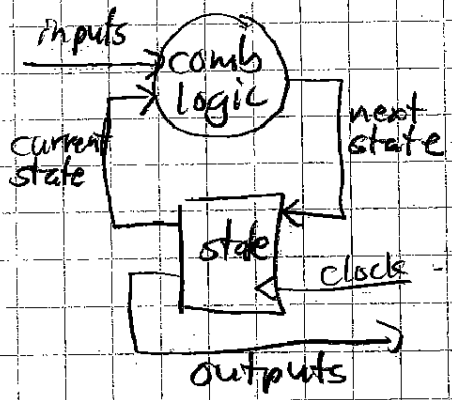
Use thm: An edge $i \to j$ with weight a_{ij} belongs to MST iff $\nexists$ path from i to j with every edge having weight $< a_{ij}$.

Taxonomy of fixed-connection networks

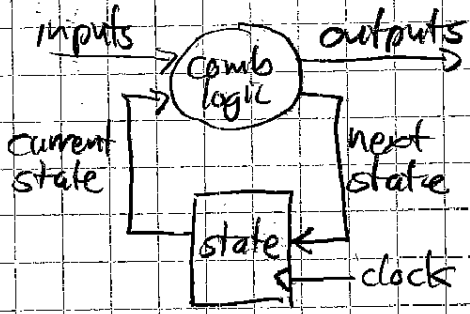
- systolic - all state and interprocessor communication is clocked.
- semisystolic - all state is clocked.
- combinational - no state, no clocks

Finite-state machines

Moore machine



Mealy machine



A systolic network is a network of Moore machines.
A semisystolic " " " " " Mealy "
(with no comb loops).

Example. Semisystolic palindrome recognizer

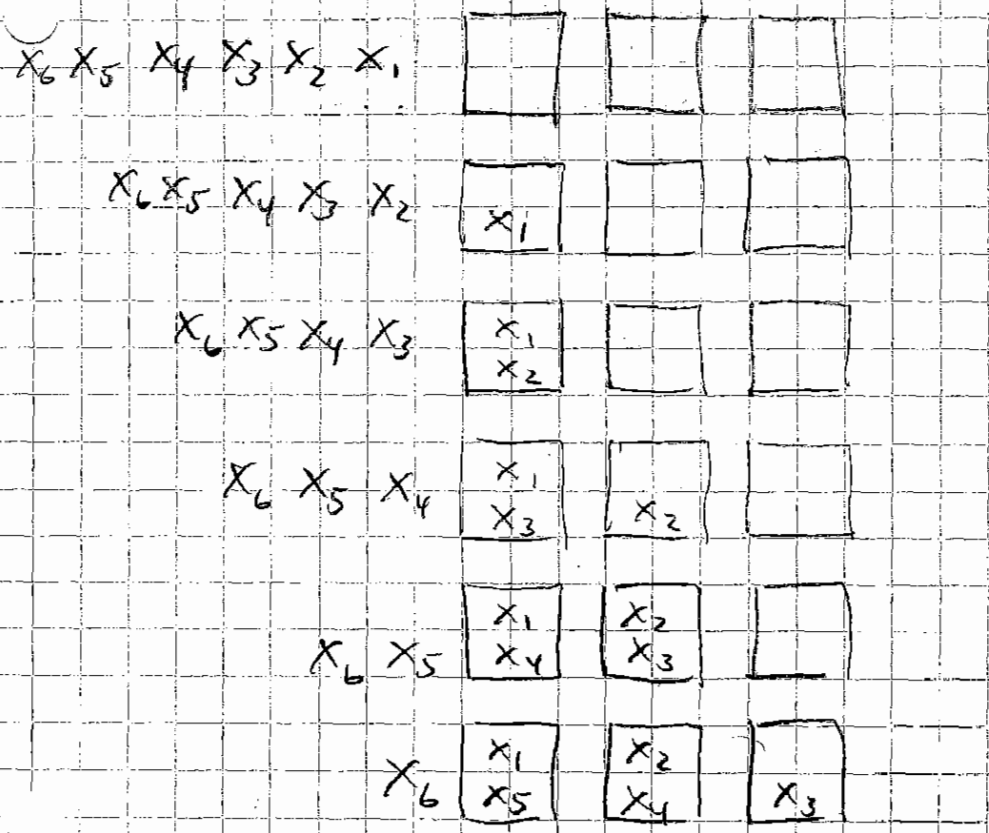
(Palindrome if $x = x^R$ ← reverse)

Takes seq. $\langle x_1, x_2, \ldots \rangle$ as input. Produces $\langle y_1, y_2, \ldots \rangle$ as output, where

$$y_i = \begin{cases} 1 & \text{if } x_1 x_2 \ldots x_i \text{ is a palindrome} \\ 0 & \text{otherwise.} \end{cases}$$

Eg. $x = \langle a, b, a, c, a, b, a \rangle$
$y = \langle 1, 0, 1, 0, 0, 0, 1 \rangle$

Semisystolic network
- systolic + global comb logic

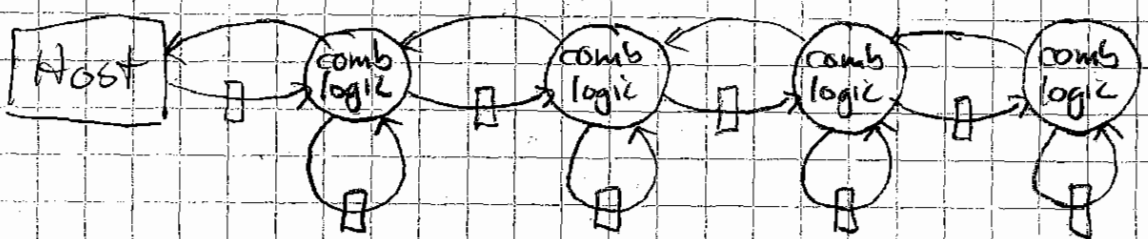


Each cell compares its two values.
Ripple a global AND from right to left.

Real time: On same tick that x_i is input, y_i is output.

Only trouble: long clock period.
$\Theta(N)$ for N-cell array.

Abstract view of circuit

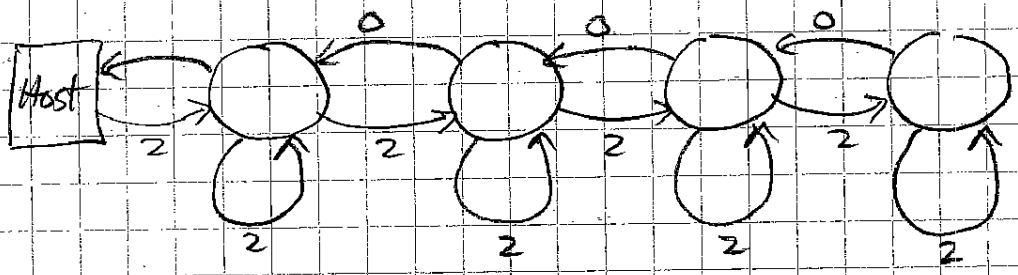


6.896
2/23/04
L6.3

Semisystolic: ≥ 0 registers on each edge → + no 0-weight cycles
Systolic ≥ 1 register on each edge

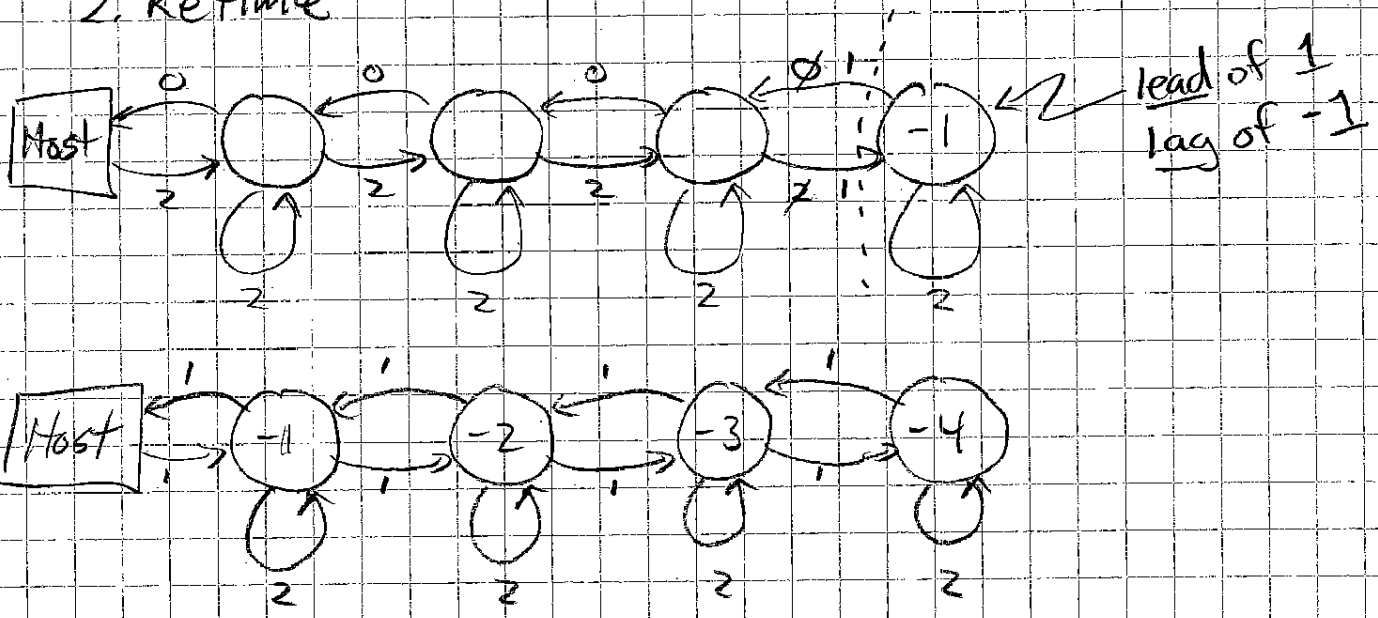
Transformation techniques

1. Slowdown: Double register counts



Same function as original semisystolic circuit, but clock twice.

2. Retime

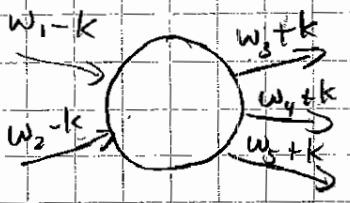


Systolic! Same functionality as doubled semisystolic circuit, but no long comb. delays.
- Initialization may differ.

Retiming a node

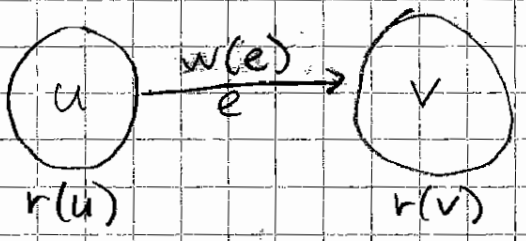


lag of $r(v) = k$



lag of $r(v) = -k$

Retimed edge weights must be ≥ 0.



$$w_r(e) = w(e) - r(u) + r(v)$$

Def. Let $G = (V, E, w)$ be edge-weighted digraph, where $w(e) \geq 0 \ \forall e \in E$. A retiming of G is a mapping $r: V \to \mathbb{Z}$. The retimed graph is $G_r = (V, E, w_r)$, where $w_r(e) = w(e) - r(u) + r(v) \ \forall e \in E$. The retiming is legal if $w_r(e) \geq 0 \ \forall e \in E$.

Note: Normally, we assume $r(\text{Host}) = 0$.

Lemma. Let $G = (V, E, w)$ be circuit, and let $r: V \to \mathbb{Z}$ be retiming. Then, for any path $u \xrightarrow{p} v$ in G, we have

$$w_r(p) = w(p) - r(u) + r(v).$$

Pf. Sup. p is $v_0 \xrightarrow{e_0} v_1 \xrightarrow{e_1} \cdots \xrightarrow{e_{k-1}} v_k$.

$$\begin{aligned} w_r(p) &= \sum_{i=0}^{k-1} w_r(e) \\ &= \sum_{i=0}^{k-1} (w(e_i) - r(v_i) + r(v_{i+1})) \\ &= \sum_{i=0}^{k-1} w(e_i) + \sum_{i=0}^{k-1} (-r(v_i) + r(v_{i+1})) \\ &= w(p) - r(v_0) + r(v_k) \quad \text{(telescope)} \end{aligned}$$ ☒

Corollary. For any cycle p, $w_r(p) = w(p)$. ☒

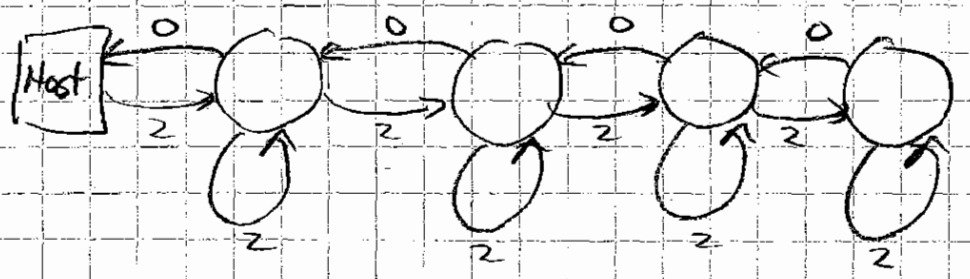
Theorem (Systolic conversion)

Let $G = (V, E, w)$ be a semisystolic circuit, and define $G^{-1} = (V, E, w')$, where $w'(e) = w(e) - 1$ $\forall e \in E$. Then, $\exists$ legal retiming $r: V \to \mathbb{Z}$ of G such that G_r is systolic iff G^{-1} has no negative-weight cycles.

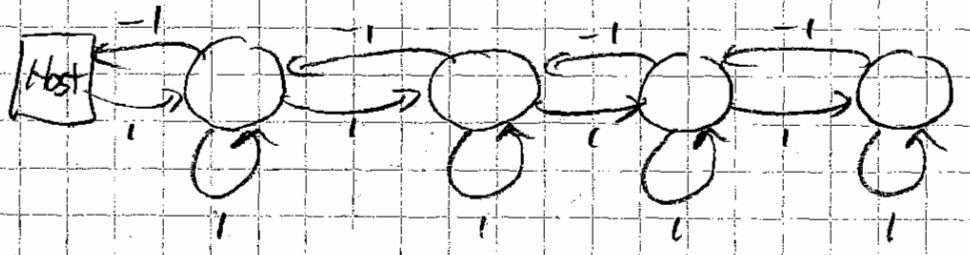
the "constraint graph"

Example

G:



G-1:



No neg-wt cycles.

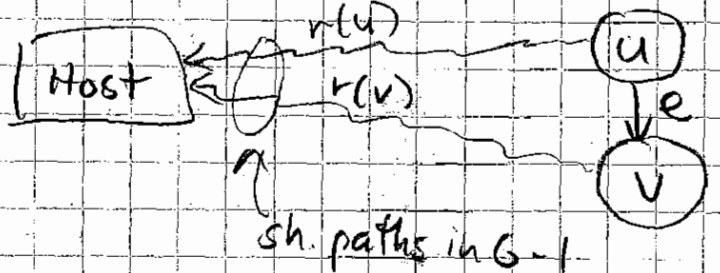
Proof.

($\Leftarrow$) Sup. G-1 has no neg-wt cycles.

Define $r(v)$ = wt of shortest path from v to host in G-1

Note: $r(v)$ defined, since G-1 has no neg-wt. cycles.

Claim: $\forall\ u \xrightarrow{e} v \in E$, $w_r(e) = w(e) - r(u) + r(v) \geq 1$.



Δ-ineq.: $r(u) \leq w(e) - 1 + r(v)$

- $r(u)$: sh. path from u in G-1
- $w(e)$: wt. of e in G-1
- $r(v)$: sh. path from v in G-1

Rewrite: $w_r(e) = w(e) - r(u) + r(v) \geq 1$.

6.896
2/23/04
L6.7

($\Leftarrow$) Sup. $G-1$ has neg.-wt cycle $v_0 \xrightarrow{e_0} v_1 \to \cdots \to v_k \xrightarrow{e_k} v_0$

Then $\sum_{i=0}^{k} (w(e_i) - 1) < 0$

$\Rightarrow \sum_{i=0}^{k} w(e_i) < k.$

But, $\sum_{i=0}^{k} w_r(e_i) \geq k$, and $\sum_{i=0}^{k} w_r(e_i) = \sum_{i=0}^{k} w(e_i)$.

Contradiction. $\boxtimes$

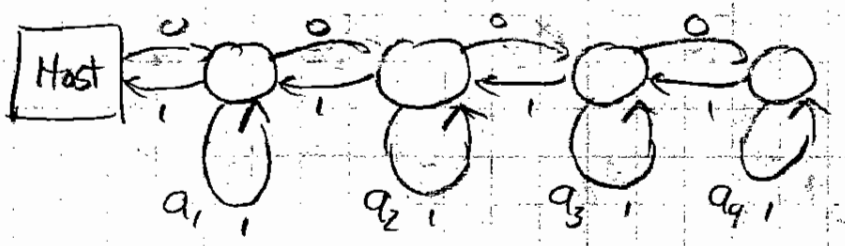
6.896
2/25/04
L7.1

Retiming

Recall Systolic Conversion Theorem: G can be retimed to be systolic if no neg-wt cycles in G-1.

Ex. Priority queue.
- Insert(x)
- Extract-Min - returns and deletes min elem.

Semisystolic design



Insert(x)
- Host broadcasts x.
- Each processor i:
  If $x \geq a_i$, do nothing
  If $x \leq a_i$, send a_i right
    if no value received from left
      then replace a_i with x
      else replace a_i with left value.

Extract-Min
- Host broadcasts "shift left"
- Each processor shifts its value left, accepts and stores value from right.

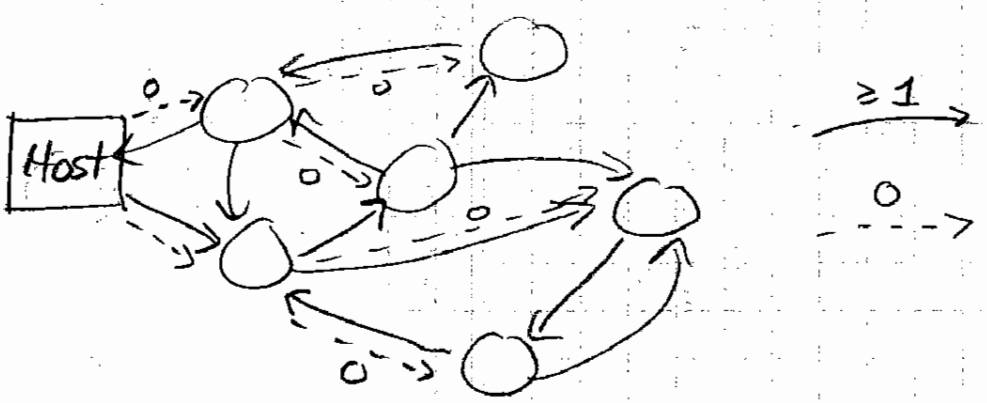
1 clock tick per operation.

2G-1 has no neg-wt cycles.
∴ 2G can be retimed to be systolic.

6.896
2/25/04
L7.2

Design methodology

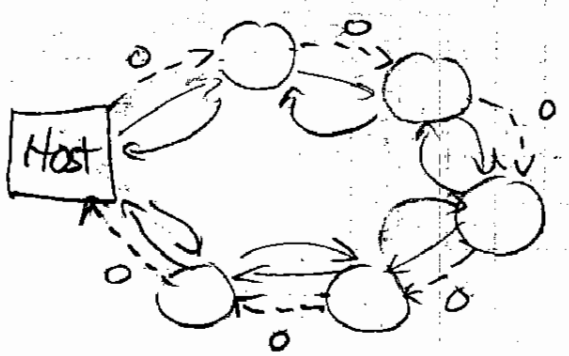
1. Build systolic circuit G to solve part of problem.
2. Augment G with global comb. logic following breadth-first spanning tree of the undirected version of G, either toward or away from host, forming G'.



3. Theorem. 2G'-1 contains no neg-wt cycles ⇒ retime 2G' to be systolic. Note: 2G' has same topology as G.

Proof. Consider cycle in 2G'-1. Each 0-wt edge in G' takes path 1 step further from host according to breadth-first distance. Each ≥1-wt edge takes path ≤1 step closer. Thus, at most ½ edges in cycle have -1 wt in 2G'-1, and rest have ≥1 wt. ☒

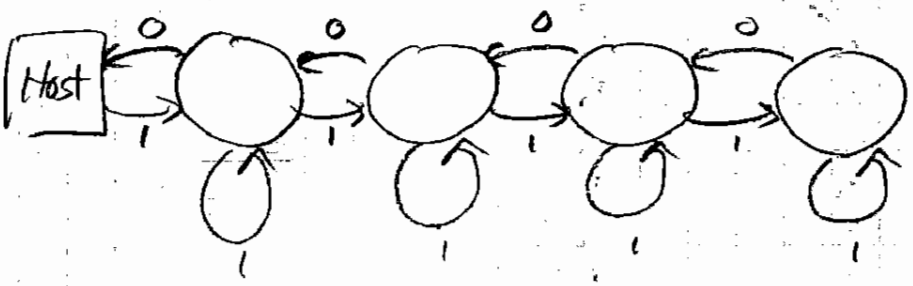
Ex. Priority queue with search



Broadcast + accumulate
- no const slowdown can be retimed to make systolic.
- must either broadcast or accumulate, not both for const. slowdown

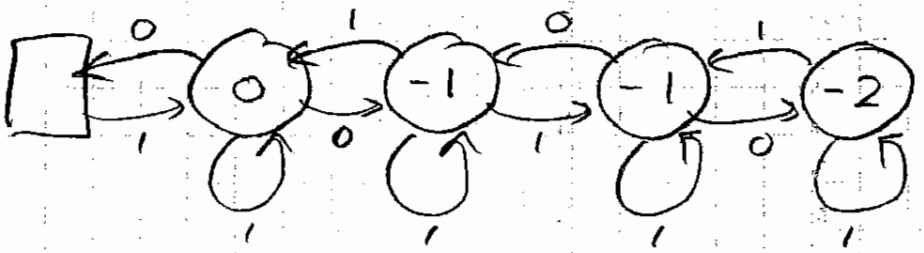
6.896
2/25/04
L7.3

Recall palindrome recognizer:



2G can be retimed to make systolic.

But, can achieve almost the same with no slowdown:



Clock period = longest path of comb. rippling.

Theorem: Retime to achieve clock period of c iff $G - 1/c$ has no neg-wt. cycles.

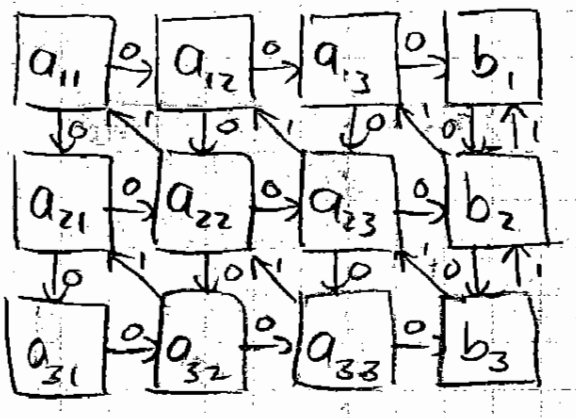
Proof. Homework ☒

6.896
2/25/04
L7.4

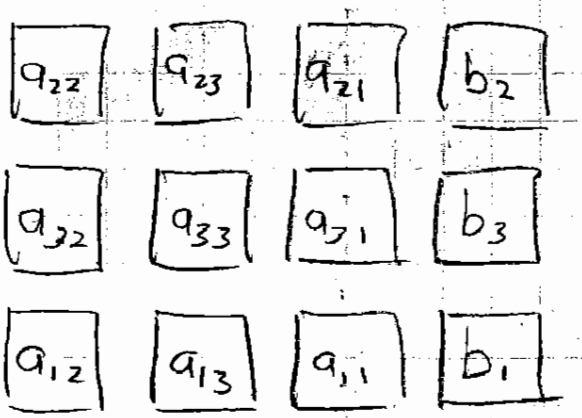
Gaussian Elimination (revisited)

$$a_{ij}^{(k)} = a_{ij}^{(k-1)} - \frac{a_{ik}^{(k-1)} \cdot a_{kj}^{(k-1)}}{a_{kk}^{(k-1)}}$$

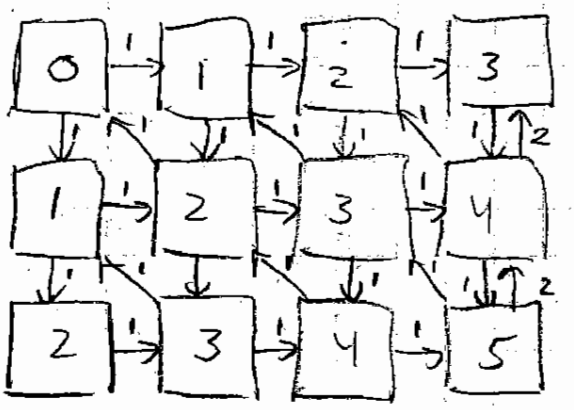
$A^{(0)}$



$A^{(1)}$



Retime 3G



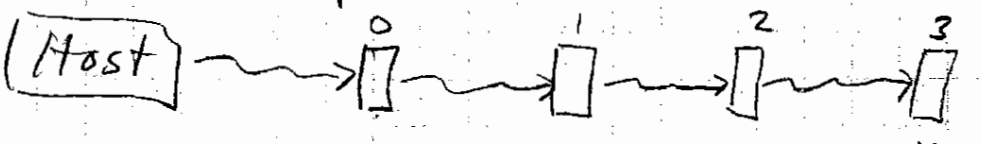
6.896
2/25/04
L7.5

Resetting

Set all state to predefined values in 1 clock tick.

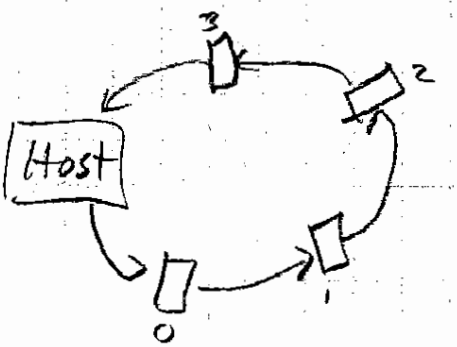
Idea 1: Broadcast, slowdown, retime
Problem: Can't use on circuits such as palindrome recognizer.

Idea 2: Sup. ≥ d latches on every path from host to latch x.
Then, after reset, x's values are fixed for d steps.



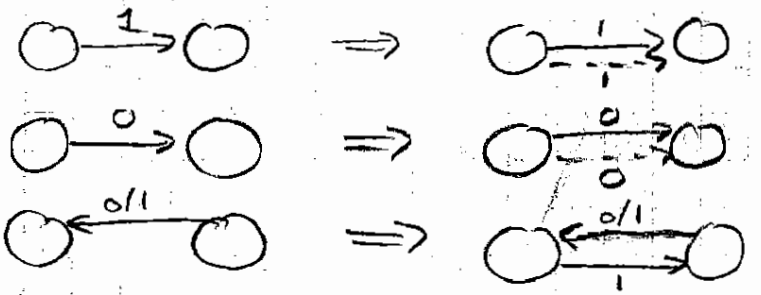
Thus, reset of x at time t_0 can be delayed up to d time steps. Then, reset to value x should have at time $t_0 + d$.

Bug!



Latch 3 doesn't get reset till time $t_0 + 3$, but old values still output

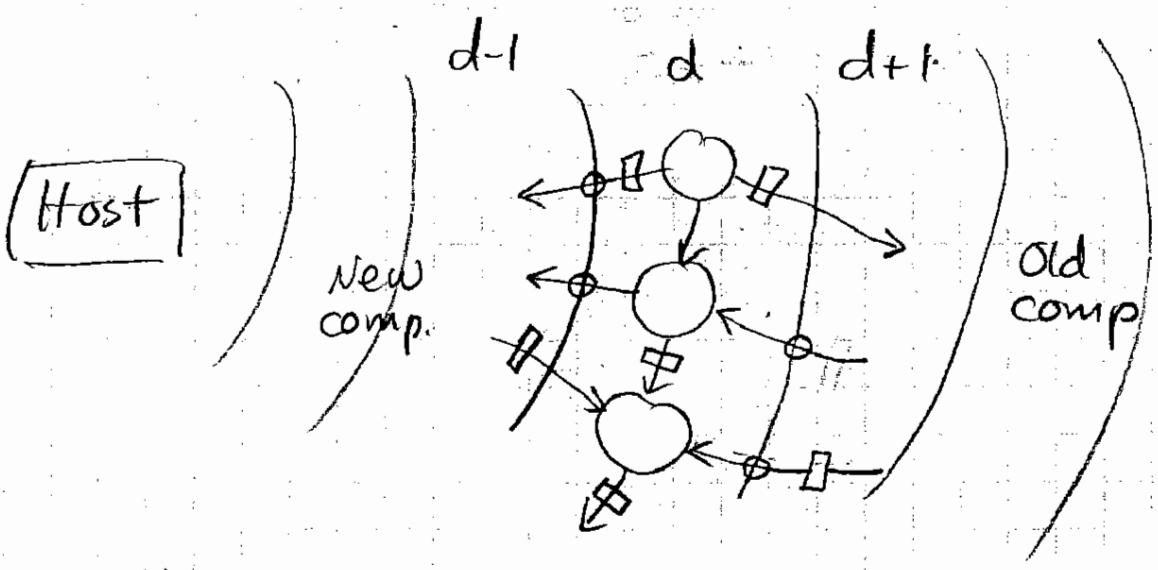
Idea 3: Propagate wavefront of resets along graph edges (WLOG, each edge weight 0 or 1):



Create shortest-path tree from host in this graph.
Reset latches at level d to precomputed value at time $t_0 + d$.

6.896
2/25/04
L7.6

Reset wavefront.



Bug! Back contamination.

Fix: At $t_0 + d + 1$, edges from level $d+1$ to level d are altered to act as if reset at time $t_0 + d$.

Note: Reset edges do not affect systolic conversion.

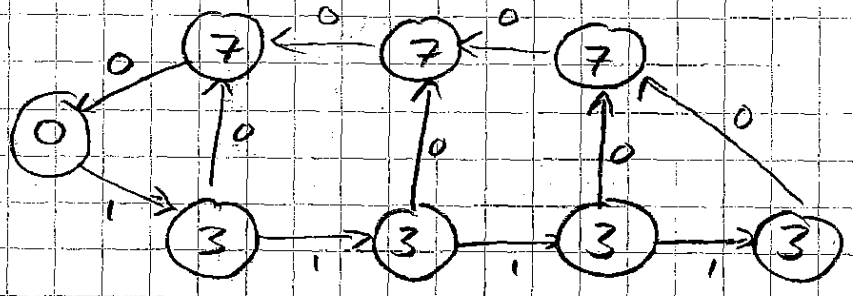
Application Palindrome recognizer with reset.

6.896
3/3/04
L8.1

Clock period

Circuit $G = (V, E, d, w)$
- V = comb elements
- E = interconnections
- d = propagation delay of comb elem
- w = latch (register) count on edge.

Ex.



Def. Path weight $p = v_0 \xrightarrow{e_0} v_1 \xrightarrow{e_1} \cdots \xrightarrow{e_{k-1}} v_k$ is

$$w(p) = \sum_{i=0}^{k-1} w(e_i)$$

Path delay

$$d(p) = \sum_{i=0}^{k} d(v_i)$$

<<Note fencepost.>>

Legality constraints

D1. $d(v) \geq 0 \quad \forall v \in V$
W1. $w(e) \geq 0 \quad \forall e \in E$
W2. In any directed cycle p of G, $\exists e \in p$ $\ni w(e) \geq 1$.

W2: synchronous circuits - no unclocked state.

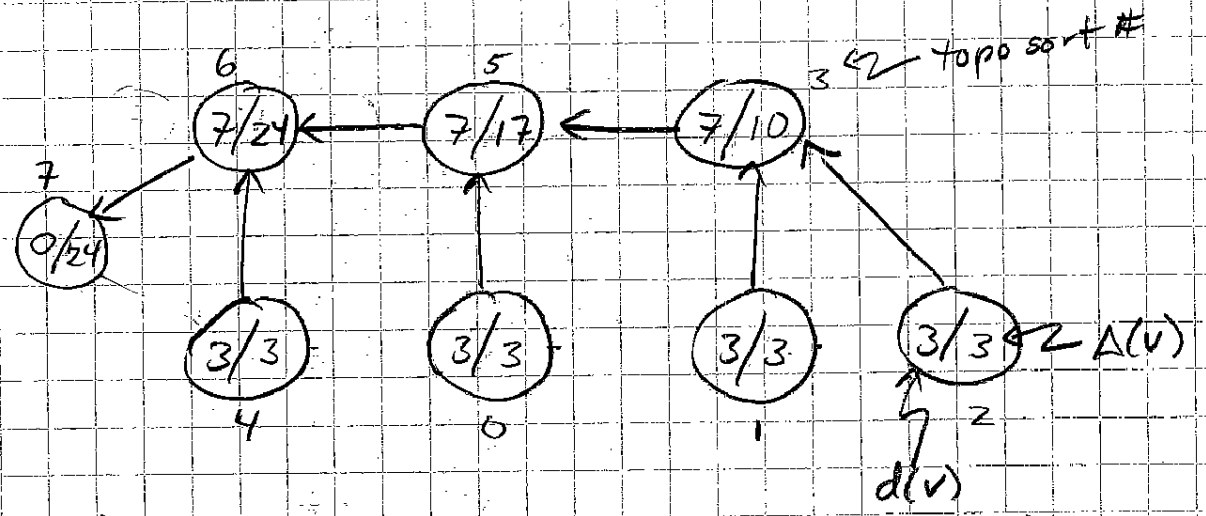
Def. The clock period of G is

$$\Phi(G) = \max_p \{d(p) : w(p) = 0\}. \quad (\text{Ex. } \Phi(G) = 24)$$

Computing clock period

1. Let G_0 be subgraph of G with only 0-wt edges. G_0 is acyclic (by WZ).
2. Topologically sort vertices of G_0.
3. Scan through vertices in topo sort order.
   a. If no incoming edge to v, set $\Delta(v) = d(v)$.
   b. Otherwise, $\Delta(v) = d(v) + \max_{u \to v \text{ in } G_0} \{\Delta(u)\}$
4. $\Phi(G) = \max_{v \in V} \Delta(v)$ «Longest path in acyclic graph»

Ex.

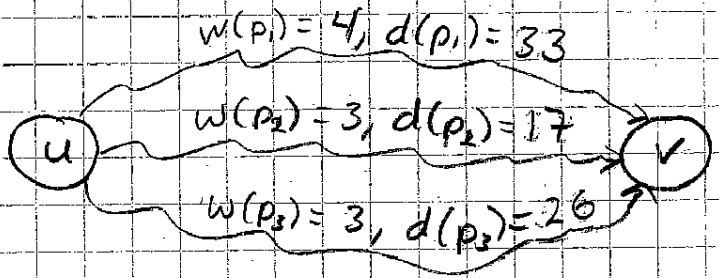


Running time: $\Theta(V+E) = \Theta(E)$ if G connected.

Retiming to minimize clock period

Recall: For path $u \overset{p}{\rightsquigarrow} v$,
$w_r(p) = w(p) - r(u) + r(v)$.

What paths might realize $\Phi(G_r)$?



Retiming reweights each path from u to v by same amount.

- Ignore p_1, since $w_r(p_1) \geq 1$ $\forall$ legal retimings.
- Ignore p_2, since $w_r(p_2) = 0 \Rightarrow w_r(p_3) = 0$ and $d(p_3) > d(p_2)$.

6.896
3/3/04
L8.3

Define $W(u,v) = \min\{w(p) : u \overset{p}{\rightsquigarrow} v\}$.

$u \overset{p}{\rightsquigarrow} v$ is a <u>critical path</u> if $w(p) = W(u,v)$.

Define $D(u,v) = \max\{d(p) : u \overset{p}{\rightsquigarrow} v \text{ and } w(p) = W(u,v)\}$.

<u>Lemma 1.</u> For any $c > 0$, $\emptyset(G) \le c$ iff whenever $D(u,v) > c$, we have $W(u,v) \ge 1$.

Proof. ($\Rightarrow$) Sup. $\emptyset(G) \le c$, and let $u,v \in V$ s.t. $D(u,v) > c$.
Thus, $\exists\ u \overset{p}{\rightsquigarrow} v$ s.t. $d(p) > c$ and $w(p) = W(u,v)$.
Must have $W(u,v) \ge 1$, or else p would be a 0-wt path with $d(p) > c$. Contradiction.

($\Leftarrow$) Sup. $\forall u,v \in V$, $D(u,v) > c \Rightarrow W(u,v) \ge 1$.
Let $u \overset{p}{\rightsquigarrow} v$ be 0-wt path. Then, $W(u,v) = w(p) = 0$,
which implies $d(p) \le D(u,v) \le c$. ⊠

<u>Computing W and D</u>

1. Create graph with edge weights of $u \to v$ of $\langle w(e), -d(u)\rangle$
2. All-pairs sh. paths ((Floyd-Washall $O(V^3)$
   Johnson + Fib heaps $O(VE + V^2 \lg V)$
3. For weight $\langle x, y\rangle$ betw. u and v,
   $W(u,v) = x$
   $D(u,v) = d(v) - y$

<< Next time: problem ---- >>

6.896
3/8/04
L9.1

Optimal retiming

Recall: $W(u,v) = \min\{w(p) : u \overset{p}{\rightsquigarrow} v\}$
$D(u,v) = \max\{d(p) : u \overset{p}{\rightsquigarrow} v \text{ is a critical path}\}$

Lemma 1. $\Phi(G) \le c$ iff $W(u,v) \ge 1$ whenever $D(u,v) > c$. ☒

Lemma 2. Let r be legal retiming of G.

1. p is a crit path of G iff p is a crit path of G_r.
2. $W_r(u,v) = W(u,v) - r(u) + r(v)$
3. $D_r(u,v) = D(u,v)$. ☒

Lemma 3. $\Phi(G_r) = D(u,v)$ for some $u, v \in V$.

Proof. Let $u \overset{p}{\rightsquigarrow} v$ be path in G_r $\$$ $\Phi(G_r) = d(p)$ and $w_r(p) = 0$ (def of clock period). Thus, $W_r(u,v) = w_r(p) = 0$, and $D_r(u,v) = d(p)$, since no 0-wt path in G_r has larger delay than p. Thus, $\Phi(G_r) = D_r(u,v) = D(u,v)$. ☒

Theorem. Let $r: V \to \mathbb{Z}$. Then, r is a legal retiming of G $\$$ $\Phi(G_r) \le c$ iff
1. $r(u) - r(v) \le w(e)$ $\forall u \overset{e}{\to} v$.
2. $r(u) - r(v) \le W(u,v) - 1$ $\forall u, v$ $\$$ $D(u,v) > c$.

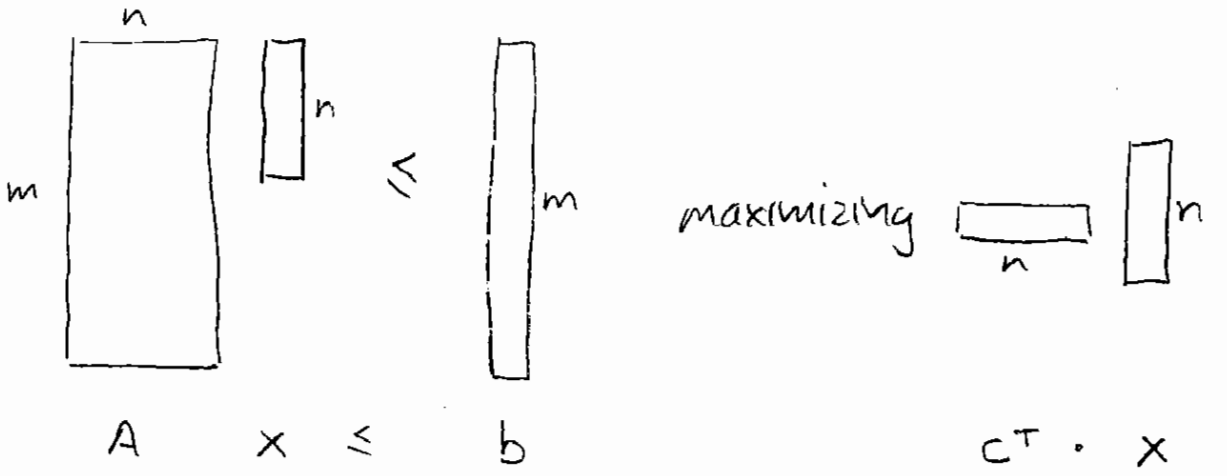
Proof.
1. $r(u) - r(v) \le w(e)$ iff $w_r(e) = w(e) - r(u) + r(v) \ge 0$.
2. $\Phi(G_r) \le c$ precisely when $W_r(u,v) \ge 1$ $\forall u, v \in V$ $\$$ $D_r(u,v) > c$, by Lemma 1. Rewrite using Lemma 2. ☒

$O(E)$ constraints of type 1. $O(V^2)$ " " " 2. } Linear!

6.896
3/8/04
L9.2

Linear programming

Let A be an $m \times n$ matrix, b be an m-vector, and c be an n-vector. Find an n-vector x that maximizes $c^T x$ subject to $Ax \leq b$, or determine no solution exists.



General algs
- simplex - practical, but w-c exp. time
- interior-pt algs - poly time, becoming practical.

"Feasibility" problem: No opt. criterion.
Find x s.t. $Ax \leq b$.

Difference constraints

Each row of A contains exactly one 1 and one -1, and rest are 0's.

Ex.
$$\left.\begin{array}{l} x_1 - x_2 \leq 3 \\ x_2 - x_3 \leq -2 \\ x_1 - x_3 \leq 2 \end{array}\right\} \; x_j - x_i \leq a_{ij}$$

Solution
$x_1 = 3$
$x_2 = 0$
$x_3 = 2$

Linear prog., but simpler.

6.896
3/8/04
L9.3

Constraint graph

- vertex v_i for each unknown x_i
- edge $v_i \longrightarrow v_j$ with weight a_{ij} if $x_j - x_i \le a_{ij}$ is constraint.

Thm. If constraint graph has neg-wt cycle, then no solution. (Constraints unsatisfiable).

Pf. Sup. cycle is $v_1 \to v_2 \to \cdots \to v_k \to v_1$.

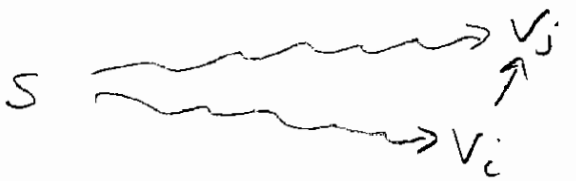
Then,
$$\begin{aligned} x_2 - x_1 &\le a_{12} \\ x_3 - x_2 &\le a_{23} \\ &\vdots \\ x_k - x_{k-1} &\le a_{k-1,k} \\ x_1 - x_k &\le a_{k1} \end{aligned}$$
$$0 \le \text{wt of cycle} < 0$$

∴ No values for x_i satisfy constraints. ☒

Thm. No neg-wt cycle ⇒ constraints satisfiable.

Pf. Add new vertex s to V with 0-wt edge to each $v_i \in V$. (No neg-wt cycle introduced).

Let $\delta(s, v_i)$ = wt of sh. path from s to v_i. (Sh. paths exist, since no neg-wt cycle)
Claim: $x_i = \delta(s, v_i)$ is solution.



$\delta(s, v_j) \le \delta(s, v_i) + a_{ij}$ (Δ-ineq.)
x_j x_i ⇒ $x_j - x_i \le a_{ij}$ ☒

6.896
3/8/04
L9.4

Bellman-Ford algorithm

Sh. path from source $s \in V$ to all $v \in V$ or determine neg-wt cycle exists.

Init: $d[v] = \begin{cases} 0 & \text{if } v = s \\ \infty & \text{otherwise.} \end{cases}$

for $i \leftarrow 1$ to $|V|-1$
  do for each edge $(u,v) \in E$
    do $d[v] \leftarrow \min\{d[v], d(u) + w(u,v)\}$.

if $\exists\ (u,v)$ \$ $d[v] > d[u] + w(u,v)$
  then neg-wt cycle exists.

No neg-wt cycle $\Rightarrow$ $d[v] = \delta(s,v)$.

Correctness: induction (see CLRS)
Running time: $O(VE)$

Opt. clock period

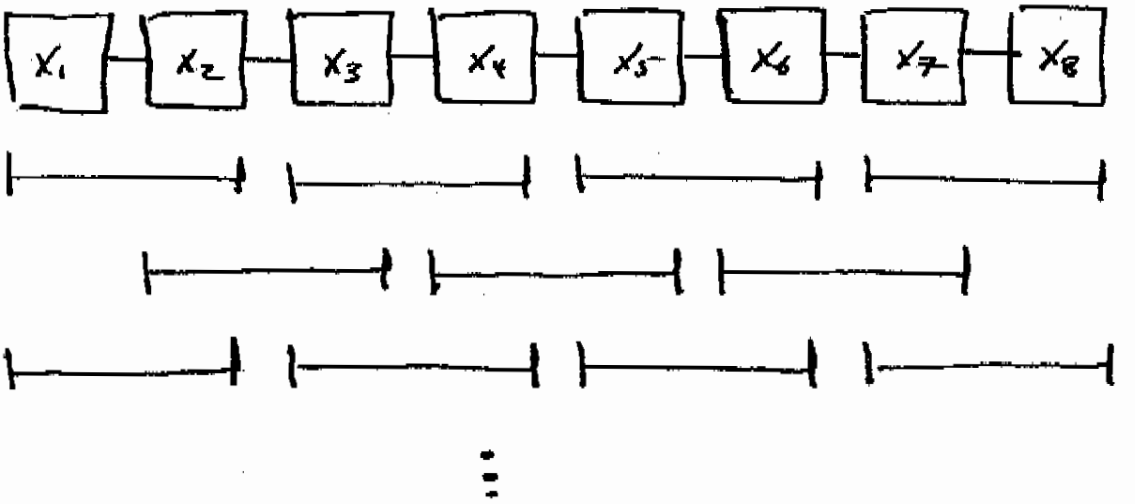
1. Compute W and D — $O(V^3)$
2. Sort elems of D (clock period is one of them) — $O(V^2 \lg V)$
3. Binary search among D elems using Bellman-Ford to test feasibility of LP — $O(V^3 \lg V)$
4. Use values found by B-F to retime.

<< Reminder: problem session next time >>

Mon 3/15/04
Michael Bender Lecturing

Sorting on 1 and 2D Arrays

Linear array: odd-even transposition sort.



~~This: odd-even trans sort runs in N steps (with 1 of opt).~~

Def: Oblivious comparison-exchange alg. Comparisons prespecified. Independent of results of prev comparisons.
<<eg. quicksort not oblivious>>

Thm: If an obliv comparison-exchange alg sorts all 2^N sequences of 0's and 1's, it sorts all sequences of arbitrary #s

3/15/04

Proof: In 2 parts:

(1) Let f be monotonically increasing function.

Then $\min\{f(x), f(y)\} = f(\min\{x, y\})$

$\max\{f(x), f(y)\} = f(\max\{x, y\})$.

By induction on timesteps, if alg transforms

$\langle a_1, x_2, \ldots, a_N \rangle \rightarrow \langle b_1, b_2, \ldots, b_N \rangle$,

then it transforms

$\langle f(a_1), f(a_2), \ldots, f(a_N) \rangle \rightarrow \langle f(b_1), f(b_2), \ldots, f(b_N) \rangle$

<<see CLR>>

(2) Suppose false. Ie, network sorts all 0-1 seq, but $\exists \langle a_1, a_2, \ldots, a_N \rangle$ st $a_i < a_j$, but a_i comes after a_j in output.

Define $f(x) = \begin{cases} 0, & \text{if } x \le a_i \\ 1, & \text{if } x > a_i \end{cases}$.

But network fails to sort $\langle f(a_1), f(a_2), \ldots, f(a_N) \rangle$

Contradiction. ∎

<<threshold intuition>>

$\Rightarrow$ Need only construct 0-1 sorting algs!

3/15/04

Thm: Odd-even transposition sort runs in N steps (with 1 of OPT).

<< Result less interesting than proof method >>

Pf: Consider movement of rightmost 1.

1st step: may not move 0 1

During subsequent steps, moves forward.

⇒ cannot block other 1s.

⇒ k^{th} leftmost 1 begins moving by step: $k+1$.
Must reach position $N-k+1$.

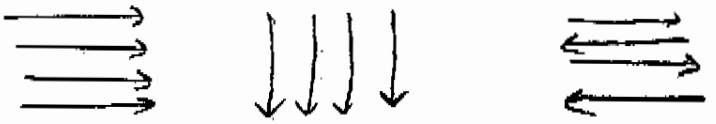
⇒ All elmts in final position by time N

3/15/04

Sorting on 2D Grid

Lower bounds: $2\sqrt{N} - 2$ (diameter)
$\Omega(\sqrt{N})$ (bisection)

Natural Grid Orders:

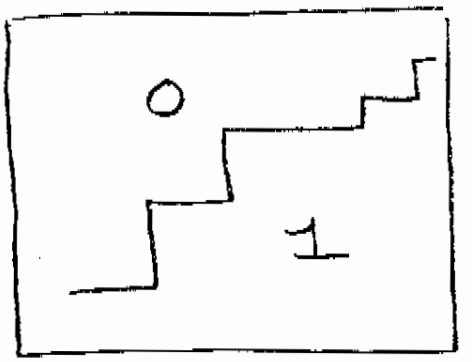


"Broken" Alg:

Repeat:

(1) ↓↓↓↓

(2) ⇉



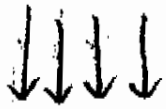
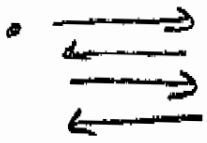
Doesn't yield unique order:

$$\begin{matrix} 0 & 1 \\ 1 & 0 \end{matrix} \Rightarrow \begin{cases} \begin{matrix} 0 & 0 \\ 1 & 1 \end{matrix} & \downarrow\downarrow \rightrightarrows \\ \begin{matrix} 0 & 1 \\ 0 & 1 \end{matrix} & \rightrightarrows \downarrow\downarrow . \end{cases}$$

3/15/04

Shearsort

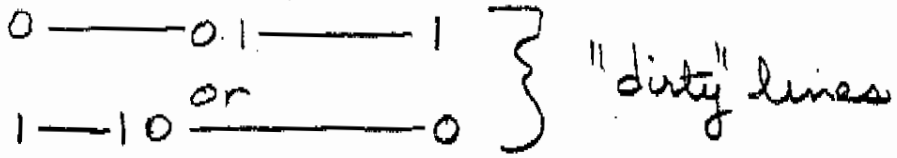
Repeat

- 
- 

Thm: Shearsort produces unique sorting order after time $O(\sqrt{N} \lg N)$. Eg, $O(\lg N)$ phases sufficient to sort.

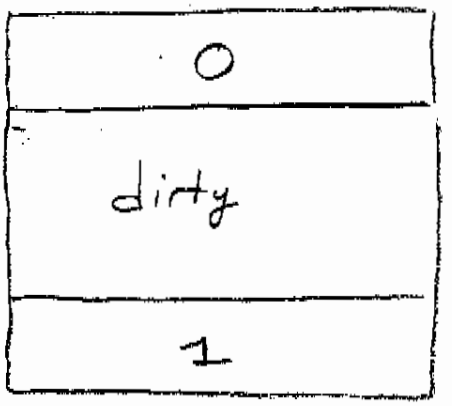
Pf: Apply 0-1 lemma.

Def: 



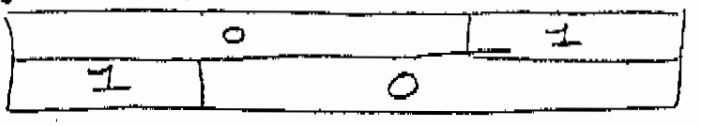
Claim: After each phase, # dirty rows decreases by at least half.

Grid has 3 regions:

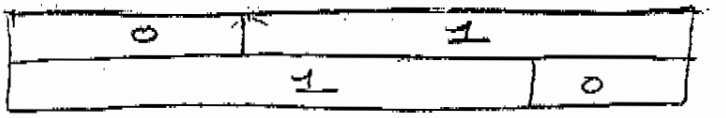


Divide dirty into pairs of rows:

either:

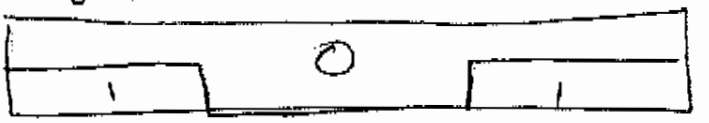


or:

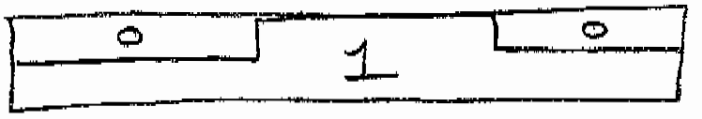


After sorting columns:

either



or

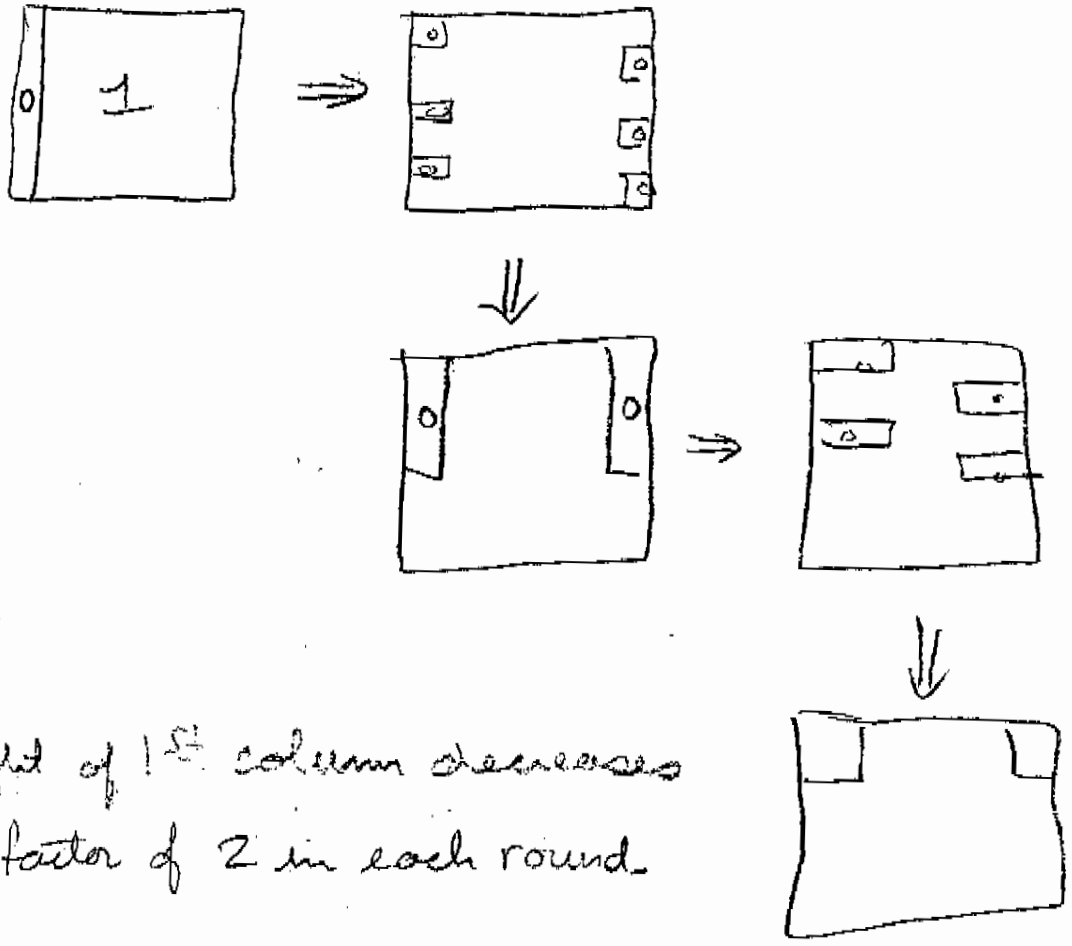


$\Rightarrow$ dirty region decreases by $\geq 1/2$.

$\Rightarrow$ after $\lg N$ phases [$\Theta(\sqrt{N} \lg N)$ time]
all sorted.

Lemma: Shearsort runs in $\Omega(\lg N)$ phases.

Pf: Bad example 0's in 1st column.



height of 1st column decreases by factor of 2 in each round.

Average Case:

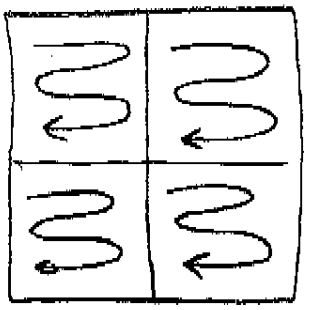
Substitute 0's for $\sqrt{N}$ smallest elm's

- row sort first: $E[\#0\text{'s in } 1^{st} \text{ column}] = \Theta(N)$. ..
- column sort first: not true.
  best LB = $\Omega(N \lg\lg N)$.

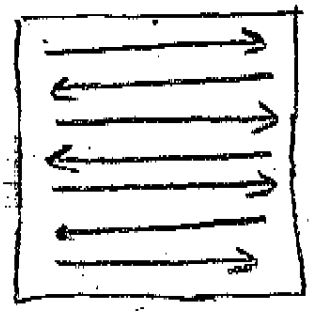
$O(\sqrt{N})$ Algorithm ($\leq 8\sqrt{N}$)

Assume $\sqrt{N}$ is power of 2

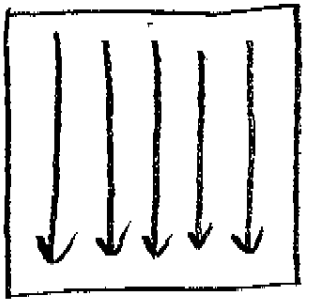
(1) Recursively sort each quadrant in snake order



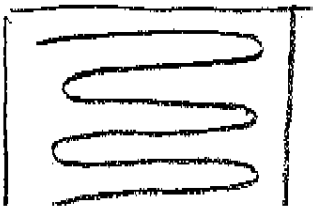
(2) Sort rows in alternate order



(3) Sort columns



(4) Do $2\sqrt{N}$ steps of 1D odd-even transposition on overall snake order.



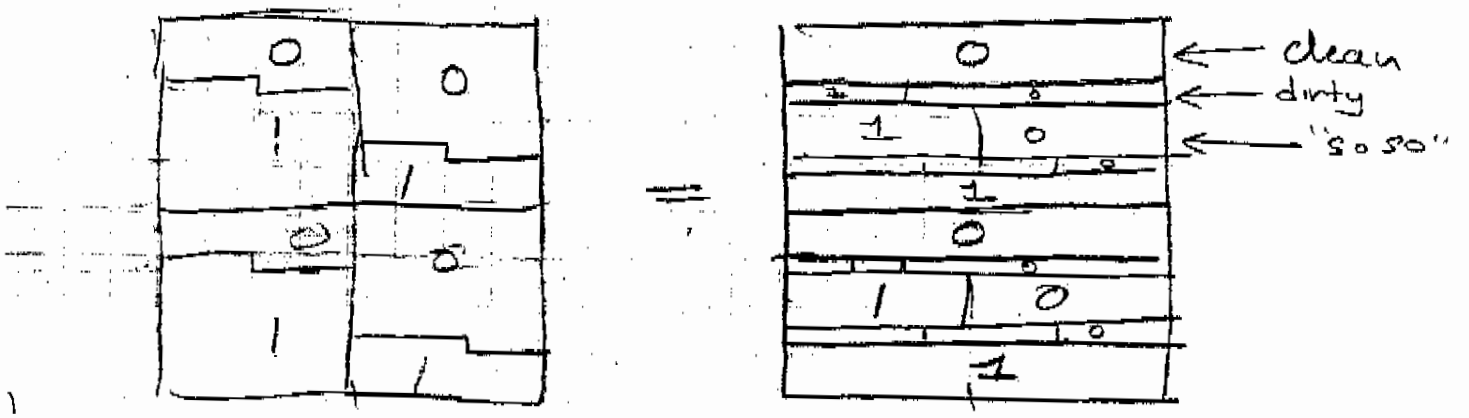
Running time:

$$T(N) \leq T(N/4) + \sqrt{N} + \sqrt{N} + 2\sqrt{N}$$

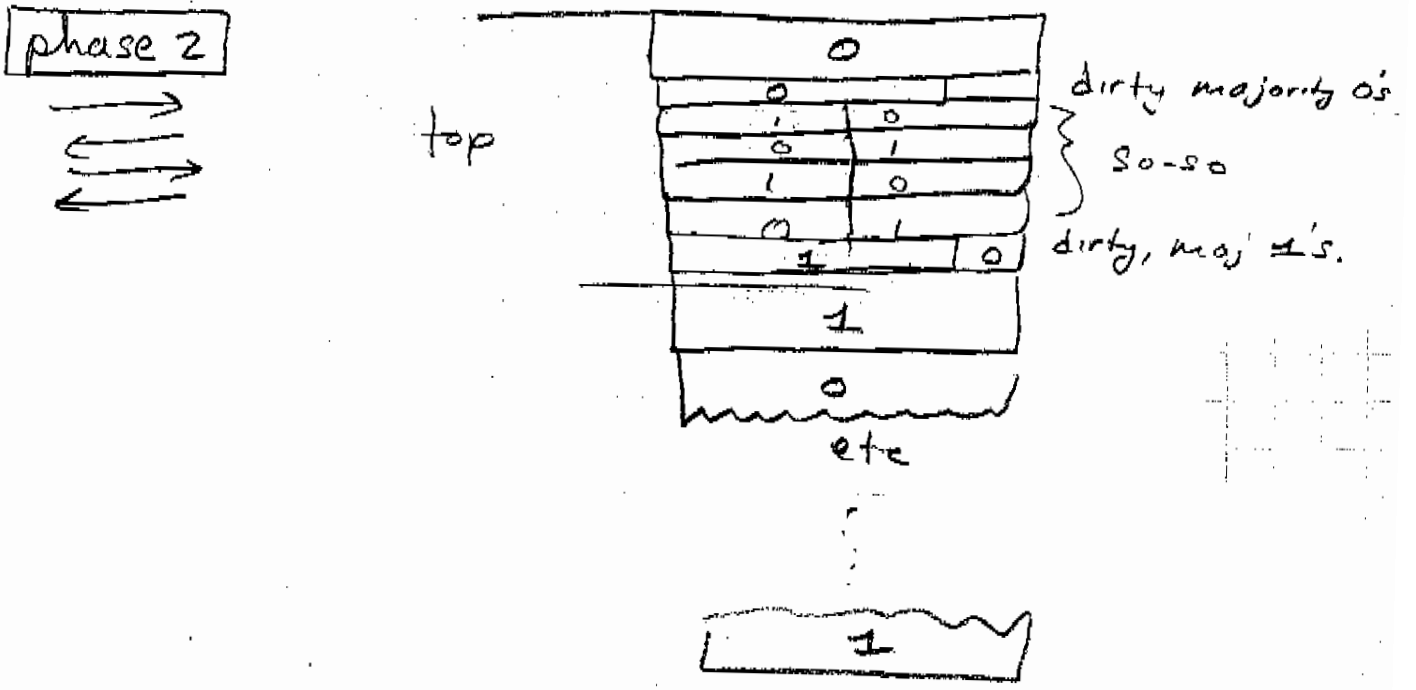
$$\leq 8\sqrt{N}$$

Proof of Correctness:

Phase 1 In each quadrant at most on of rows is dirty and rest are clean:



phase 2

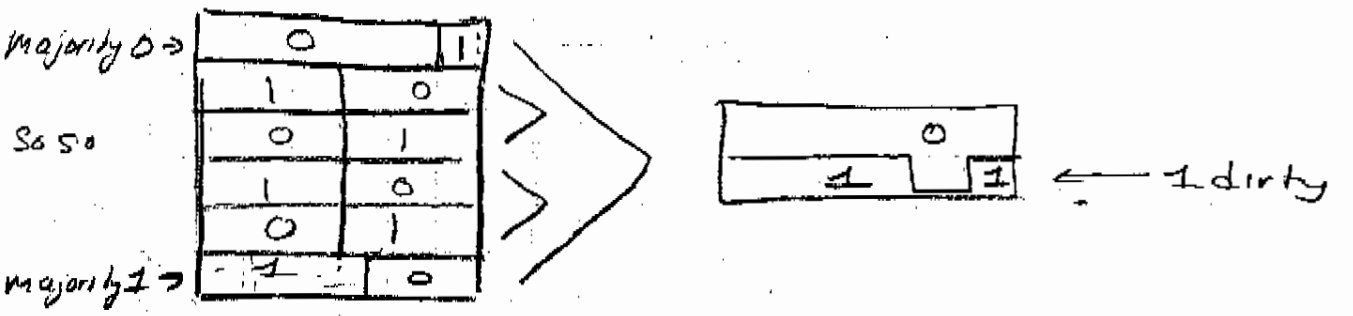


phase 3 ↓↓↓↓

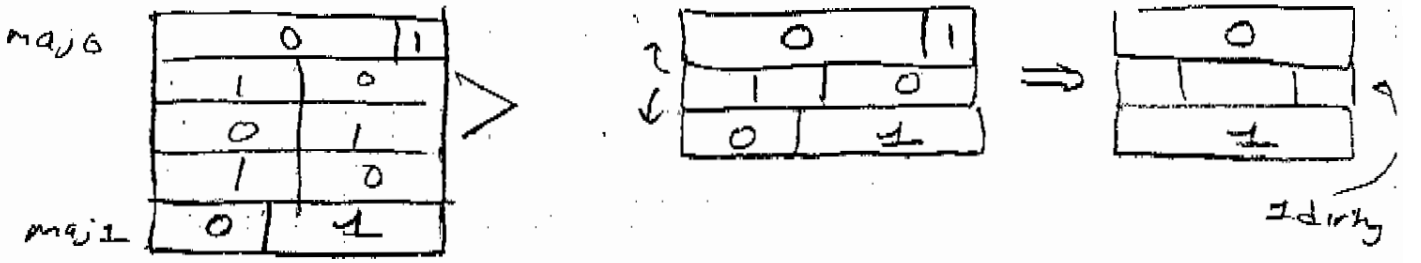
clm: ≤ 1 dirty row in top half &
" " " " bottom half.

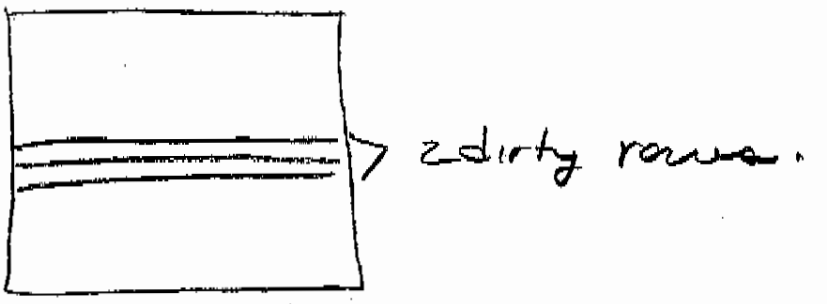
<< gilding the lily - we already have a $\Theta(\sqrt{N})$ alg >>

Pf: Case 1: # so-so rows even:



Case 2: # so-so rows odd:

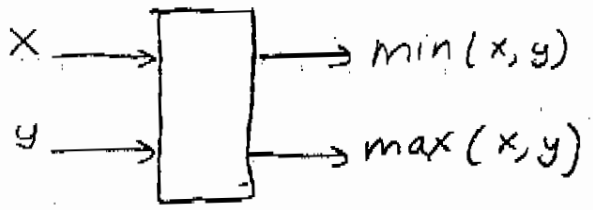




⇒ Sorted after phase 4 !

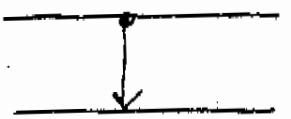
Comparison Networks

Wed 3/17
Michael Bender Lecturing

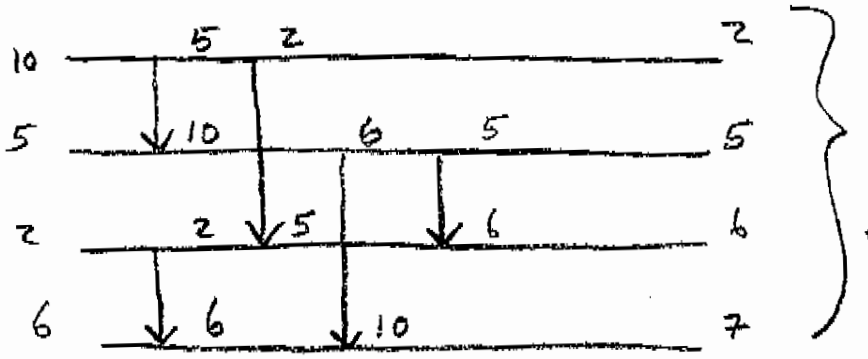


Comparitor

Notation:

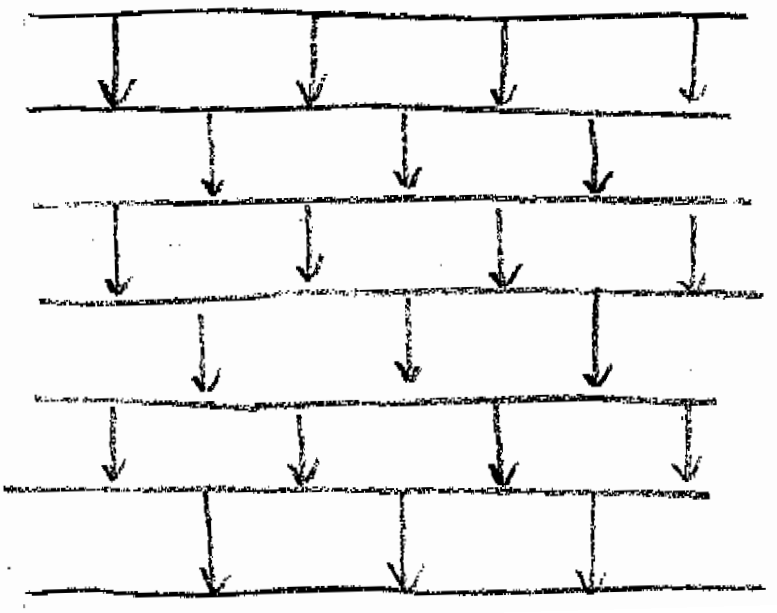


Sorting Network [developed in 50s]



Running time = depth = longest path of comparitors (=3)

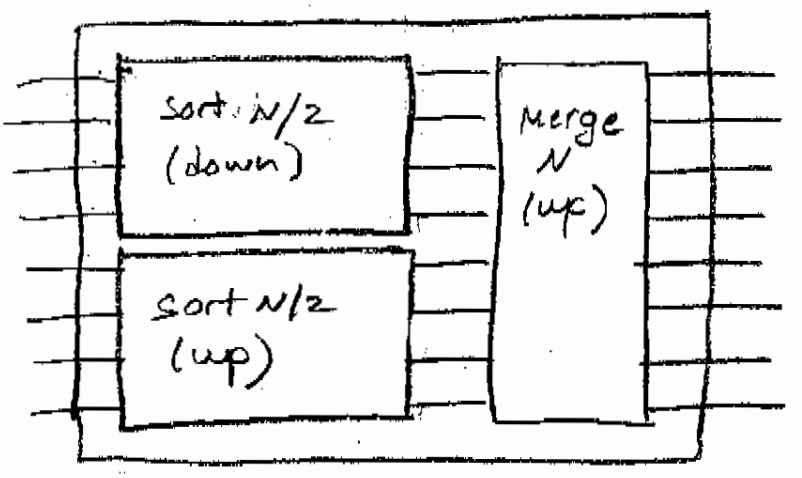
Odd-Even Transposition Sort



Depth = N

<< How low can you go? >>

step 3 — Sorting network = mergesort [Batcher]



merge

Depth $D(N) = D(N/2) + \lg N$

$= \Theta(\lg^2 N)$

Size $S(N) = 2S(N/2) + \Theta(N \lg N)$

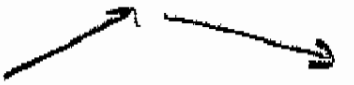
$= \Theta(N \lg^2 N)$

Example:

Bitonic Sorting Network (Batcher)

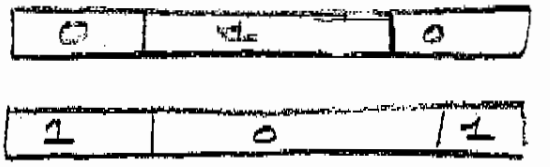
Step 1 Sort "bitonic" sequence

def: A bitonic sequence:

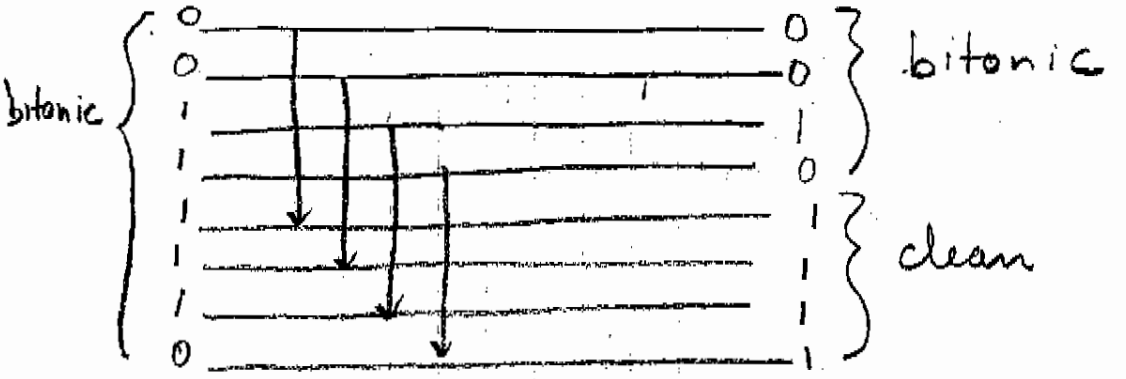


or cyclic rotation

⇒ 0-1 bitonic sequence:



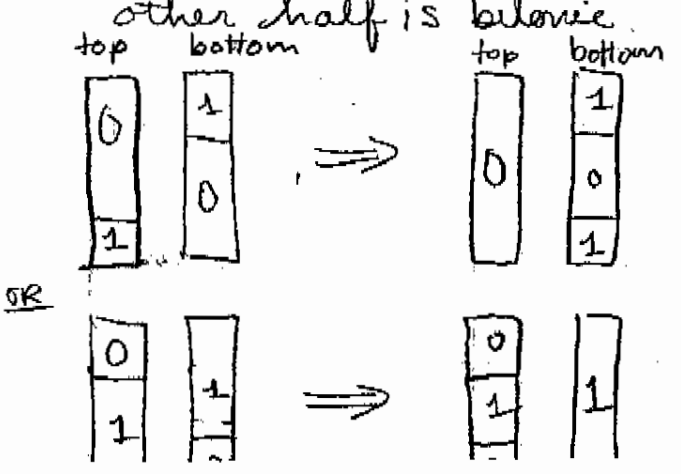
key subnetwork: half cleaner



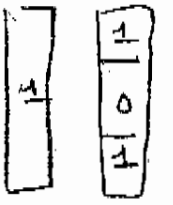
claim: half output is clean (& bitonic)

other half is bitonic

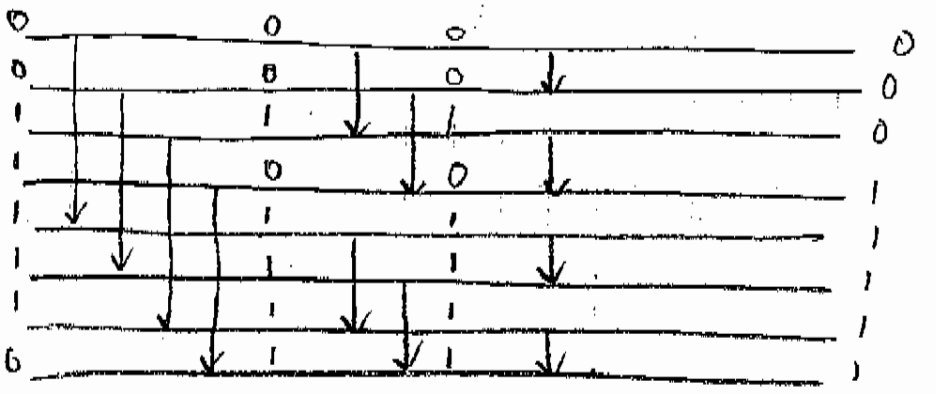
Pf:



<< other case >>



Sort bitonic sequence



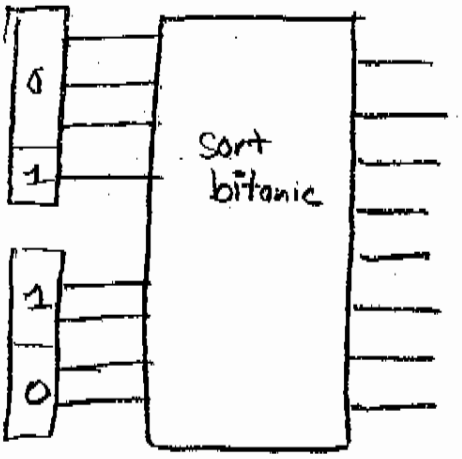
Depth: $D(N) = D(N/2) + 1$

$= \lg N$

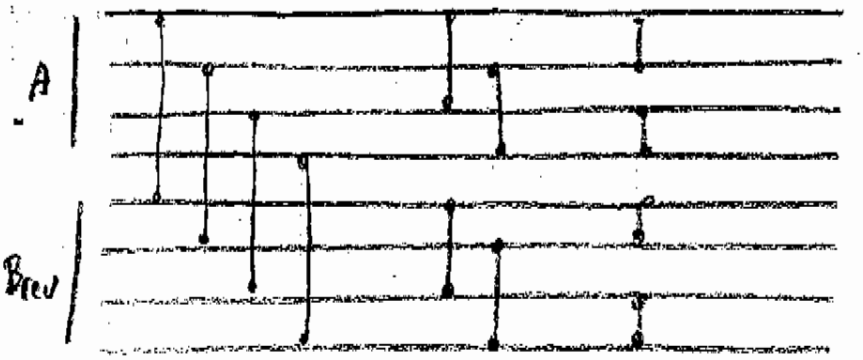
Size: $S(N) = 2S(N/2) + N/2$

$= \Theta(N \lg N)$.

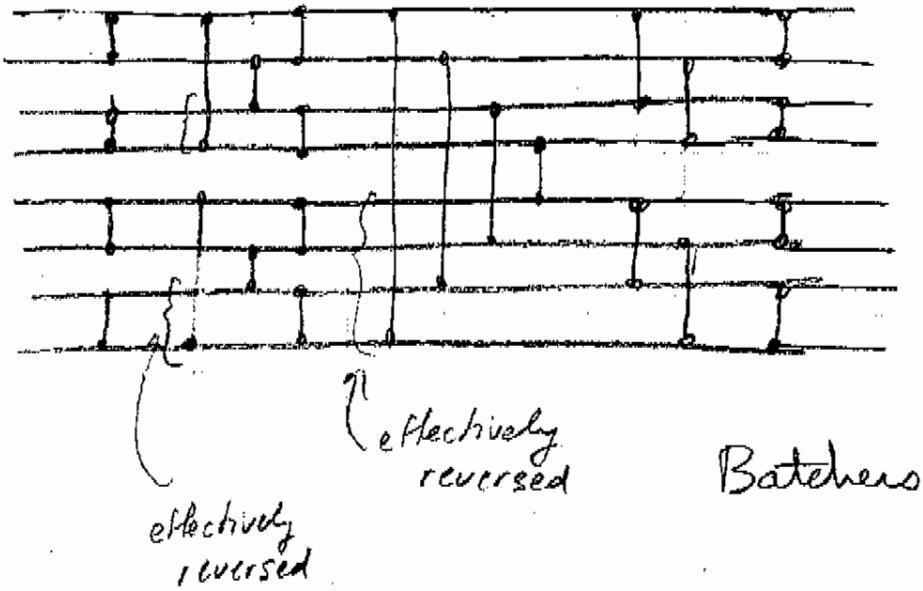
Step 2: Construct merging network, one sorted up, other down.



Frequently drawn where ↕ means ↧



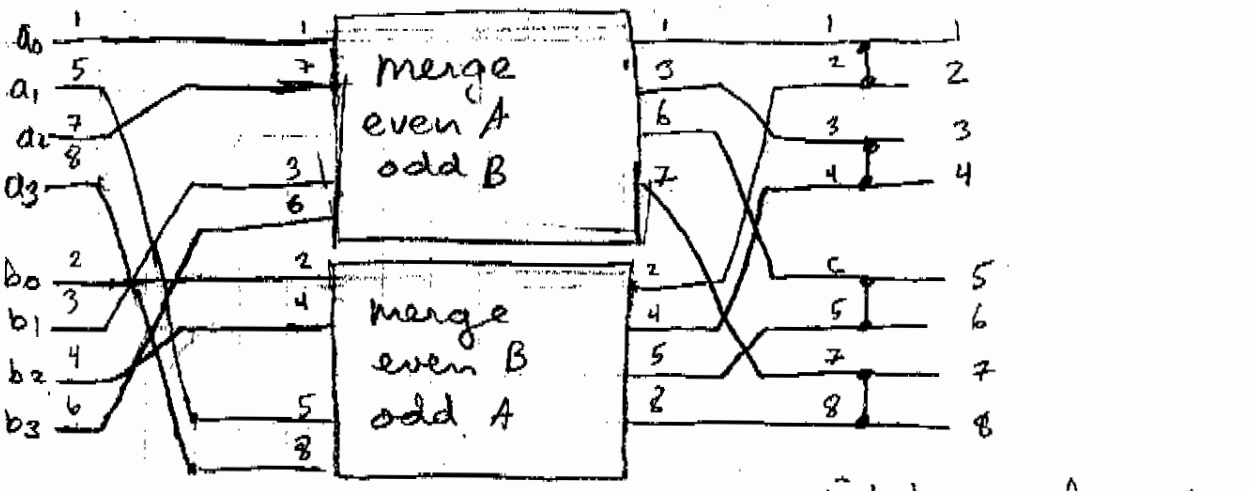
Merging Circuit



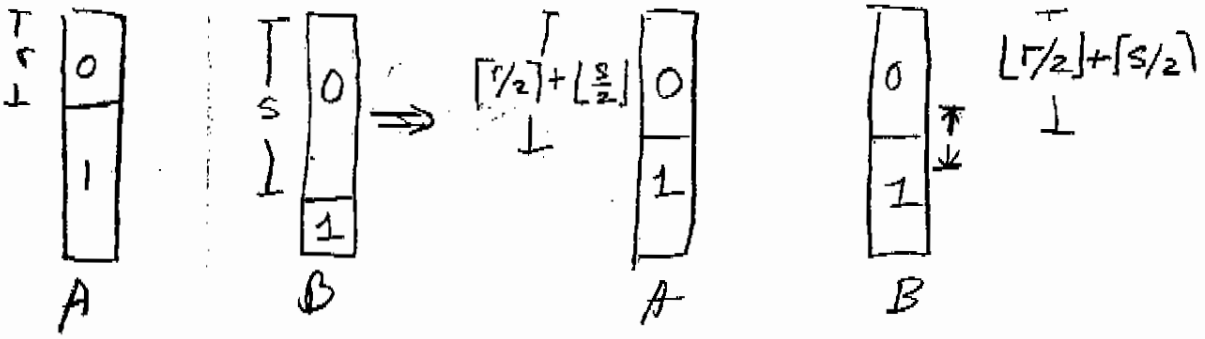
Batchers circuit

Batcher's Odd-Even Mergesort

Step 1: build merger of $A = a_0 \ldots a_{N/2}$, $B = b_0 \ldots b_{N/2}$



Proof: 0-1 Lemma.



$\Rightarrow$ #0's in each list differs by 1.

Merge $M(N) = M(N/2) + 1$
$= \Theta(N \lg N)$

Sort: $S(N) = S(N/2) + M(N)$
$= \Theta(N \lg^2 N)$.

Longstanding open question:

Does there exist sorting network with depth $O(\lg N)$?

1983: yes! AKS sorting network (Ajtai, Komlós, Szemé

Depth: N

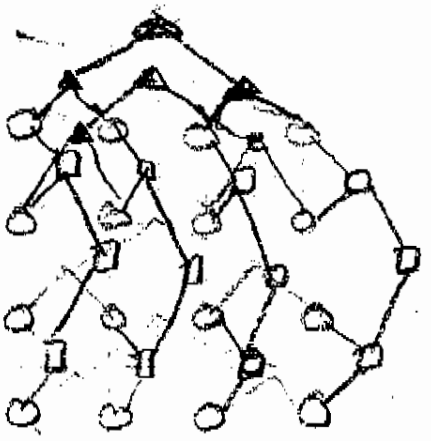
comparisons: $O(N \lg N)$

Unfortunately VERY large constants: many thousand $\lg N$

Sorting on Mesh of Trees.

Def: 2-dimensional mesh of trees (MOT) $M_{2,N}$.

$N \times N$ Grid → · remove grid edges
· add tree above every row & column



Nodes: $N(2N-1) + N(N-1) = 3N^2 - 2N$

diameter: $4 \lg N$

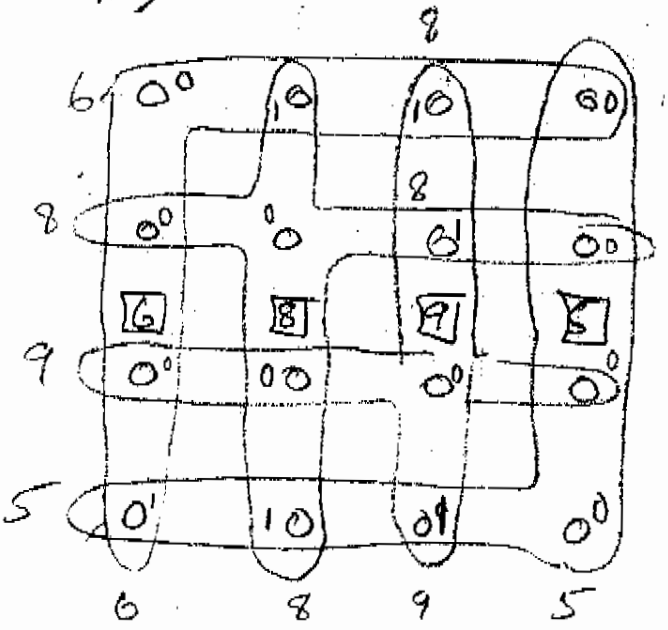
bisection width: N

recursive decomposition: remove all roots
⇒ 4 separate $M_{2,N/2}$.

Sort: N^2 elmts: $\Omega(N)$ time (bisection LB)

Sort N elmts: $\Theta(\lg N)$ time

$(w_1, \ldots, w_N)$



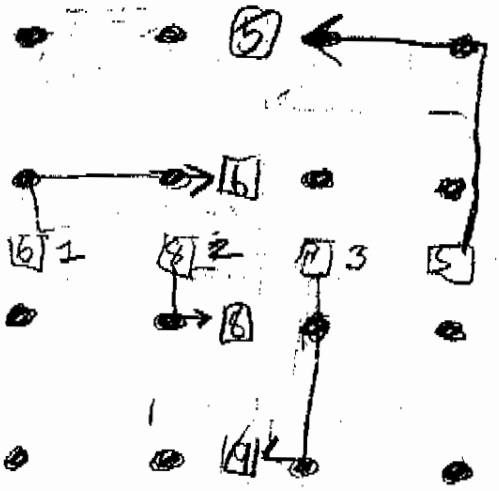
(1) pass w_i along i^{th} row & column

(2) In node p_{ij} (row i, column j) store $\begin{cases} 1, & w_i \le w_j \\ 0, & w_j > w_i \end{cases}$

(3) count #1's in j^{th} tree $\Rightarrow$ rank of w_j in sorted order

(4) if rank$(w_j) = r$ $\Rightarrow$ send w_j to r^{th} row tree.

$\Rightarrow$



<< Wow. Routing ! >>

$2k + 5\lg N$

Note: $\Theta(k + \lg N)$ bitsteps for k bit #s.

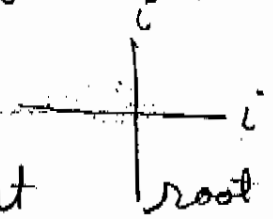
Send MSB first.

Sort N^2 elmts: $\Omega(N)$ time (bisection LB)

Sort N elmts = $w_1, w_2, \ldots, w_N$ $\Theta(\lg N)$ time

Idea: brute force. Do all comparisons.

Given N k-bit #s, following bit-step algo sorts in $2k + 5\lg N$ st

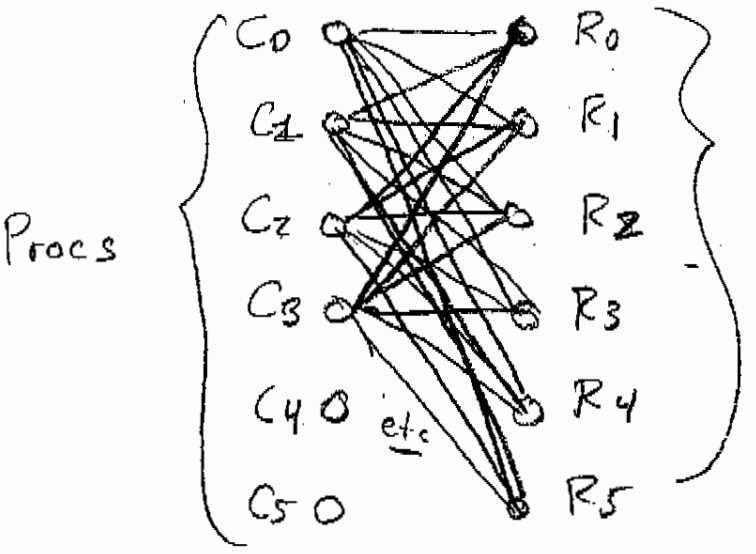
(1) $\forall i$, pass w_i along  i ith column & row
(MSB first.) Store at root of each column tree.

(2) For each leaf, bitwise compare $w_i \stackrel{?}{=} w_j$.
(Break ties with index i, j)
Leaf p_{ij} stores $\begin{cases} 1 & \text{if } w_i < w_j \\ 0 & w_i \geq w_j \end{cases}$

(3) $\forall j$, count # 1's in leaves of jth column tree
$\Rightarrow$ rank of w_j in sorted order.

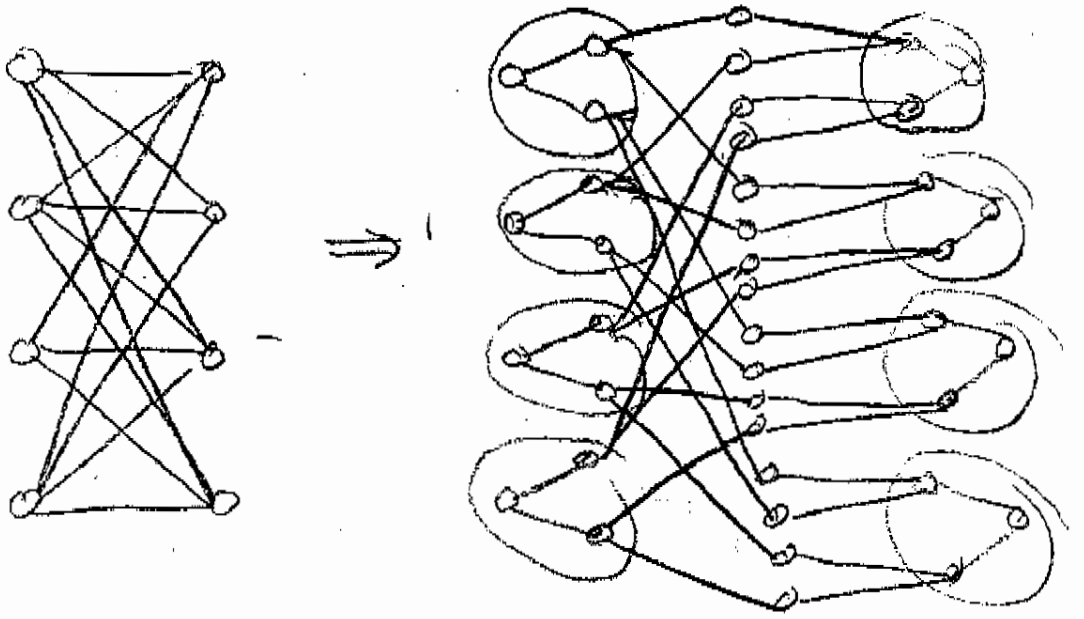
(4) If $\text{rank}(w_j) = r$, send w_j to root of rth row tree

Simulating Bipartite Graph / Ideal Computer on MOT



For large N, K_{NN} not realistically implementable.

$\Rightarrow$ Simulate ~~K_{NN}~~ by M_{2N} with $2 \lg N$ delay

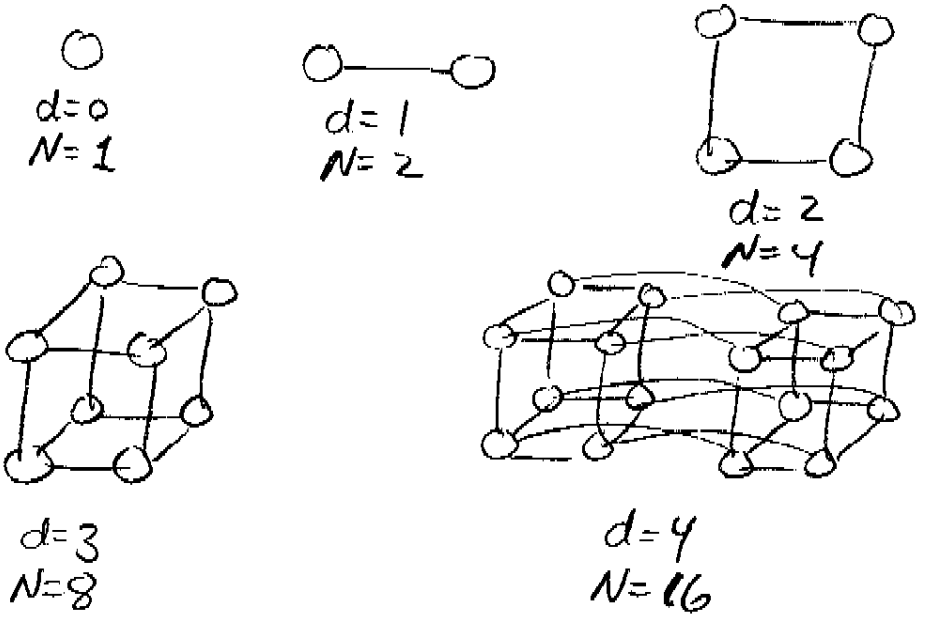


The catch: quadratic blowup in space/hardware.

6.896
3/29/04
L12.1

Hypercube network

d dimensions
$N = 2^d$ nodes



Label each of the 2^d nodes with a d-bit binary string:

$b_{d-1}\, b_{d-2} \cdots b_0$

Connect two nodes if they differ in exactly 1 bit:
(Hamming distance = 1)

↑ # bit positions in which they differ.

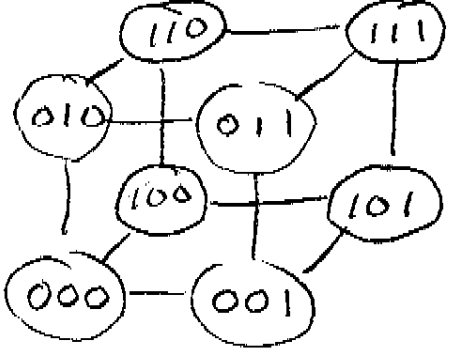
$b_{d-1}\, b_{d-2} \cdots b_0$ connected to

$\overline{b_{d-1}}\, b_{d-2} \cdots b_0$

$b_{d-1}\, \overline{b_{d-2}} \cdots b_0$

$\vdots$

$b_{d-1}\, b_{d-2} \cdots \overline{b_0}$



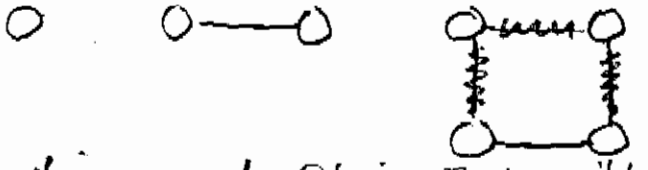
Diameter = $d = \lg N$
Degree = $d = \lg N$
BW = $N/2$
#wires = $Nd/2 = \Theta(N \lg N)$

6.896
3/29/04
L12.2

Embeddings in the hypercube

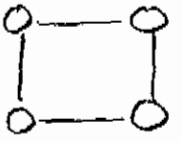
Theorem The N-node hypercube contains an N-node linear array as a subgraph (i.e., a hamiltonian path).

Pf. True for $N = 1, 2, 4$:

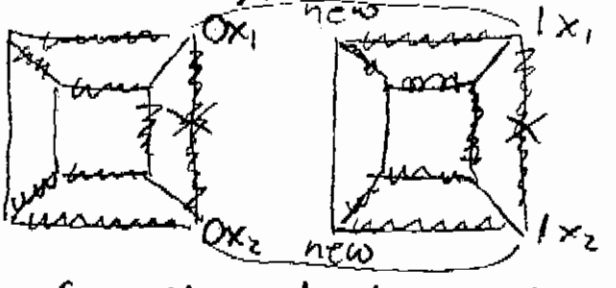


Induction on d. Claim $\exists$ hamiltonian cycle for d-dim hypercube for $d \geq 2$.

Base:



Assume claim true for $N/2$-node hypercube.
Consider $N = 2^d$ hypercube



Consists of 2 $N/2$-node hypercubes containing (identical) hamiltonian cycles (by IH). Let $(0x_1, 0x_2)$ be any edge in 1st subcube that cycle goes through, and let $(1x_1, 1x_2)$ be corresp. edge in 2nd subcube. Replace these two edges with $(0x_1, 1x_1)$ and $(0x_2, 1x_2)$. ☒.

Def. A d-bit Gray code is an ordering of the 2^d d-bit bit-strings such that each string differs from the previous in exactly one bit.

Ex. $d=3$

0	0 0 0
1	0 0 1
2	0 1 1
3	0 1 0
4	1 1 0
5	1 1 1
6	1 0 1
7	1 0 0

"Reflecting" Gray code

Corollary. d-bit Gray codes exist $\forall d$. ☒

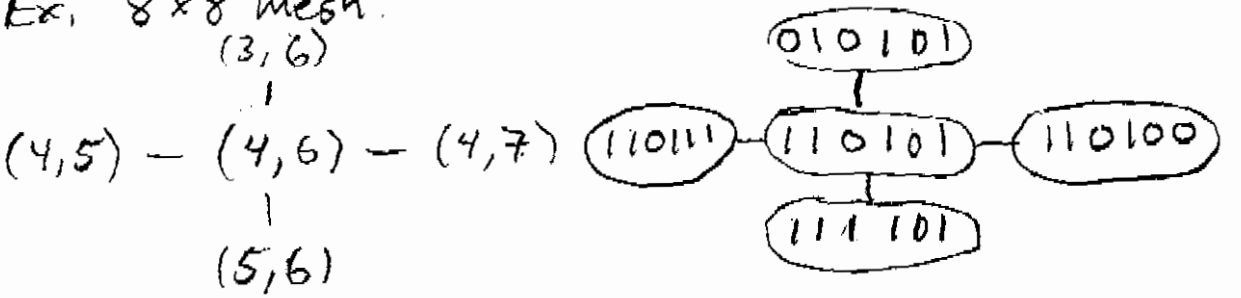
Hamiltonian path in hypercube = Gray code.

Theorem. Let $d_1 + d_2 = d$. Then a $2^{d_1} \times 2^{d_2}$ mesh (or torus) can be embedded in an $N = 2^d$-node hypercube.

Pf. Let $g_1(x_1)$ be d_1-bit Gray code of x_1, where $0 \le x_1 < 2^{d_1}$.
$g_2(x_2)$ d_2 x_2 $0 \le x_2 < 2^{d_2}$.

Map node (x_1, x_2) of mesh to node $g_1(x_1) \| g_2(x_2)$ of hypercube. ☒
↑ concatenate

Ex. 8×8 mesh.



Corollary. $2^{d_1} \times 2^{d_2} \times \cdots \times 2^{d_k}$ mesh embedded in $2^{d_1 + d_2 + \cdots + d_k}$ hypercube.

Fact: 3×5 mesh cannot be embedded in 16-node hypercube. But, $m \times n$ mesh can be embedded in $2^{\lceil \lg(mn) \rceil}$-node hypercube with dilation 2.

6.896
3/29/04
L12.4

Embedding trees in hypercubes

Thm. Not possible to embed $(N-1)$-node complete binary tree in N-node hypercube.

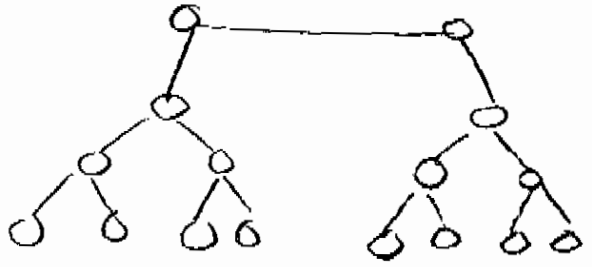
Proof. Sup. possible. Root mapped to node 00...0.
Depth-1 nodes mapped to nodes with odd parity.
Depth-2 " " " " " even "
Depth-3 " " " " " odd "

(**Def.** Parity = { odd if # 1's is odd / even if # 1's is even })

leaves = $N/2$: all have same parity
grandparents of leaves = $N/8$: same parity as leaves.

But, hypercube has $N/2$ nodes with even parity and $N/2$ nodes with odd parity, and tree must have $\geq N/2 + N/8$ nodes with same parity. #

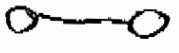
Def. Double-rooted complete binary tree :



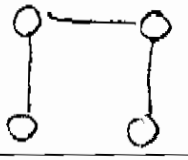
Thm. N-node double-rooted cbt is subgraph of N-node hypercube, for $N \geq 2$.

Proof. Induction on d.

$d = 1$ $(N = 2)$:

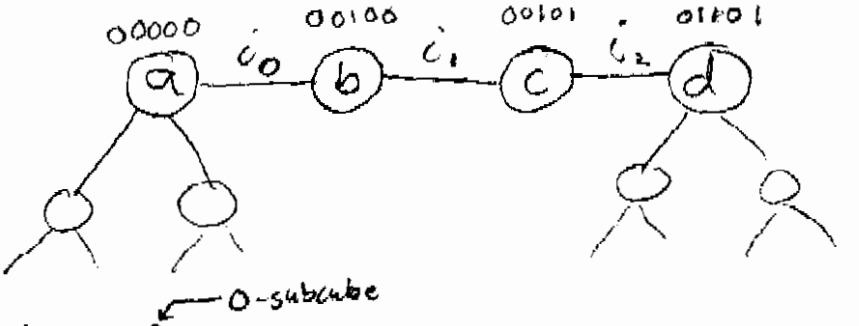


$d = 2$ $(N = 4)$



Mar 31 04 01:12P

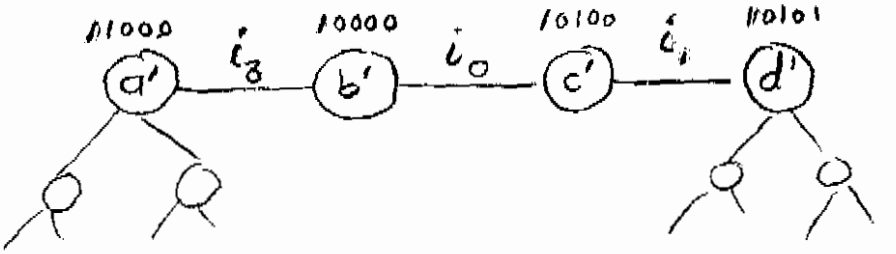
$d \geq 3$ ($N \geq 8$): Embed double-rooted cbt on $N/2$ nodes in $N/2$-node 0-subcube. Consider top 4 nodes:



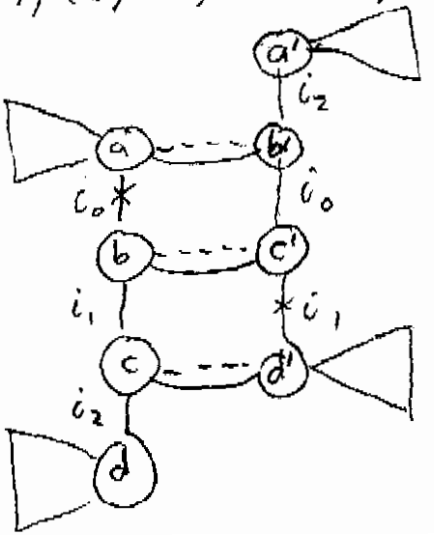
WLOG, $a = 00\cdots0$
a, b differ in dim i_0
b, c " " " i_1
c, d " " " i_2

Note: $i_0 \neq i_1 \neq i_2$ (or else $a=c$ or $b=d$).

Embed double-rooted cbt on $N/2$ nodes in $N/2$-node 1-subcube identically, except $b' = 100\ldots0$ and permute dimensions $\$$ $i_1 \to i_0$ and $i_2 \to i_1$.



Thus, (a,b'), (b,c'), and (c,d') adjacent.

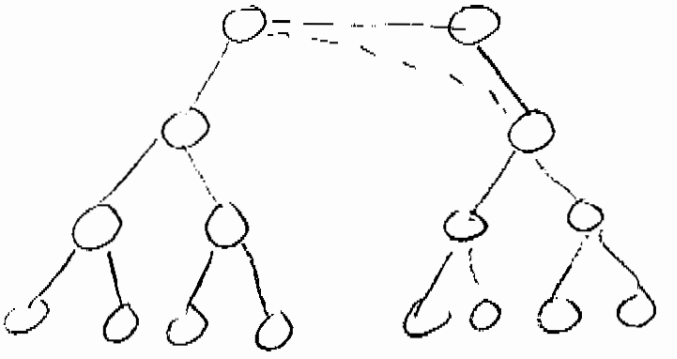


b, c' new roots

□

Corollary $(N-1)$-node CBT embeds in N-node hypercube with dilation 2.

Pf.



Embed CBT into double-rooted CBT with 1 edge having dilation 2. ☒

Fact: All N-node binary trees can be embedded into N-node hypercube with $O(1)$ dilation.
↳ ≤ 5

6.896
3/31/04
L13.1

"Ideal" parallel computer (Slides 2-3)
Problem: # wires = $\Theta(N^2)$ bad
degree = $\Theta(N)$ bad
diameter = $\Theta(1)$ good

Implement as low-degree network (Slides 4-8)
$N \times N$ mesh of trees:
switches = $\Theta(N^2)$ bad
degree = $\Theta(1)$ good
diameter = $\Theta(\lg N)$ good

Direct network: every node is a processor
Indirect network: processors + switches (inputs/outputs)

Routing on $N \times N$ MOT
N messages at row roots
Route to column roots.
- Assume perm, since otherwise hotspot could make any network look bad.
Time = $\Theta(\lg N)$ — but lots of hardware.

Hypercube (Slides 9-10)

Routing: flip any bit that's wrong by routing on that dimension.

1 0 1 1 0 1 0 → 0 1 1 0 1 1 0

Bitwise XOR of current msg location and dest.
1 1 0 1 0 1 0 0 → 0 0 0 0 0 0 0 0
But, msgs may collide.
Also, degree = $\lg N$.

6.896
3/31/04
L13.2

Cube-connected cycles (Slide 11)

$N = n \lg n$ nodes
degree $= \Theta(1)$
diam $= \Theta(\lg N)$

Butterfly (FFT) network (Slides 12-13)

n inputs, n outputs <<direct vs. indirect>>
$N = n \lg n$ nodes
$\Theta(1)$ degree
Diameter $= \Theta(\lg N)$ <<little tricky if not 1 or 0>>

Isomorphic to CCC, but authors didn't realize.

Decomposing a butterfly (Slides 14-24)

Remove major cycles ⇒ 2 n/2-input butterflies
" minor cycles ⇒ " " " "

Routing on butterfly (Slide 25)

- Just like hypercube, but uses a specific order of dimensions

{ dest = 0 ⇒ go up, 1 ⇒ go down } or { XOR = 0 ⇒ straight, 1 ⇒ cross }

Tree embeddings in butterfly (Slides 26-27)

- CBT rooted at each input
- CBT " " " output

6.896
3/31/04
L 13.3

Packet routing on butterfly

source → dest
$x_{d-1} x_{d-2} \cdots x_0 \rightarrow y_{d-1} y_{d-2} \cdots y_0$

Route major to minor:

$x_{d-1} x_{d-2} \cdots x_0$
$y_{d-1} x_{d-2} \cdots x_0$
$y_{d-1} y_{d-2} \cdots x_0$
$\vdots$
$y_{d-1} y_{d-2} \cdots y_0$

$d = \lg n$ steps, but might have congestion!

n packets on n-input butterfly
What is worst-case perm?
- $\sqrt{n}$ packets at sources $x_1x_2x_3x_4 0000$ go to dests $0000 x_1x_2x_3x_4$

All go through line 00000000 halfway through network ⇒ congestion = $\sqrt{n}$.

Beneš network (Slides 28-29)

Thm. Any n-perm can be routed (off-line) on an n-input Beneš with node-disjoint paths.

Pf. Induction on n.
Base ($N=2$): 0 or 1 / 1 or 0 ☒
Inductive case (Slides 31-39) ☒

Corollary A dn-input Beneš network can simulate any n-node, degree-d network in $O(\lg(dn))$ time. ☒

Bounded-degree ⇒ $O(\lg n)$ time

Beneš network is $O(\lg n)$-universal for offline simulation of bounded-degree networks.
«Analogy to universal Turing machines»

6.896
4/5/04
L14.1

Bitonic Sorting on Butterfly (Slides 40 - 53)

$O(\lg^2 n)$ time

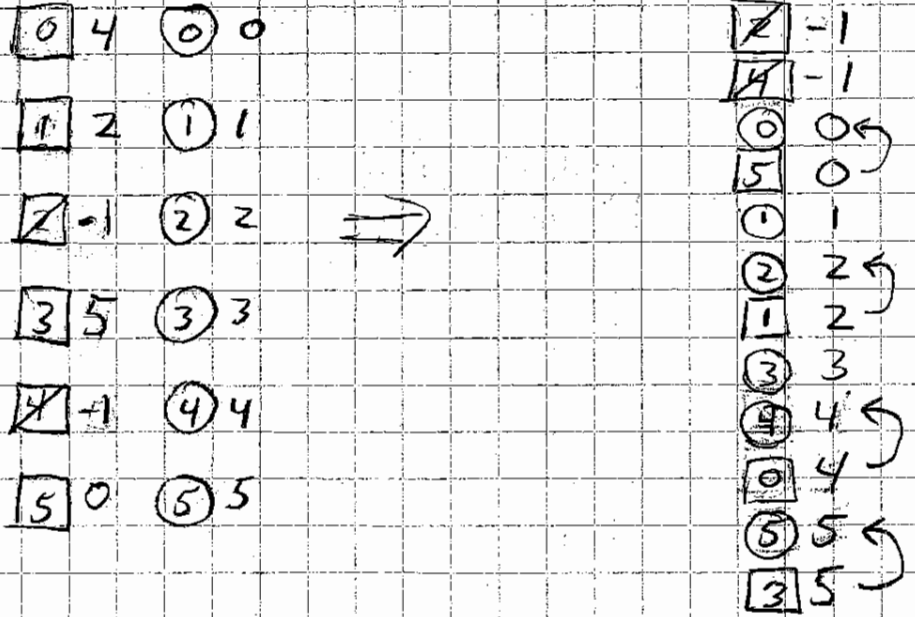
Routing perms by sorting

- Label each packet with dest. ID
- Sort ID's
- Packet with dest i ends up at processor i.

Works on any network capable of sorting.
But, what if some procs have nothing to send?
— Routing subpermutations.

Thm. Routing is no harder than sorting (to within $O(1)$)

Pf.
1. Give "empty" messages a label $= \infty$
2. Each real message has label of dest.
3. Each proc i creates "dummy" msg with label i.
4. Sort the $2n$ msgs.



5. Each real msg goes back to proc that sent adjacent dummy msg.

Route n-perms on butterfly in $O(\lg^2 n)$ time.
In fact, can route N-subperms in $O(\lg N)$ expected time.

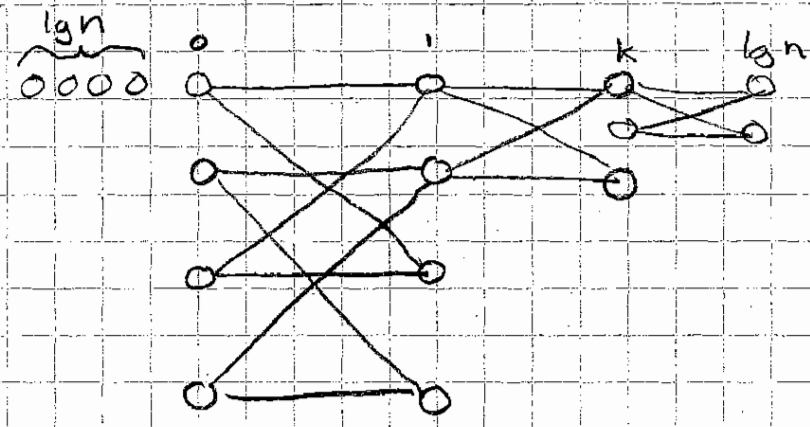
6.896
4/5/04
L14.2

Theorem Consider the N^N N-packet routing problems on an N-node ($n = \Theta(N/\lg N)$-input) butterfly. At least $N^N(1 - 1/N^{\Omega(1)})$ of these problems can be routed in $O(\lg N)$ time.

Proof. We'll do a congestion bound only, which leads to an $O(\lg^2 N)$-time result.

Algorithm

1. Route packet along row to output. $O(\lg n)$
2. Route to dest row using greedy
3. Route along row to dest node. $O(\lg n)$



Consider level-k node x during Phase 2.
packets that can reach x is $2^k \lg n$.
(tree in butterfly, Slide 27).

Prob that given packet passes through node x
$\leq 2^{-k}$ (might not be able to reach x)

Consider any set of r specific packets.
Prob they all pass through node x
$\leq (2^{-k})^r = 2^{-kr}$ (independence)

6.896
4/5/04
L14.3

Prob that $\geq r$ packets pass through node x

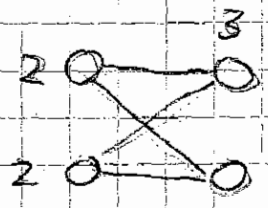
$$\leq \binom{2^k \lg n}{r} 2^{-kr}$$

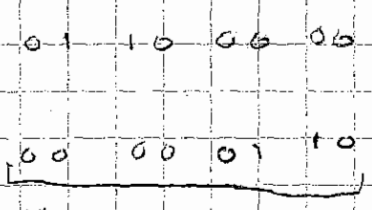
$\binom{2^k \lg n}{r}$ ← # ways to choose r packets

2^{-kr} ← prob they all go through x.

Note: this overcounts. If $r+\Delta$ packets pass through x, this event is counted $\binom{r+\Delta}{r}$ times within the $\binom{2^k \lg n}{r}$ ways.

Eg.





$\binom{4}{3} = 4$

$$\leq \left(\frac{e 2^k \lg n}{r}\right)^r 2^{-kr} \qquad \boxed{\binom{a}{b} \leq \left(\frac{ea}{b}\right)^b}$$

$$= \left(\frac{e \lg n}{r}\right)^r$$

Choose $r = 2e \lg N$

$$\text{Prob} \leq \left(\frac{e \lg n}{2e \lg N}\right)^{2e \lg N}$$

$$\leq \left(\frac{1}{2}\right)^{2e \lg N}$$

$$= N^{-2e}$$

$$\leq 1/N^{5.4}$$

6.896
4/5/04
L 14.4

Prob that $\geq 2e \lg N$ packets go through <u>any</u> node
$\leq N \cdot (1/N^{5.4})$
$\uparrow$ # packets
$= N^{-4.4}$

<u>Boole's Ineq.</u>
Prob of union $\leq \Sigma$



No indep. needed

$\therefore \geq N^N (1 - 1/N^{4.4})$ problems see $\leq 2e \lg N$ congestion

Thus, each level takes $O(\lg N)$ time $\times$ $\lg N$ levels
$= O(\lg^2 N)$ time. ☒ <<Can show $O(\lg N)$ whp.>>

<u>Corollary</u> $E[\text{routing time}] = O(\lg N)$.

Pf. $E[\,] = \Sigma t \cdot \Pr\{\text{routing takes time } t\}$

$\leq O(\lg N) \cdot (1 - 1/N^{4.4}) + O(N) \cdot \frac{1}{N^{4.4}}$

$= O(\lg N)$ ☒

Still have bad routing problems.
Valiant: Use randomization to ensure no input prob can elicit w-c behavior (perm routing).

1. Route from source to rand intermediate dest.
2. Route from interned. node to true dest.

Each accomplished in $O(\lg N)$ time whp.
Each is random routing prob.
No bad perm. Only unlucky choices for randomization.

Butterfly is $O(\lg N)$-universal for on-line simulations.

6.896
4.7-2004
L15.1
Bradley C Kuszmaul

"VLSI"

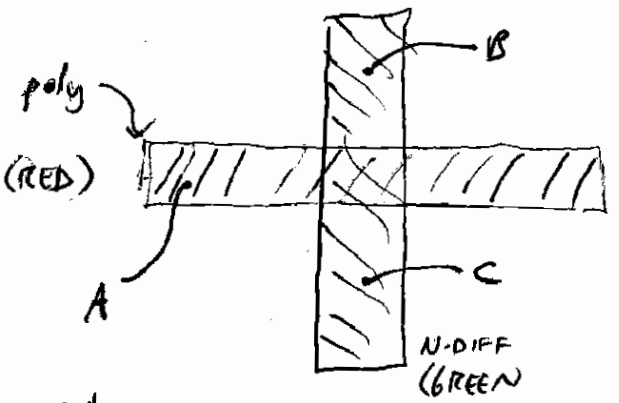
Now we switch gears + talk about the physical design of circuits. Circuits take area on a chip, or volume in a computer. ~~The goal is to design analyse circuits,~~
We can analyse circuits for their area just like for their time.

VLSI circuits are drawn with rectangles

Sidebar: ~~Rectangles~~ Color Code

polysilicon	RED
N-diffusion	~~Yellow~~ GREEN
~~P-diff~~	<MAGIC>
	YELLOW (WHITE)

I use MAGIC colors



When you cross green + red you get a transistor.

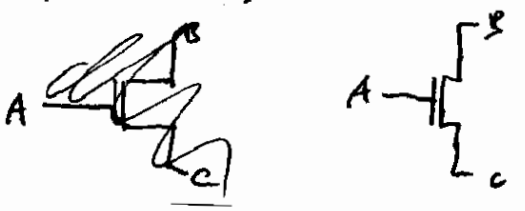
Metal
Oxide
Semiconductor
Field
Effect
Transistor

~~Three regions~~:
~~A = all red~~
~~B = green on red~~ ~~be on top of red~~
~~C = green below red~~

This is an N-FET because it uses N-diffusion

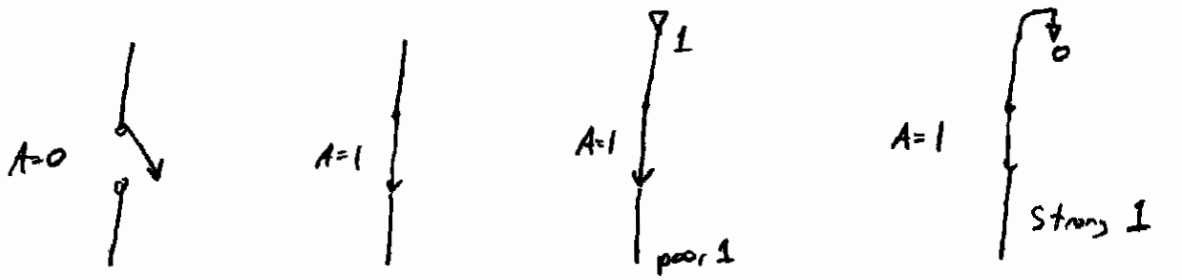
3 Regions:
A - all red area
B = green ~~on~~ above red
C = green below red

In schematic this is



6.896 4-7-04 L5.2

N-FET behavior: (A switch)



Switch open

switch closed

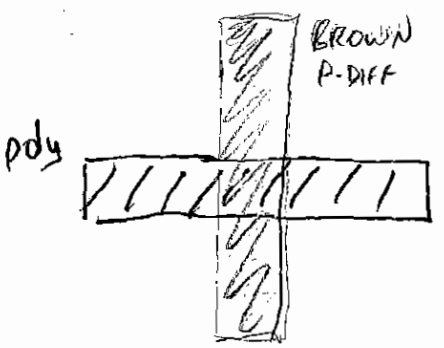
But the ~~bubble~~ N-FET is only good at transmitting 0's.

P-FET

Could to sidebar:

P-diffusion

BROWN (MAGIC)
GREEN (MOSIS)



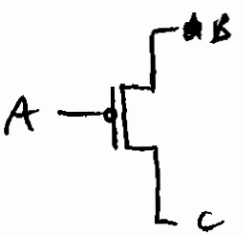
P-FET

~~Opposite behavior~~

~~A=1 open~~

A=0 ⇒ closed, passes 1's well

A=1 ⇒ open

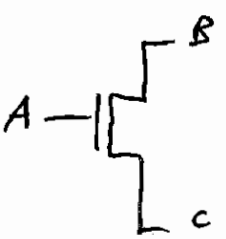


"Bubble means P-FET"
"active low"

N-FET

A=0 open

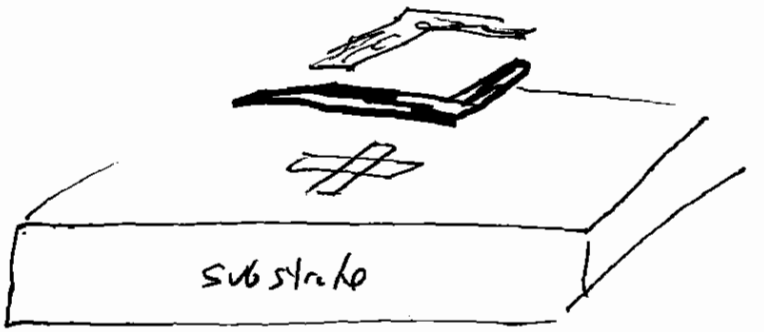
A=1 closed, passes 0's well



No bubble = NFET
"active high"

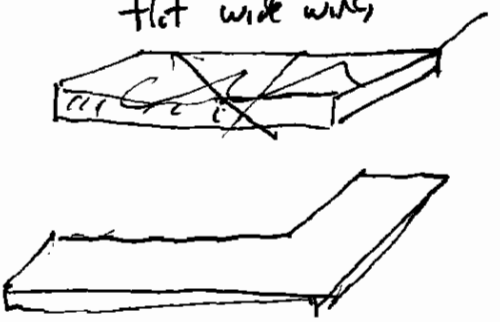
6.896 4-7-04 15.3

VLSI is a club sandwich

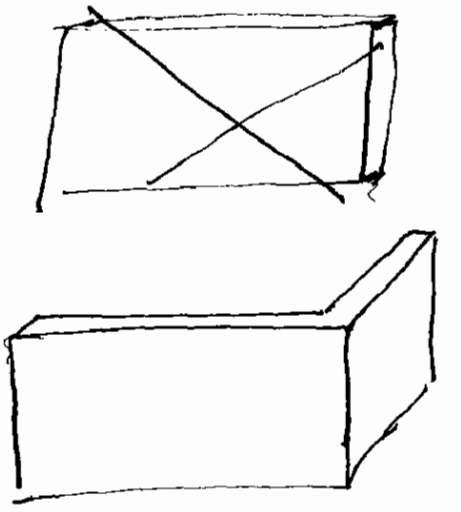


Bottom layer: substrate
next: diff-stuff + poly
next: metal 1 (blue) [add to sidebar]
next: metal 2 (purple) [add to sidebar]

Old Days:
1 or 2 layers of metal
flat wide wires



Now:
20 layers of metal
tall thin wires



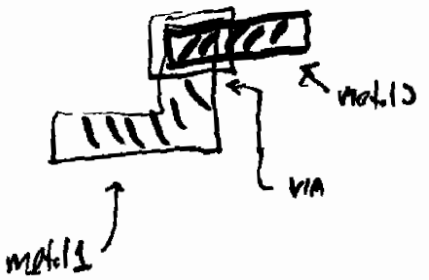
(For EE's)
Implications: in old days capacitive couplings mostly between wire + substrate. $C = O(\text{Area of rectangle})$

Now: most coupling is between adjacent wires. $C = O(\text{Perimeter of rectangle})$

Lots of crosstalk.

6.896 4-7-04 15.4

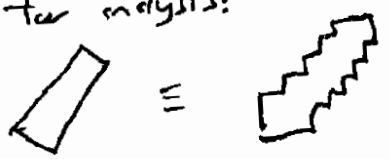
Adjacent layers connected by vias



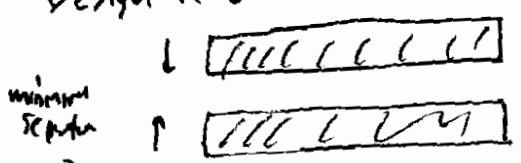
Build a 3-D maze of conductor out of metal.

Angles: We'll stick to 90° angles
Modern technologies include 45° + other angles.

Only a Constant factor for analysis:



Design Rules:

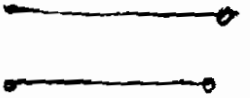


minimum line width (to avoid open circuit)

to avoid short circuit.

Can be combined to "center-to-center spacing"

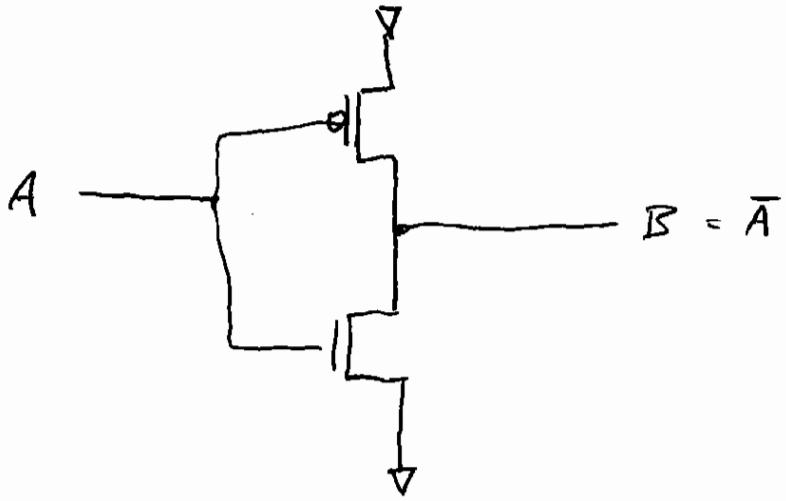
Can draw the lines on a grid, + capture all info.



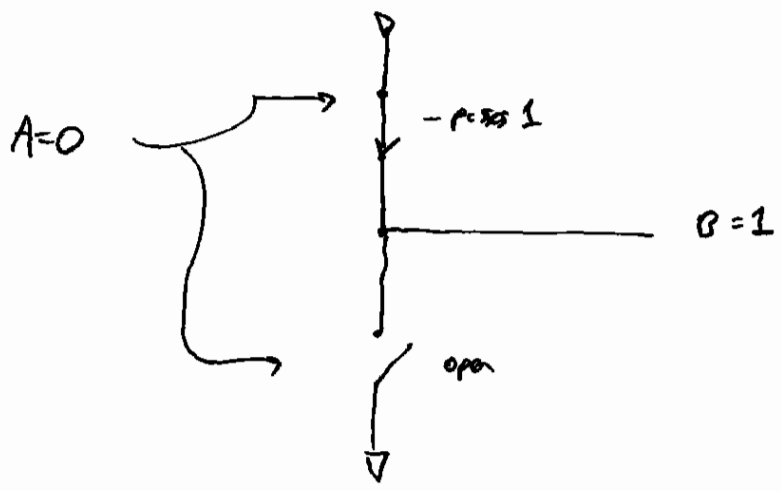
6.896 4.7-04 15.5

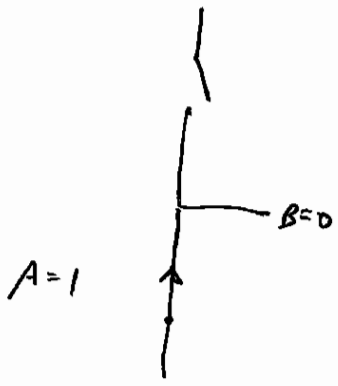
A CMOS inverter.

Schematic:



How it works

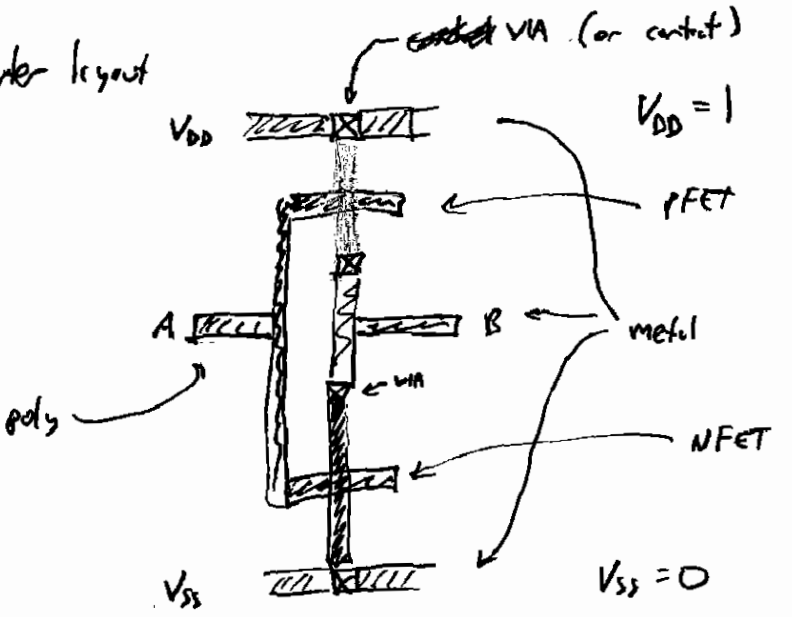




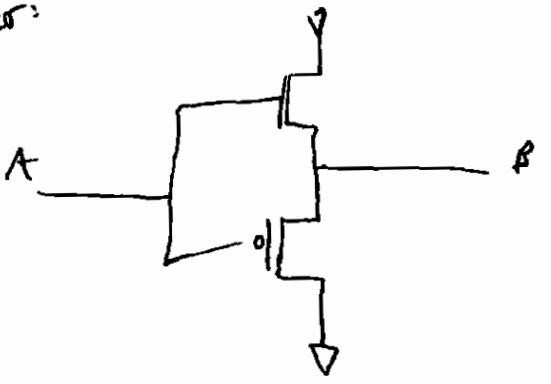
The P-FETs are used to pass 1's, the NFETs to pass 0's.

6.896 4.7-04
15.6

Inverter layout



a repeater:



$B = A$

WRONG NFET can't pass a good 1

Must use two inverters



Q: Why would you want a repeater?

A: To send long distance, a repeater takes a poor 1 + restores it to a good 1.

6.896 4-7-04

15.7

NAND GATE:

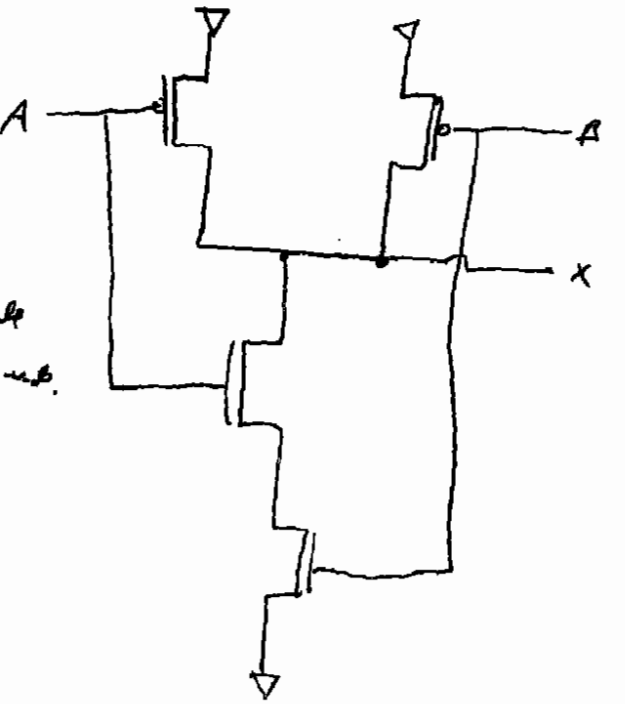
TRUTH TABLE

A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

LOGIC SYMBOL



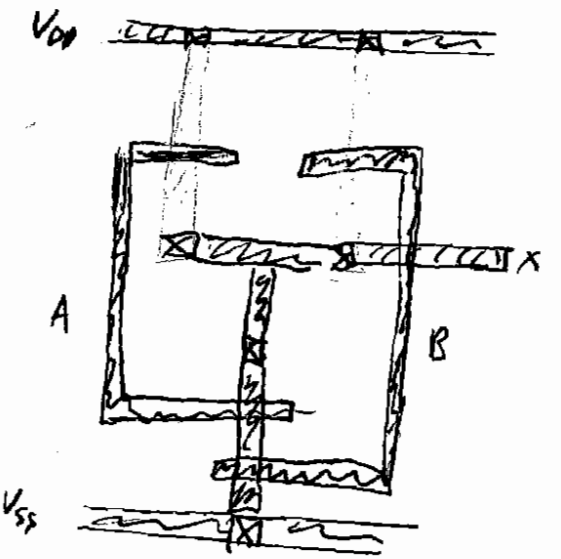
ELECTRICAL SCHEMATIC



If A or B is 0 then output to 1 will make

If A + B are both 1 then output to 0 is made.

LAYOUT



Exercise: Design + Draw a 2-input NOR gate

6.896 15.8 4.7-04

Side-Trip: De Morgan's Law

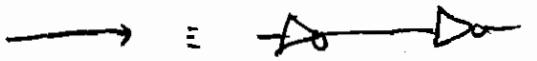
Let A be a circuit comprising inverters, wires, ORs, inputs, outputs

Let $\bar{A}$ be the circuit with all ANDs replaced by ORs, & all ORs replaced by ANDs.

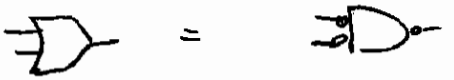
Then if we add inverters to the inputs & outputs of $\bar{A}$ we get a circuit functionally equivalent to A [De Morgan's Law]

Proof: By induction on circuit size.

Base cases







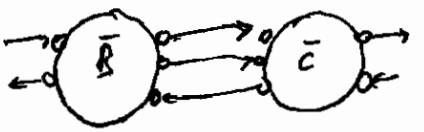


Inductive step:

A:



$\equiv$ By ind. hyp



$\equiv$ cancel



$\equiv$





6.896 4-7-04

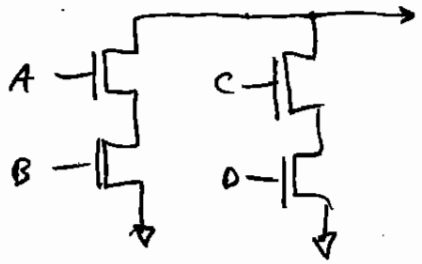
15.9

A bisser example: $F = \overline{A \cdot B + C \cdot D}$

Need: a path to 0 iff F is false
a path to 1 iff F is true

The N-side (path to 0)

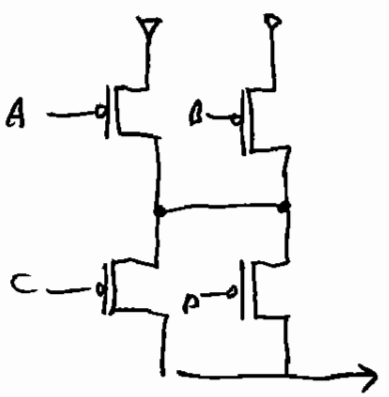
N-fets create path when $\overline{A \cdot B + C \cdot D}$ is false,
i.e. $A \cdot B + C \cdot D$ is true



The P-side: path to 1 if $\overline{A \cdot B + C \cdot D}$ is true.

Path made of P-FETS, so all inputs must be inverted to make the p-fets switch on. Use Demorgan

$$\overline{A \cdot B + C \cdot D} \equiv (\overline{A} + \overline{B}) \cdot (\overline{C} + \overline{D})$$





Exercise: Design Electrical-Level circuit for
$F = \overline{(A+B+C) \cdot D}$

6.896
4-12-2004
Lecture 16.1
Bradley C. Kuszmaul

More CMOS circuits

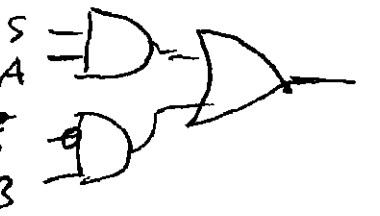
MUX:



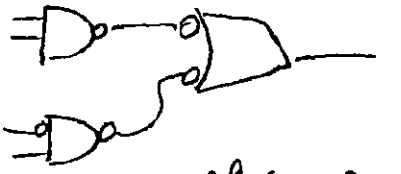
TRUTH TABLE

S	A	B	D
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

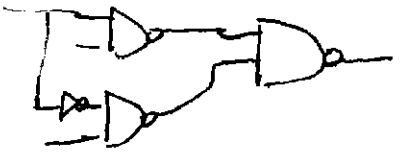
MADE OF GATES



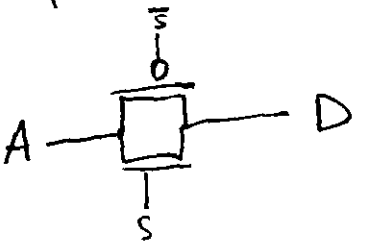
!!! add matched inverters



!!! by Demorgans



We can implement muxes with pass-~~transistors~~ gates

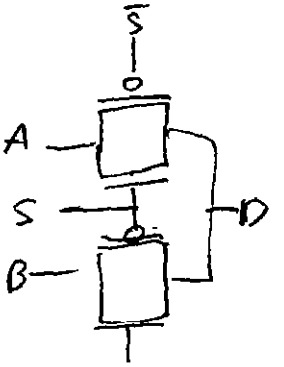


truth table

~~S~~A	S	A	
~~0~~0	0	0	?
0~~1~~1	0	1	?
1~~0~~0	1	0	0
1~~1~~1	1	1	1

} no value provided
} somewhat weaker

MUX



if S is 1 the top pass gate ~~pass~~ allows A through
else the bottom gate lets B through.

6.896
4-12-2004
Lecture 16 & 16.2

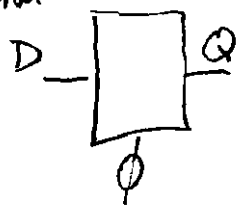
More circuits:

~~Registers + latches~~

STATE:

Level-~~sensitive~~ clocked latches:

Symbol



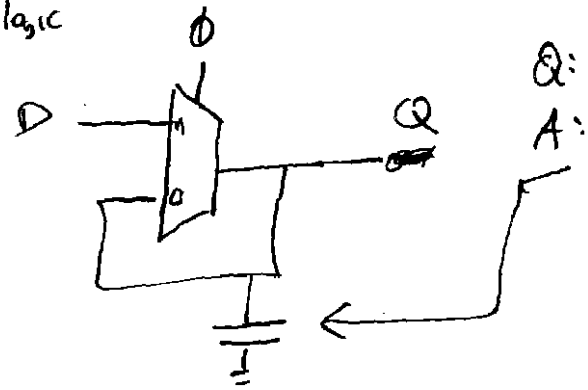
~~Logic eqt~~

Truth Table

D	Φ	Q
0	0	D
1	0	D
X	1	Q

Φ if Φ=0 pass data through
if Φ=1 freeze output

In logic

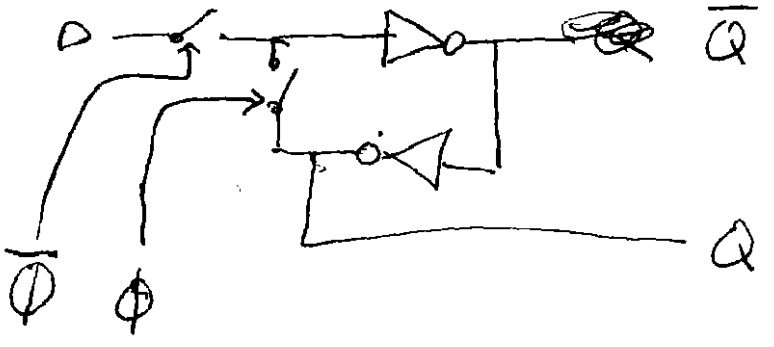


Q: Why does this work?

A: capacitance here holds the old value long enough for the circuit to stay stable when Φ=1

Brief review of EE by analogy to water
~~electric circuit~~ ⇔ water circuit
capacitor ⇔ holding tank
~~resistor~~ ⇔ pipe

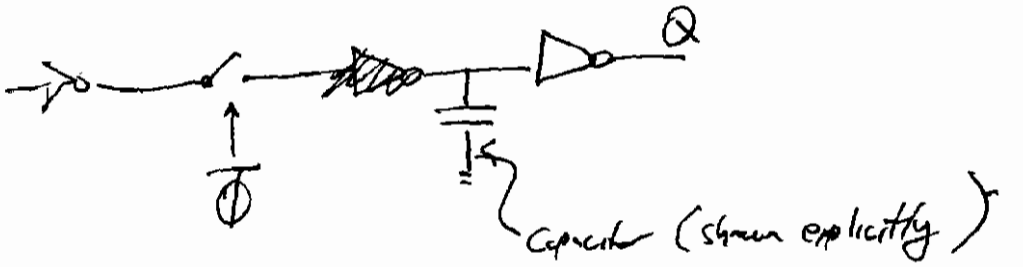
Using pass gates



6.896
4.12.2004
Lecture 16.3

Level sensitive Dynamic Latch.

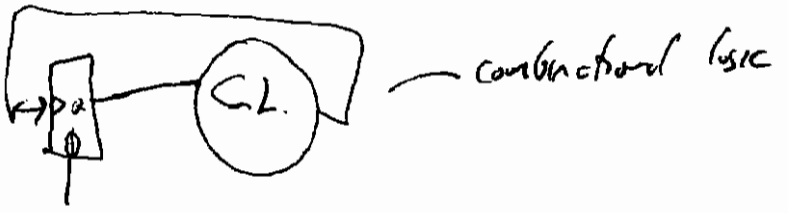
Why bother to ~~re~~ "restore" it with feedback?
∃ a capacitor.



Often ∃ a lot of capacitance in ~~circuits~~ CMOS
if Φ runs fast enough (> 1000 Hz e.g.) then this is OK.

Two-phase clocking

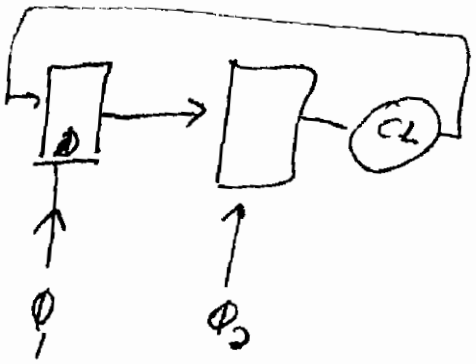
Level-sensitive latches are a little tricky — can't make
a state machine with ~~out~~ that can't

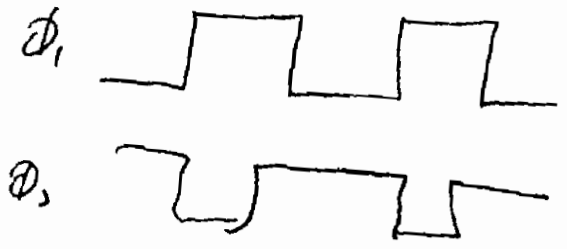


when Φ = 0 there is a loop through the C.L.

6.896
4-12-2004
lecture 16.4

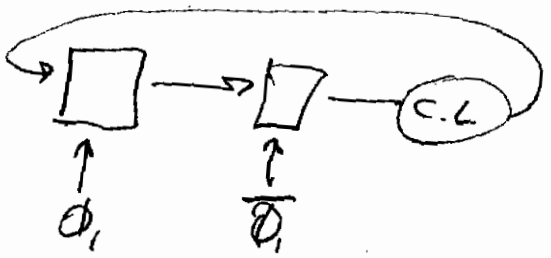
Two phase nonoverlapping clocks





ϕ_1 ~~don't~~ low + ϕ_2 low don't happen at the same time

When ϕ_1 is low data passes through first latch, + ϕ_2 holds its old value. Then when ϕ_2 is low, ϕ_1 is up, so it holds the value coming out of C.L., and data flows through ϕ_2



Will this work?
What if $\overline{\phi_1}$ and ϕ_1 get a little misaligned so they low parts overlap?

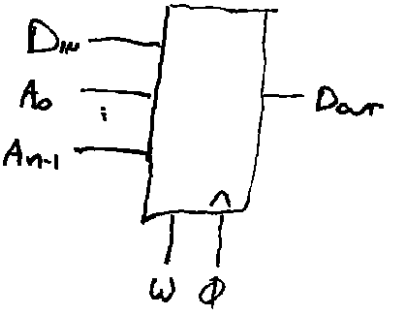
It is ok. if the C.L. has a long enough ~~delay~~ contamination delay.

Or if you arrange $\overline{\phi_1}$ to be non-overlapping

6.896
4-12-2004
16.5

RAM

Random Access Memory





An array of $N=2^n$ bits

$A_0, \ldots, A_{n-1}$ names the bit B_i

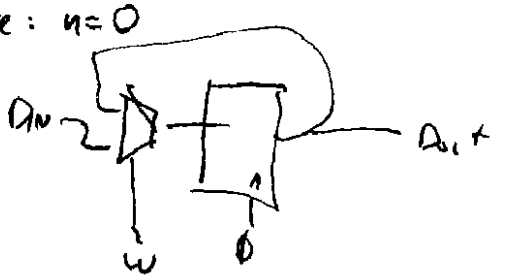
at Φ↑

if W then write D_{IN} to B_i

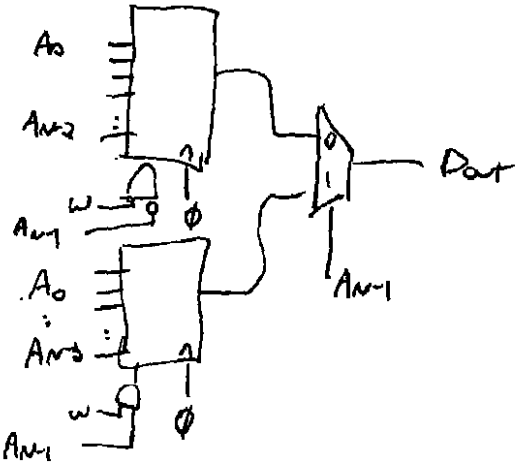
else $D_{OUT} = B_i$

Divide and Conquer

Base case: $n=0$



$n>0$



6.896
4-12-2004
16.6

Analysis of RAM

Critical path:

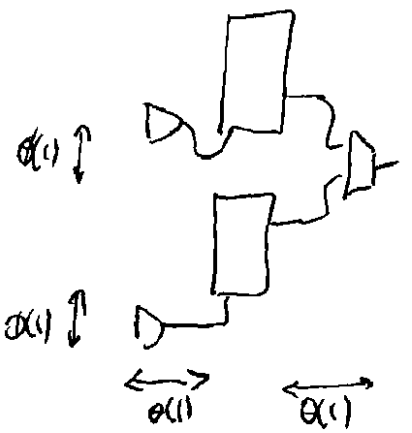
$$T(0) = \Theta(1)$$

$$T(n) = 1 \text{ gate} + 1 \text{ mux} + T(n-1)$$

$$= \Theta(n)$$

$$= \Theta(\log N)$$

Area: Need a layout



width:

$$W(n) = \Theta(1) + W(n-1)$$

$$= \Theta(n) = \Theta(\log N)$$

height

$$H(n) = \Theta(1) + 2 \cdot H(n-1)$$

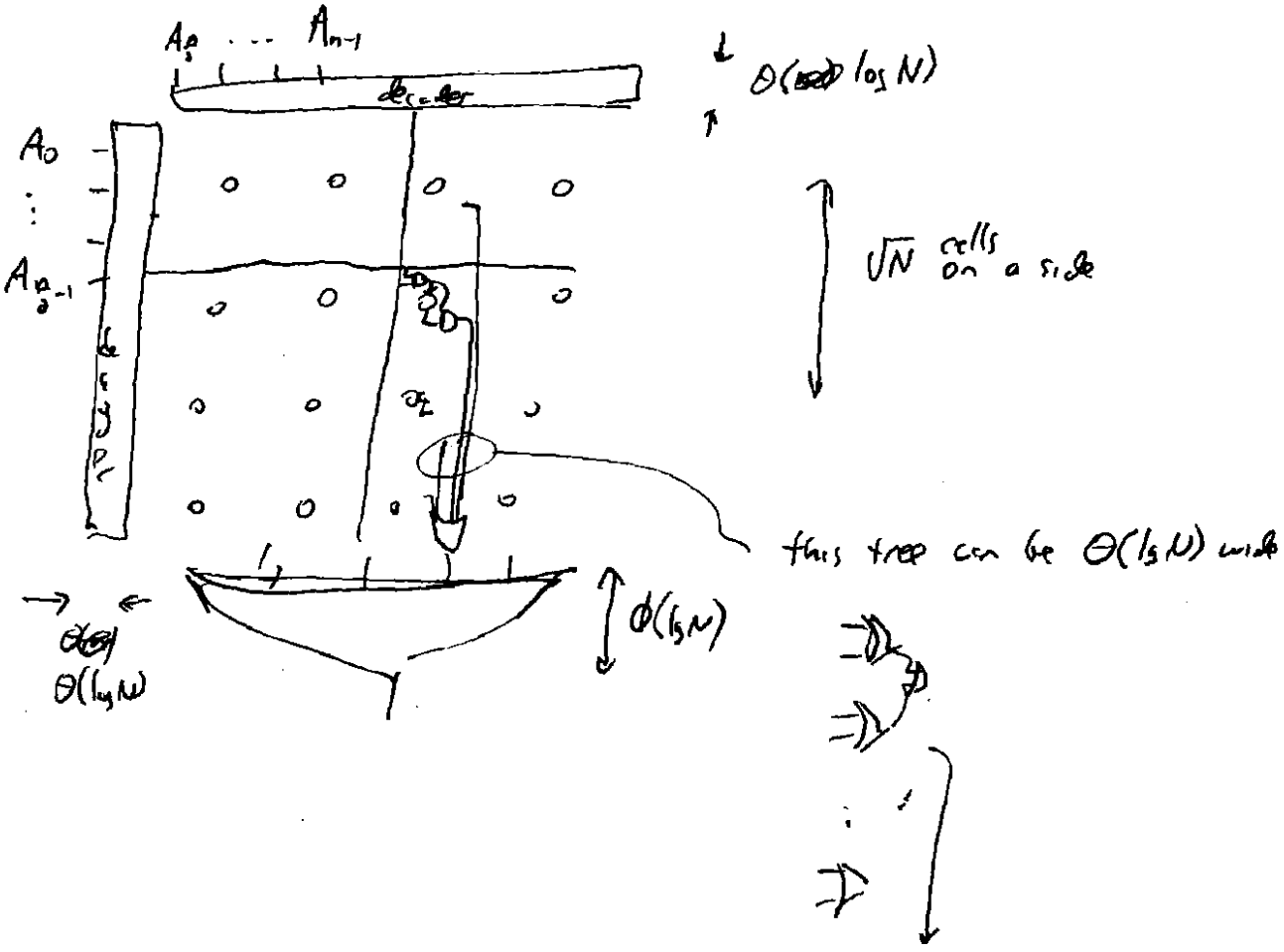
$$= \Theta(2^n) = \Theta(N)$$

Q: Can we do better?

A: yes



6.896
16.7
4-13-2004



total access width:

$$\Theta(\lg N) + \Theta(\sqrt{N} \cdot \lg N)$$

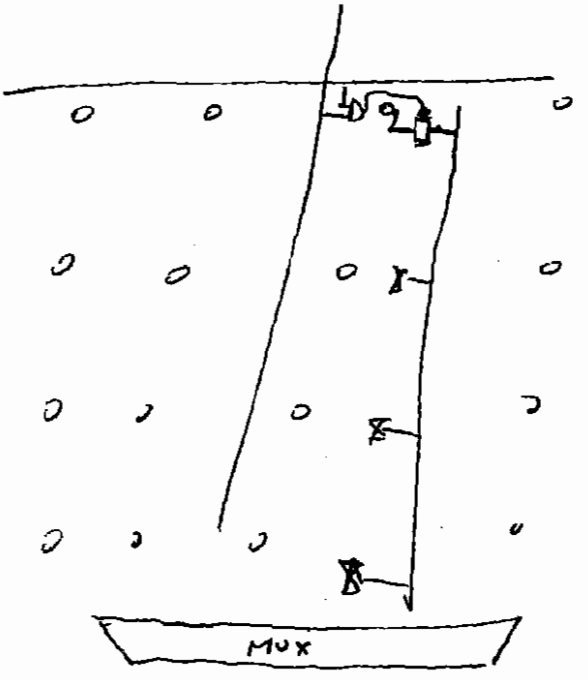
total height

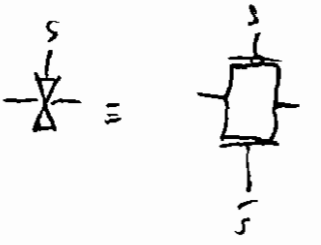
$$\Theta(\lg N) + \Theta(\sqrt{N})$$

total area = $\Theta(N \lg N)$ not better yet!

6.896
16.8
4-13-2004

But for small N we do "wired or"





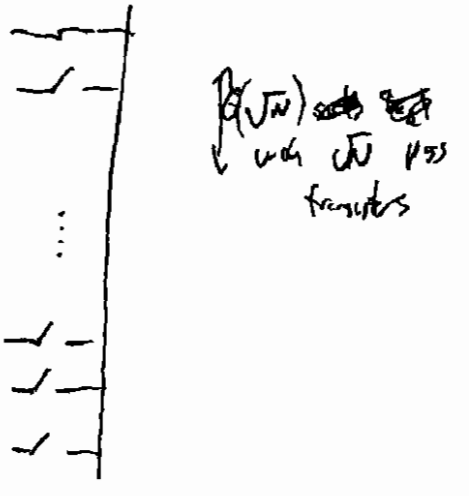
width = $\Theta(\sqrt{N})$
area = $\Theta(N)$

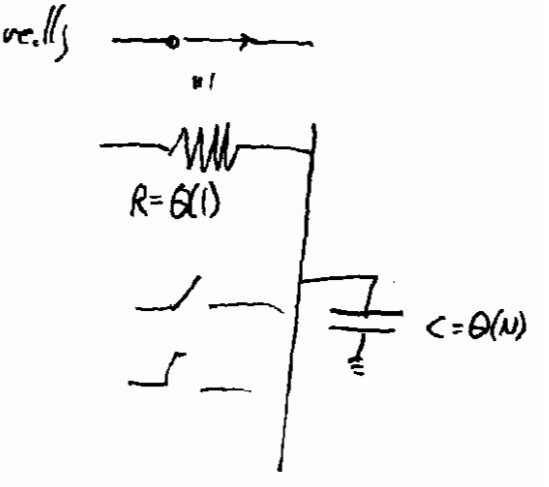
two issues:

Q1) if no one drives the line, what happens?

A: Don't care. The MUX doesn't look at it.

Q2: performance: to drive a long wired or takes time.

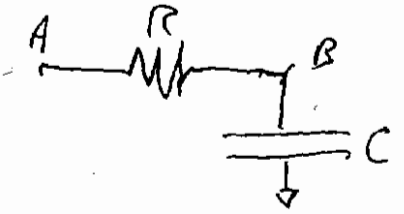


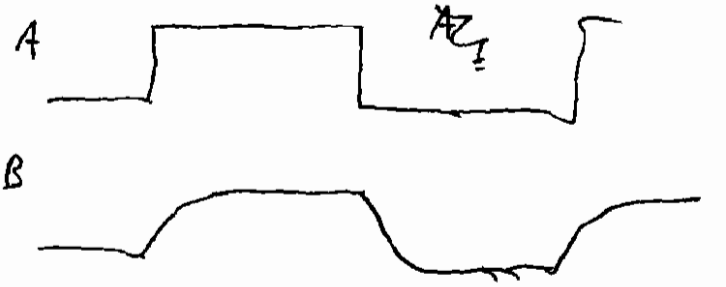


6.896
11.9
4-13-2004

"Now we need to understand RC circuits"

RC circuits





two properties of electricity
current (AMPS) (V) [think current of water in a pipe]
~~voltage~~
potential (VOLTS) (I) [think pressure of water in pipe]

Resistor: $V = I \cdot R$ or $\frac{V}{R} = I$ (more flow if you push harder)

Capacitor: $\int I\,dt = V \cdot C$ (as you pump current through, the voltage goes up)

$= I = \frac{dV}{dt} \cdot C$

OR RC circuit

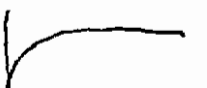
~~$\int I\,dt = I \cdot C$~~ $V = \frac{dV}{dt} \cdot RC$

this is a differential equation with solution of the form

$V = e^{t/RC} + b$ for some τ and b

$e^{t/\tau} + b = \frac{1}{\tau} e^{t/\tau} \cdot RC$

$\Rightarrow \tau = RC$ τ is the "time constant" for the RC circuit



16.10

So our circuit has

$R = \Theta(1)$

$C = \Theta(b \cdot \sqrt{n})$

So $\tau = \Theta(\sqrt{n})$

Next time we'll improve on this

6.896
4-14-2004
Lecture 17.1

Layout:

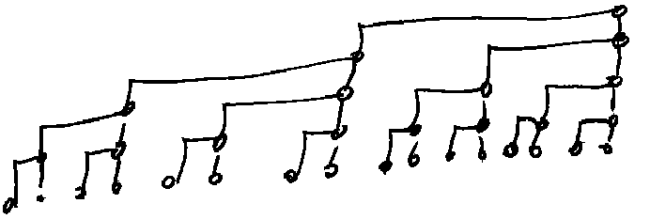
Complete Binary Tree

colinear layout:

Divide + Conquer



e.g.



Analysis:

- width $= \Theta(n)$
- height $= \Theta(\log n)$
- Area $= \Theta(n \log n)$

In fact, can show

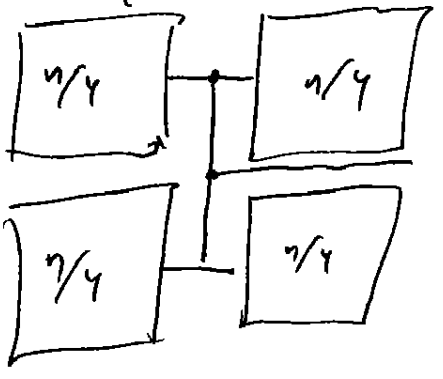
if all leaves are on a line, ~~with area~~

total wire length $= \Omega(n \log n)$

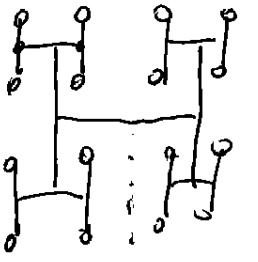
~~Htree layout~~

H-tree layout

Divide + Conquer



e.g.



Always space for wire to come out.

Side length $= 2n$

Analysis $W(n) = 2W(n/4) + \Theta(1)$

$= \Theta(n)$

Longest wire? $\Theta(\sqrt{n})$ in this layout

6.896
17.2
4-14-04

Reduce longest wire?

Can get longest wire to $\Theta(\sqrt{n}/\lg n)$

Thm: Cannot do better:

proof: diameter of net is $\lg n$,

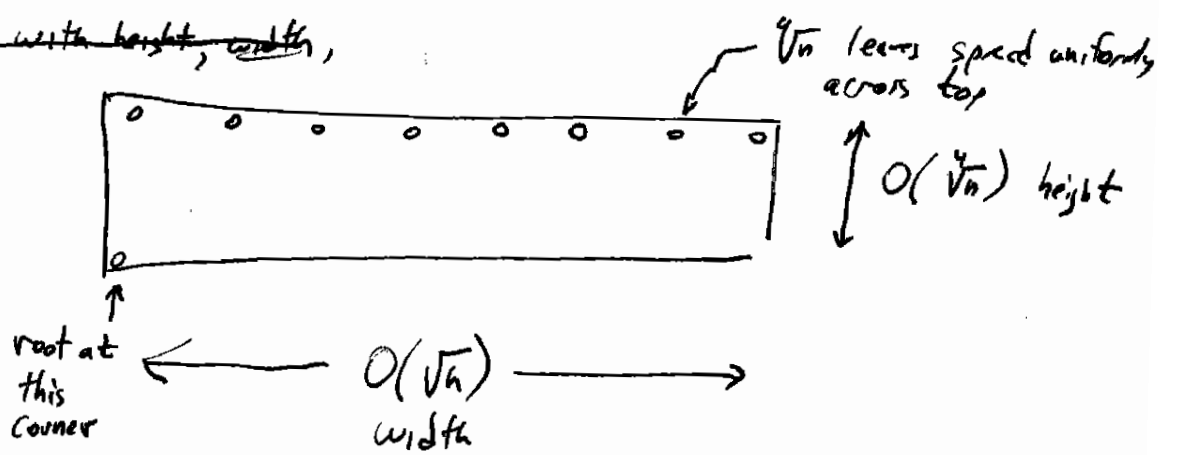
diameter of chip is $\Omega(\sqrt{n})$ (or you can't fit leaves)

$\Rightarrow$ some wire is at least $\Omega\left(\sqrt{n}/\lg n\right)$

Thm: can achieve $O(\sqrt{n}/\lg n)$

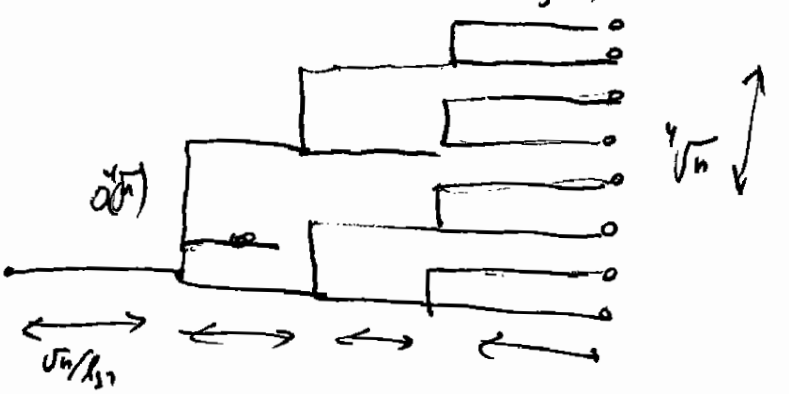
Lemma: Can layout a tree with $\sqrt[4]{n}$ leaves in this box

~~box with height, width~~,



with max. wire length $O(\sqrt{n}/\lg n)$

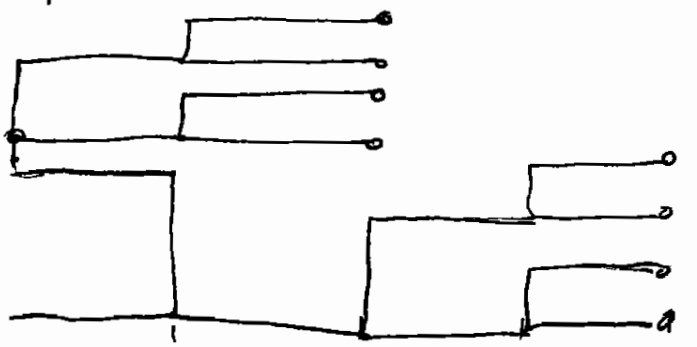
proof:



this layout has max wire length $O(\sqrt{n}/\lg n)$, and fits in the box. But the leaves are on the wrong edge

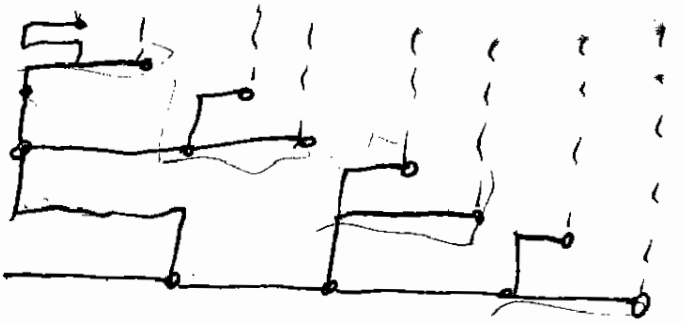
6.896
17.3
4.14.2004

Fix it up



Same height, but half the leaves moved over and some more over left.

Do it again recursively all the way down

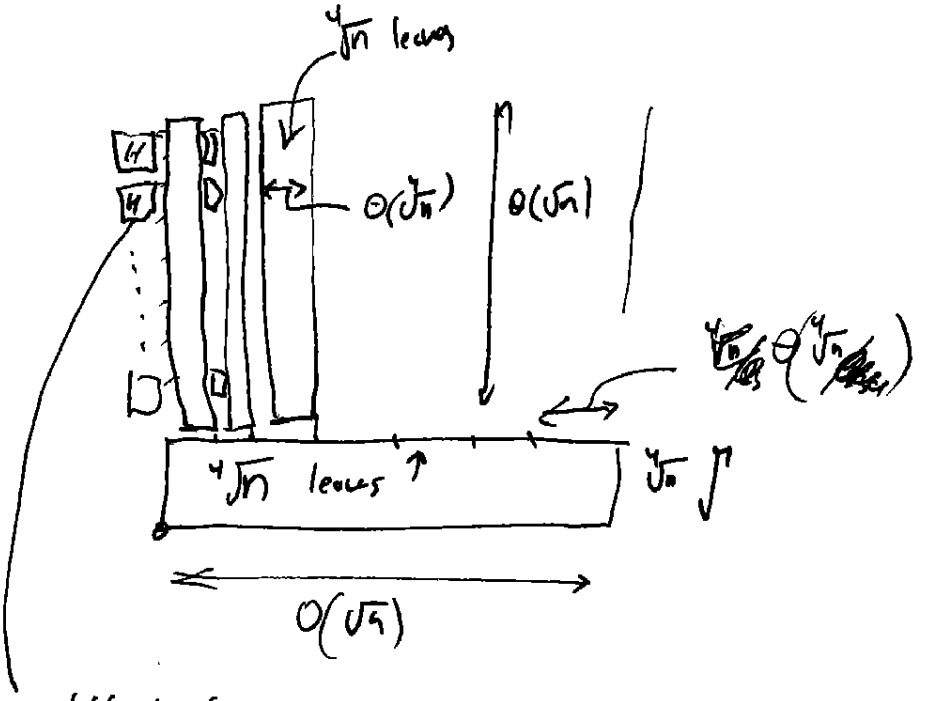


Now each leaf is in the correct column.

Simply add vertical lines to get to output



Now for proof of thm:



a little H tree containing only $\sqrt{n}$ leaves

has area $\sqrt{n}$

side length $\sqrt[4]{n}$

max wire length $\sqrt[4]{n}$.

17.5
6.896
4-14-04

Some basic layout ideas

IDEA:
Multiple Layers ~~Don't~~ Don't Matter Much.

Thm: Given a layout that uses K layers, we can reimplement the layout to use only 2 layers.

In the first layer wires go only east-west.

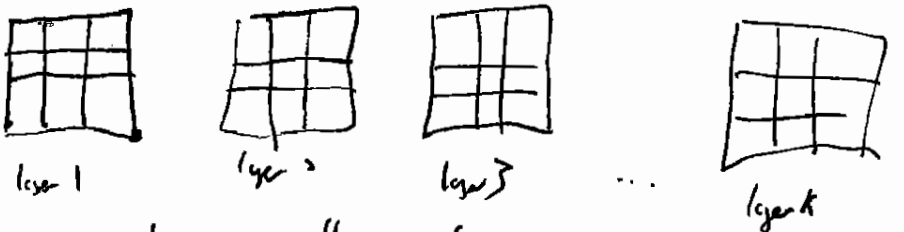
In the second layer wires go only north-south.

The side length grows by $O(k)$ in our new layout

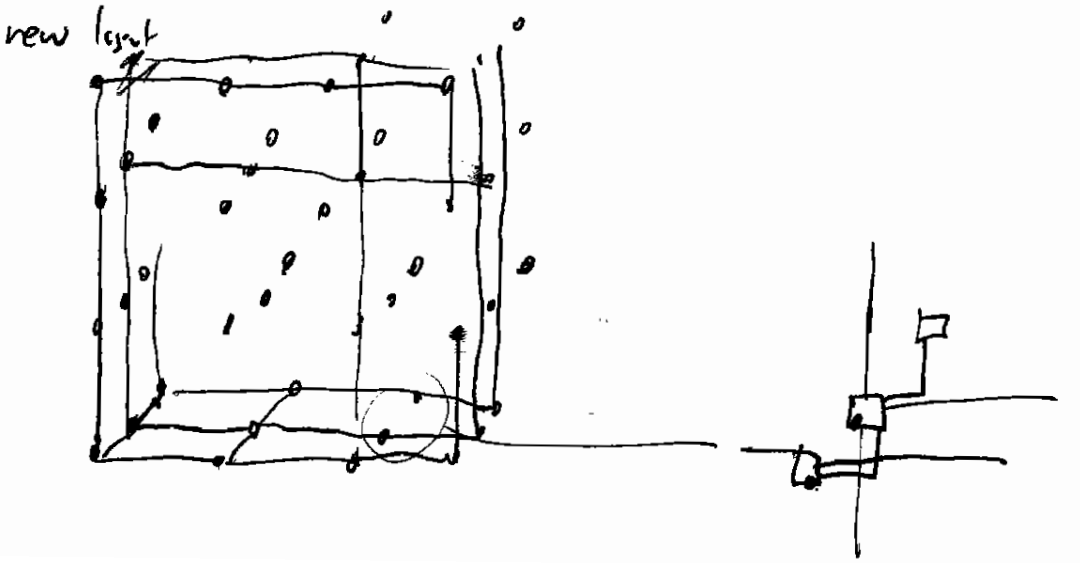
The area grows by $O(k^2)$

Proof by picture

~~Example~~.



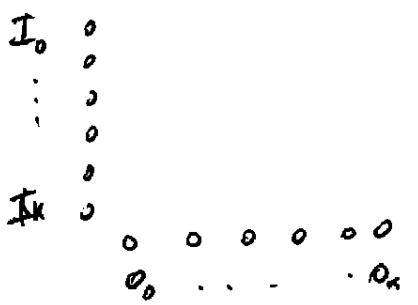
also assume all ~~connections~~ corresponding pairs connected



17.6
6.896
4-14-04

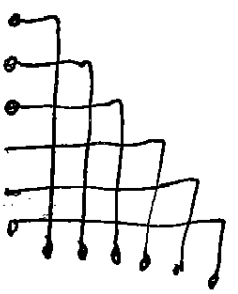
Idea: Any circuit can be made nearly square, (HW)

Idea: Turning a corner is expensive.



connect input I_i to output O_i

Here is one easy way

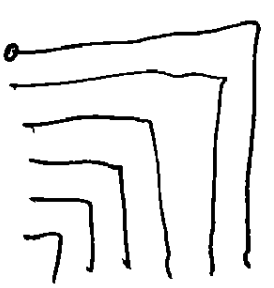


Analysis: ~~Area~~

Bounding Box area = $\Theta(k^2)$

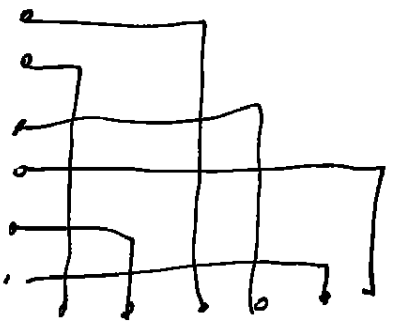
total wire length $= \Theta(k^2)$ (just following shortest path is $\Theta(k)$ per wire.

Similarly we can reverse them in area $\Theta(k^2)$



17.07
6.896
4-14-04

In fact we can perform any permutation in area $O(k^2)$

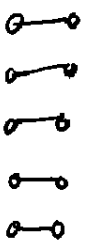


Idea: Reversing is expensive

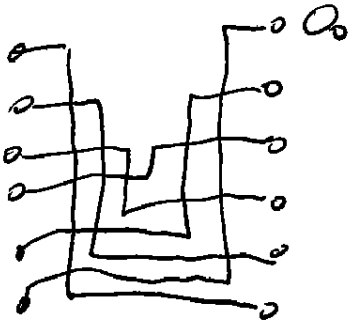
I_0 ○ ○ O_0 Connect I_0 to O_0

⋮ ⋮ ⋮

I_k ○ ○ O_k

cheap:  $O(k)$ area

But reversing is $O(k^2)$ area

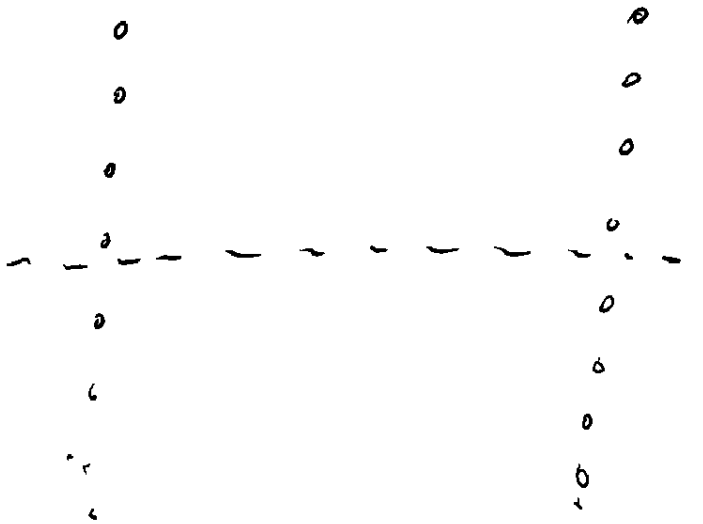
I_0  O_0

leave an extra channel in the grid

17.8
6.896
4-14-04

Thm: Reversing is area $\Omega(n^2)$ bounding box

Proof:



height is $\Omega(n)$

width? Consider cut across middle

Q: How many wires cross?

A: $\Theta(n)$ wires

$\Rightarrow$ that cut must be at $\Omega(n)$ across.

$\therefore$ area is $\Omega(n^2)$ □

Can show idea of is $\Omega(n^3)$ wire length.

17.9

6.896

4-14-04

~~Thm: Area of Beneš network is $\Theta(n^2)$~~

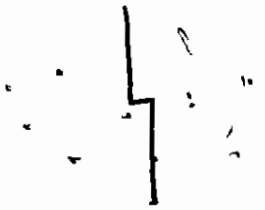
Thm: Layout of butterfly of n inputs (n log n nodes) is area $\Theta(n^2)$

proof: $\Omega(n^2)$ Count, box

bisection width argument.

Assume you layed out butterfly

There is some cut that is vertical, maybe with wires jog in it that cuts it in half.



The bisection width of butterfly is $\Theta(n)$, so

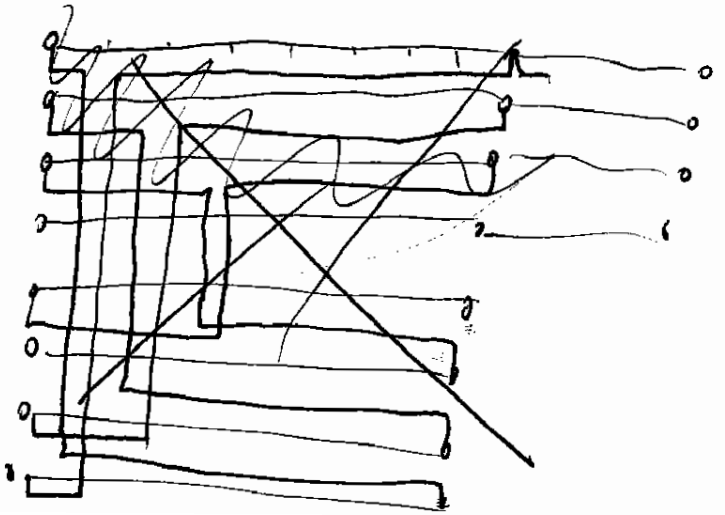
the height is $\Omega(n)$

Similarly the width.

$\Rightarrow$ $\Omega(n^2)$

~~Thm~~

proof $O(n^2)$



6.896

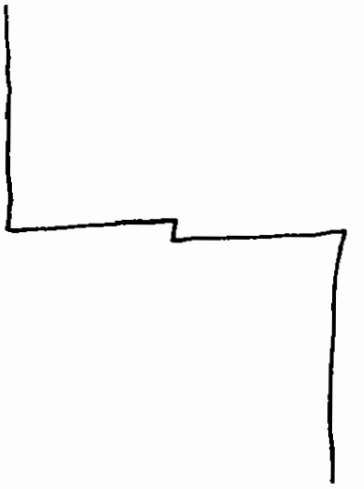
~~12.848~~ 17.10
4-14-04

To show $\Omega(n^2)$ wire area is a little harder.

consider all bisections that look like this



Need a little jog in the horizontal part to get exactly cut in half.

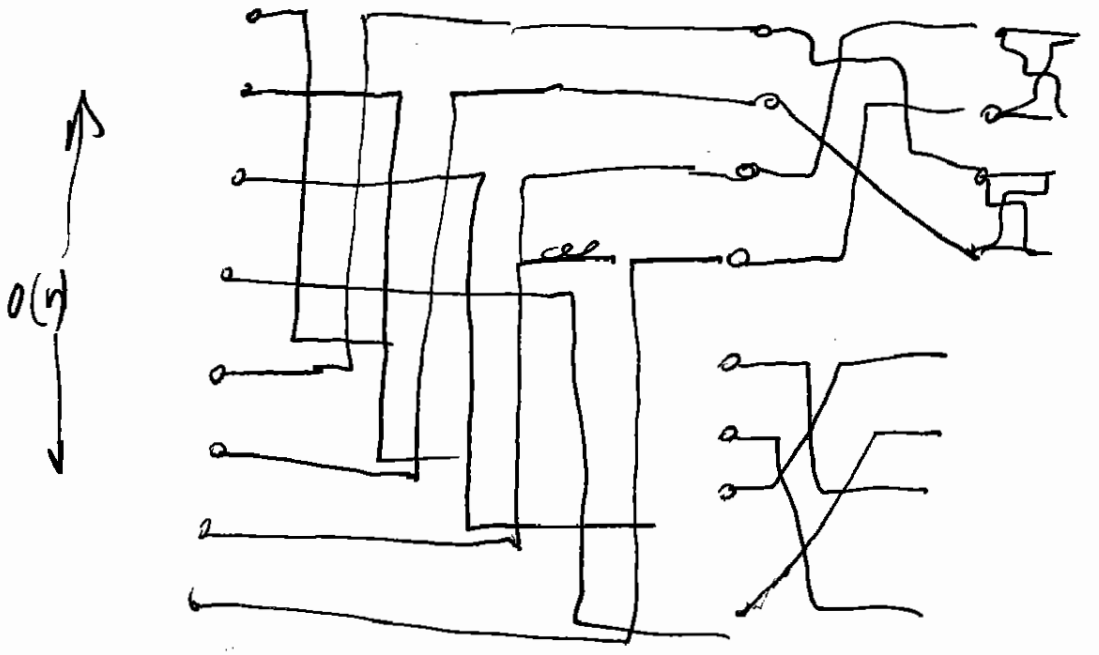


1) Each cut is $\Omega(n)$ wires
2) the main vertical parts of the cuts don't intersect
3) The # of wires crossing the main vertical parts is

$$R(n) + R(n-1) + \cdots + R(1) = \Omega(n^2)$$

17. 11
6.896
4-14-04

proof: butterfly is area $O(n^2)$



first stage is

$O(n)$ $O(n/2)$ $O(n/4)$ $\cdots$ $O(1)$

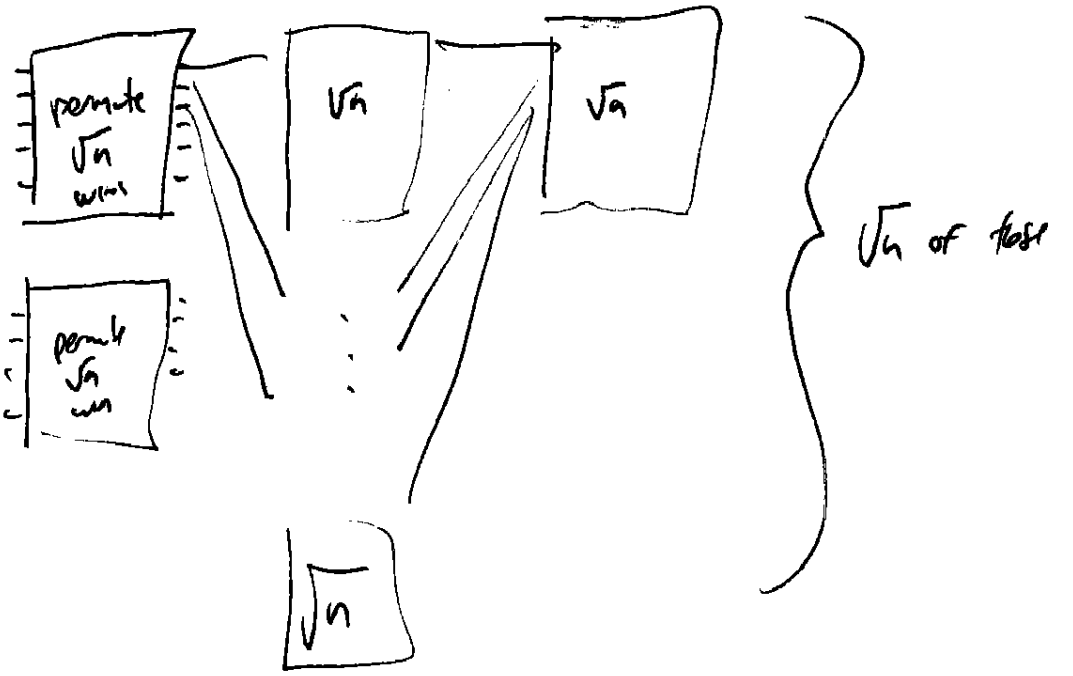
$\Rightarrow O(n)$ wide

17 March #12

Any permutation in 3D is $\Theta(n^{3/3})$

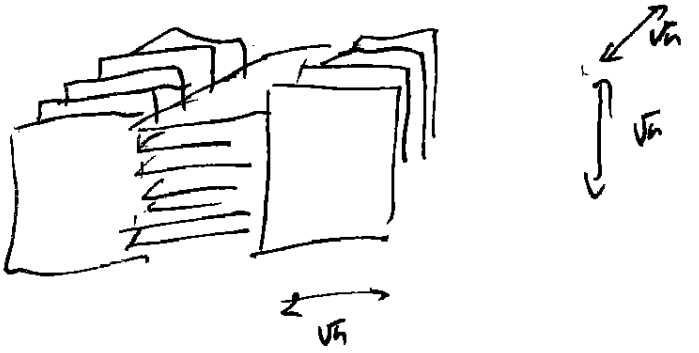
3D- similar wire model except wires take volume not area

Logical network



This is a Benes network.

3 D layout



17.13

6.896

4-14-04

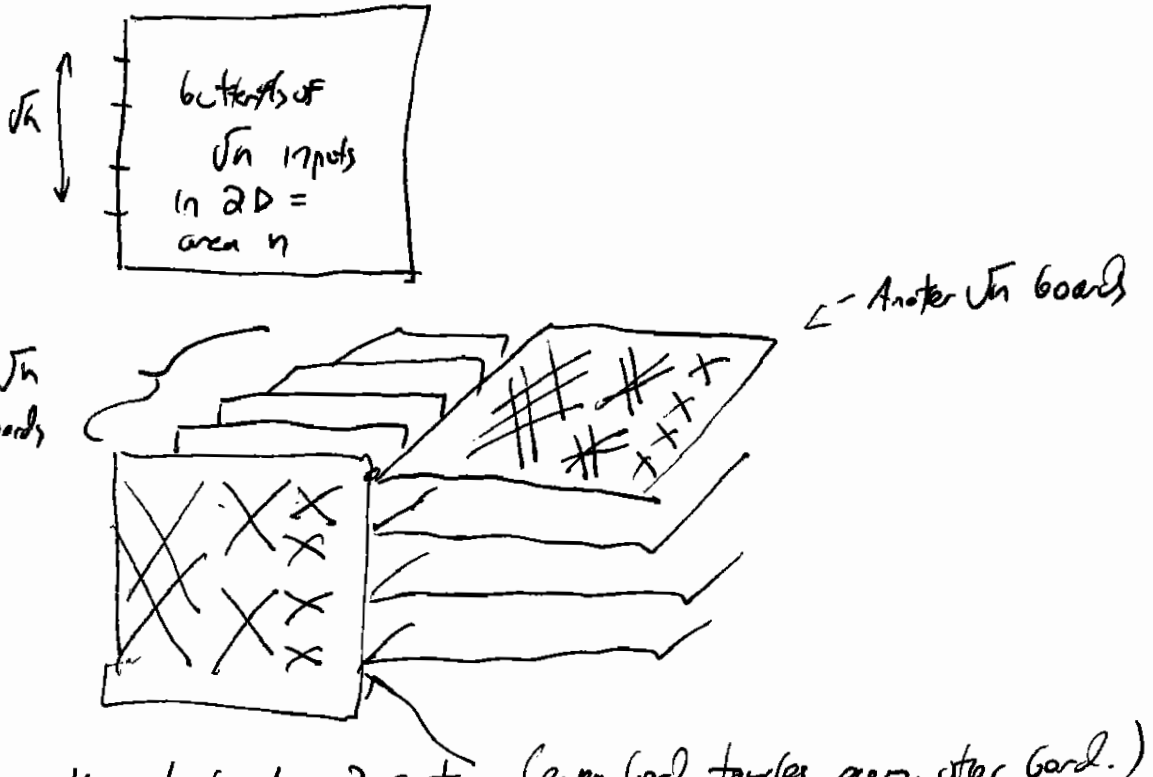
Consider a 3-D VLSI model.

wires ~~can run~~ → ↑ or ↗, but they ~~take volume~~

proportional to their bandwidth.

Butterfly layout in 3D is volume $\Theta(n^{3/2})$

Proof:



they touch at n^2 spots (every board touches every other board.)

1) It is a butterfly on n inputs

2) It has ~~area~~ volume $2 \cdot \sqrt{n} \cdot n$

2 ← area per board

$\sqrt{n}$ ← number of boards

$\Rightarrow$ volume is $O(n^{3/2})$

17. [illegible] 14

6.896

4-14-04

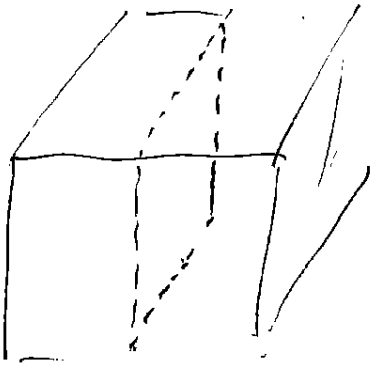
Claim volume of cube-ty is $\Omega(n^{3/2})$

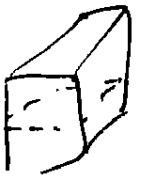
proof: Bisection argument

cut in half

that cut must cut $\Omega(n)$ wires.

So the cross section of the cut must be $\Omega(n)$





Similarly other planes cut $\Omega(n)$ wires.

does that prove bounding box is $\Omega(n^{3/2})$?

If it were square & could prove it.

~~In a HW ex~~

Can we assume it is square?

think about it.

~~The~~ Homework

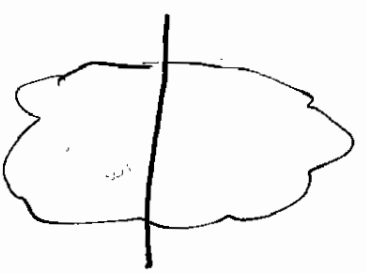
6.896
18.1 4-21-04
BRADLEY C KUSEMUL

DIVIDE-AND-CONQUER LAYOUT

~~A GENERAL ALGORITHM FOR LAYOUT PRODUCES LOW-AREA LAYOUTS FOR ANY BOUNDED DEGREE TREE OR PLANAR GRAPH, + [illegible] NEAR-OPTIMAL FOR ALL~~

A GENERAL LAYOUT ALGORITHM,
NEAR OPTIMAL LAYOUT FOR ALL GRAPHS
OPTIMAL FOR BOUNDED DEGREE TREES + PLANAR GRAPHS

~~SEPARATORS~~
~~IDEA:~~
IDEA:



LAYOUT(G):
(1) FIND SMALL NUMBER OF EDGES THAT DISCONNECT THE GRAPH INTO NEARLY EQUAL PIECES. (CAN BE TOUGH)
(2) RECURSIVELY LAYOUT THE TWO HALVES
(3) PUT THE TWO HALVES TOGETHER, + WIRE IT UP.

SEPARATORS

~~DEFN: G has an~~
DEFN: G a graph $G=(V,E)$
$S: \mathbb{Z} \to \mathbb{Z}$
G has an S-separator (i) S-separable) IF
a) G has 1 vertex, or
b) ~~Let G = [illegible] |V|~~
$\exists$ a set $A \subseteq E$ s.t. $|A| < S(|V|)$
and $(V, E-A)$ is two disconnected graphs

~~G_1, G_2~~ $G_1 = (V_1, E_1)$
$G_2 = (V_2, E_2)$
s.t. $|V_1| \geq |V|/3$ and $|V_2| \geq |V|/3$
[neither G_1 nor G_2 has $> \frac{2}{3}$ of the nodes of the other]
and
G_1 + G_2 are S-separable.

6.896
18.2 4-21-07

~~Example~~:

Thm: Binary trees are 1-separable.

proof: pick a root
travel down the tree looking for a node that is the ancestor of between $n/3$ to $2n/3$ nodes.

~~that node's parent else disconnects the tree~~



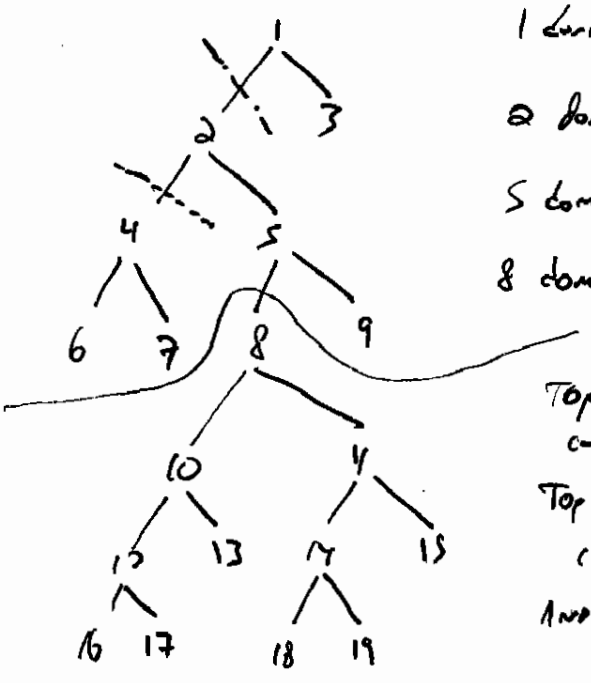
case A: this subtree is twice $\frac{n}{3} + \frac{2n}{3}$ nodes
done

case B: ~~Subtree to small. cut~~
Subtree too big. $> \frac{2n}{3}$ nodes

one of two children is at least half the nodes,
so go that path

case C: too small. cut don't go there.

Example..



1 dominates 19 nodes - too big

2 dominates 17 nodes too big

5 dominates 13 nodes, too big

8 dominates 10 nodes - OK

Top half is 8 nodes
cut here ..... into 3 + 5

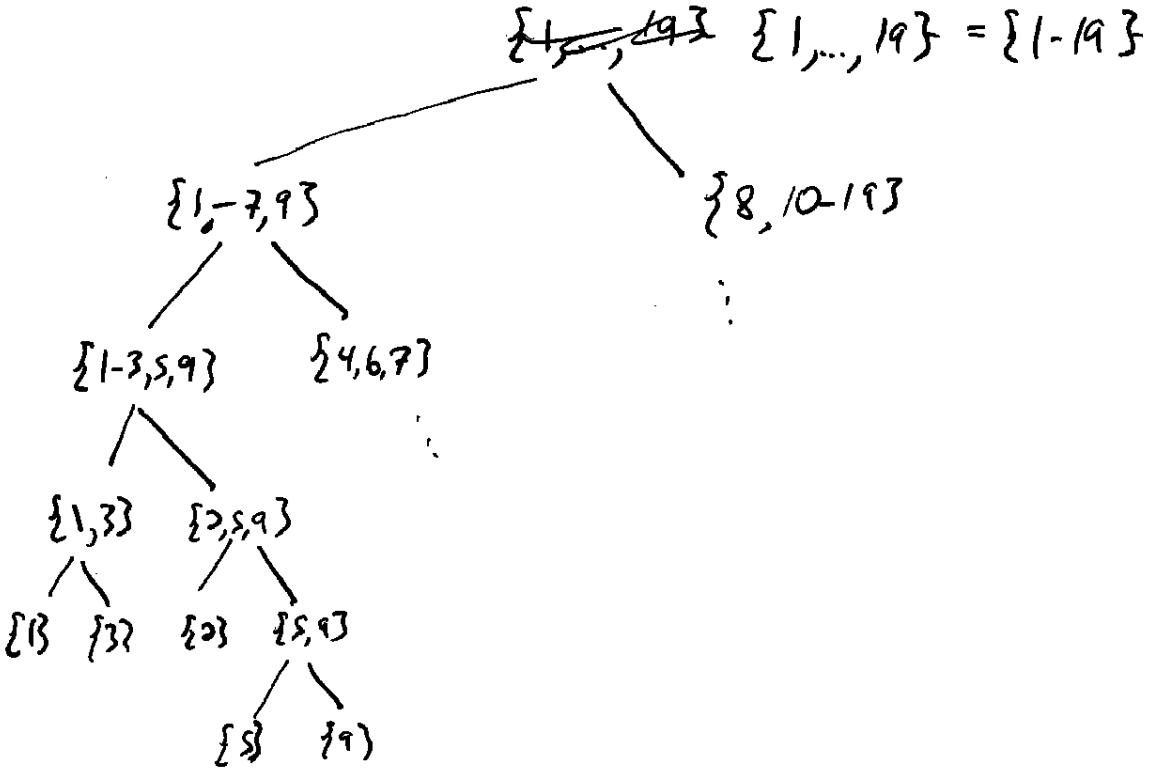
Top 1/4 in 5 nodes
cut here —·—·—·—

AND SO FORTH

SAVE THIS TREE

~~6.8.4~~
6.896
18.3 4-21-04

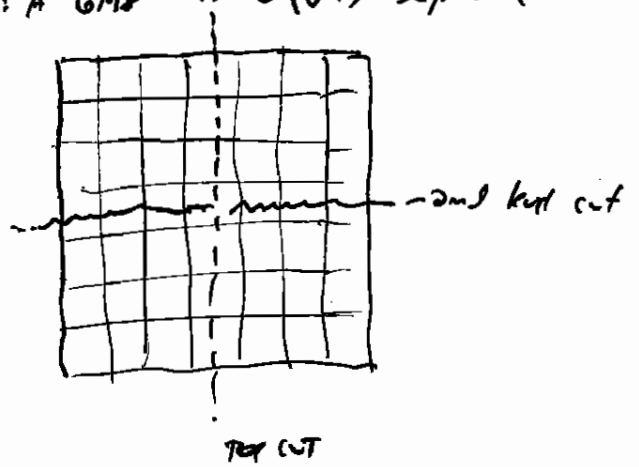
Defn:
A partition tree of a GRAM is a ~~tree where each~~
Defined by example:



[its a tree even if G is not a tree]

6.896
12.4
4-21-04

Example: A GRID is $O(\sqrt{n})$ separable



Defn: G has a strong S-separator if the sizes of the subgraphs are at most $\frac{|V|+1}{2}$.

[cut exactly in half if even, otherwise a close as possible]

Example: a grid of size $2^i \times 2^i$ is strongly $\sqrt{n}$ separable

Claim: not surprising. We'll show S is $\Omega(n^c)$ implies S-separable $\Rightarrow$ strongly separable

Defn: Γ (gamma) defined as

$$\Gamma_S(n) = S(n) + S\left(\tfrac{2}{3}n\right) + S\left(\tfrac{4}{9}n\right) + \cdots$$

$$= \sum_{i=0}^{\lceil \log_{3/2} n \rceil} S\left(\left(\tfrac{2}{3}\right)^i n\right)$$

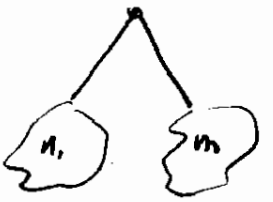
Example: $S(n) = n^\alpha$ then

$$\Gamma_S(n) = n^\alpha + \left(\tfrac{2}{3}n\right)^\alpha + \left(\tfrac{4}{9}n\right)^\alpha + \cdots = n^\alpha \cdot \left(1 + \tfrac{2}{3}^\alpha + \tfrac{4}{9}^\alpha + \cdots\right)$$

$$= n^\alpha \cdot \frac{1}{1-\left(\frac{2}{3}\right)^\alpha} = O(n^\alpha)$$

6.890
18.6 4-21-04

proof by induction: pick $t < |V|$

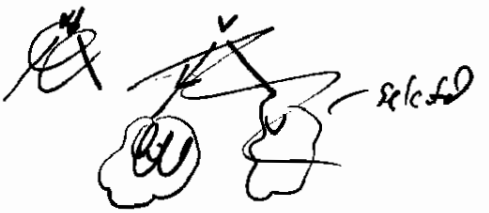


if $n_1 \leq t$ then add the left subtree to our selected set + go right to pick $t - n_1$ elts.

~~if $n_1 + n_2 \geq t$ then pick one side~~
if $n_1 > t$ then go left
don't use the right subtree at all
pick n_1 elts from left.

claim: ~~the edges connecting our selected sets to anything else~~
~~[illegible]~~ $\leq \Gamma_S(u)$ edges connecting selected sets to anything else.

pf.



~~if u is selected then at most $S(u)$ edges connect u to~~ ~~non-selected set~~ ~~anything else~~

~~if u is not selected then at most $S(u)$ edges connect u to the other~~
~~u to the selected set.~~

~~if v is~~



If ~~u~~ u is selected then at most $S(u)$ of the edges to u are ~~needed~~ to connect u to non-selected nodes.

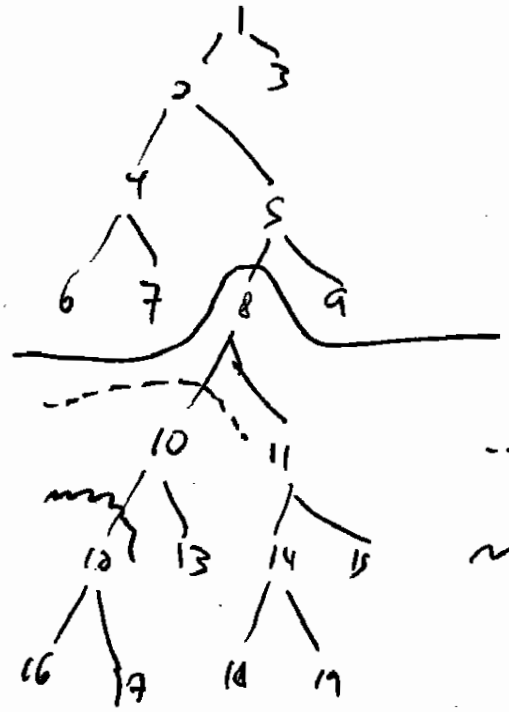
If u is not selected, then at most $S(u)$ of these edges are needed to connect u to the selected set.

The total number is then no more than $S(u) + S(\frac{2}{3}u) + S(\frac{1}{3}u)$

$= \Gamma_S(u)$.

6.896
18.7
4-21-04

Example: Binary trees are 1-separable $\Rightarrow$ they are strongly $\log_{\frac{1}{2}}$-separable.



~~Break into 10 + 9 nodes,~~

Select 10 nodes

Select top 6 if + need to select 2 nodes

----- is too big, select neither

~~~ is just right

~~3 nodes cut~~

3 edges cut if first 10 nodes

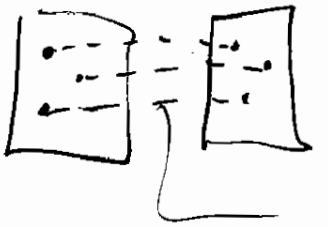
$\Gamma_1(19) = \lceil \log_{\frac{2}{3}} 20 \rceil = 8$ so we did ok.
~~~

6.896
18.8
4-21-04

Back to layout algorithm
1) Separate
2) recurse
3) reassemble



cut in 1/2 with separator
did recursive layout of G1/G2
now what?

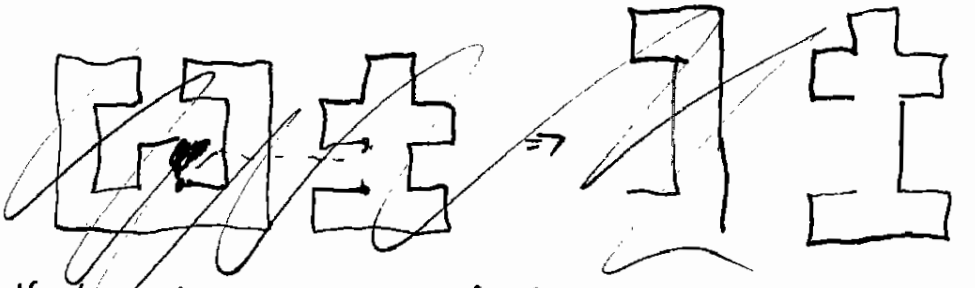


need to connect free edges.

e.g. ~~there~~ is a tree



May have flow recursive layout



insertChannels stretch layout vertically to make space
leave a space between for vertical routing

6.896

18.S 4-21-04

Example: $S(n) = O(1)$ then

$$\Gamma_S(n) = O(\log n)$$

Example: $S(n) = \lg n$ then

$$\begin{aligned}\Gamma_{\lg} n &= \lg n + \lg \tfrac{3}{2} n + \lg \tfrac{4}{3} n + \cdots \\ &= \lg n + \Gamma_{\lg}(\tfrac{3}{2} n) \\ &= \lg^2 n\end{aligned}$$

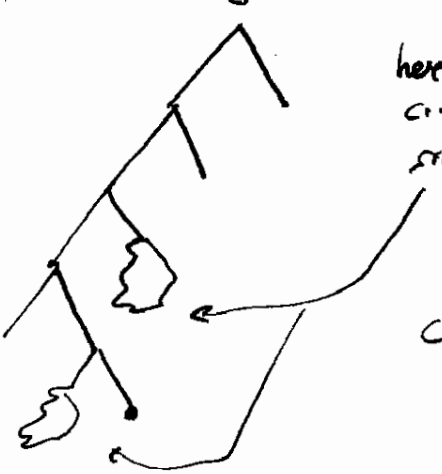
Lemma: if G is S-separable then G is strongly Γ_S separable.

~~Proof: for any $t \le |V|$ $\exists$ a path in the partition tree~~

Proof. Build a partition tree for G achieving S-separation

For any $t < |V|$ we will find ~~a collection of nodes in the partition tree~~ a path from the root of the partition tree to a leaf, & some subset of the siblings add up to exactly t nodes

e.g.

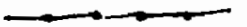


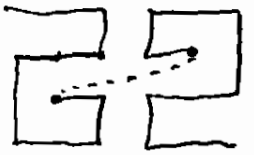
here is the P.T. and we can take any collection of the siblings. pick these two to get exactly t

call these the selected sets

6896
18.9
4-21-04

Example: layout of a linear array.

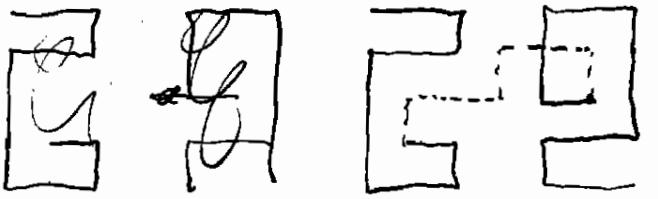




↑ two recursive subs
---- need to connect

stretch vertically to bring ---- to eq

stretch horizontally between the halves



(n a power of 4)

Defn $\Delta_S(n) = S(n) + 2S(n/4) + 4S(n/16) \cdots$

$= S(n) + 2\Delta_S(n/4)$

example:

[State as facts w/o proof]

$S(n) = n^\alpha$

$\Delta_S(n) = n^\alpha + 2\left(\frac{n}{4}\right)^\alpha + 4\left(\frac{n}{16}\right)^\alpha =$

- $\Delta_{n^\alpha}(n) = O(\sqrt{n})$ if $\alpha < \frac{1}{2}$
- $\Delta_{n^\alpha}(n) = O(n^\alpha)$ if $\alpha > \frac{1}{2}$
- $\Delta_{n^{1/2}}(n) = O(\sqrt{n}\lg n)$

If $\alpha < \frac{1}{2}$ then the tops grow faster than the bottoms

so $\Delta_S(n) = O(\sqrt{n})$

If $\alpha > \frac{1}{2}$ then $\Delta_S(n) = O(n^\alpha)$

If $\alpha = \frac{1}{2}$ then $\Delta_S(n) = \Delta_{n^{1/2}}(n) = O(\sqrt{n}\lg n)$

6.896
18.10
4-21-04

Thm: $S(n)$ monotonically non decreasing

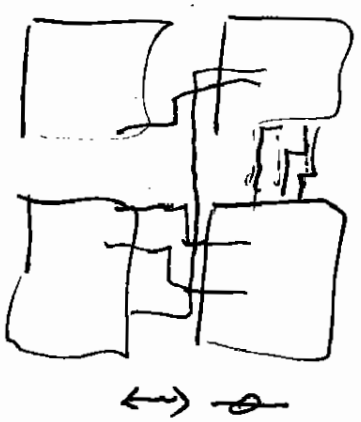
A graph of n vertices with a strong ~~$S(n)$-separator~~ S-separator can be laid out in a square with side length $O(\max(\sqrt{n}, \Delta_S(n)))$

Induction on n, assume n a power of 4

Claim side length of $\sqrt{n} + 6\Delta_S(n)$ good enough

base case: easy 1 node

induction: divide in half & half again



$$W(n) = 2\left(\frac{\sqrt{n}}{2} + 6\,\Delta_S(n/4)\right) + S(n)$$

but $\Delta_S(n) = S(n) + 2\Delta_S(n/4)$

$$= \sqrt{n} + 6\Delta_S(n/4) \qquad \square$$

$$H(n) = S(n) + S(n/2) + 2H(n/4) = O(S(n)) + H(n/4)$$

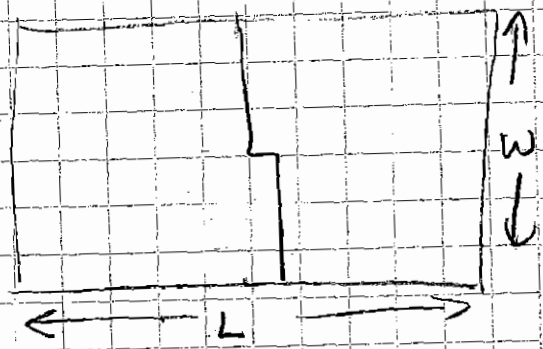
Analysis: $S(n) = O(n^\alpha)$ for $\alpha < \frac{1}{2}$: $H(n) = O(\sqrt{n}) + H(n/4) = O(\sqrt{n} + \sqrt{\frac{n}{4}} + \sqrt{\ }$

6.896
4/26/04
L19.1

VLSI lower bounds

Lemma A network with bisection width B has area $\Omega(B^2)$.

Pf.



$B \le W+1 \Rightarrow A = LW \ge W^2 \ge (B-1)^2 = \Omega(B^2)$ ☒

Good recursive bisection ⇒ small area.

Bisection width lower bounds on computation

Shifting:

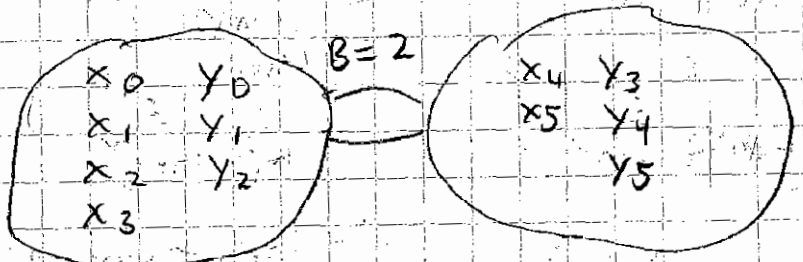
Input: $x_0, x_1, \ldots, x_{n-1}$ data
$S \in \{0, \ldots, n-1\}$ control

Output: $y_0, y_1, \ldots, y_{n-1}$ & $y_i = x_{(i-S) \bmod n}$

Network	$T(n)$	$B(n)$	$A(n)$
Linear array	$O(n)$	$\Theta(1)$	$\Theta(n)$
Tree	$O(n)$	$\Theta(1)$	$\Theta(n)$
$\sqrt{n} \times \sqrt{n}$ mesh	$O(\sqrt{n})$	$\Theta(\sqrt{n})$	$\Theta(n)$
n-input butterfly	$O(\lg n)$	$\Theta(n)$	$\Theta(n^2)$

Let $B(n)$ = # edges cut to bisect outputs.
If $I(n)$ = worst-case #bits to cross this bisection, then $B(n)T(n) \ge I(n)$.

Ex.



$S=0 \Rightarrow$ 1 bit crosses bisection (L to R)
$S=3 \Rightarrow$ 3 bits cross — worst case $\Rightarrow I=3, T \ge B/I = 3/2$.

6.896
4/26/04
L19.2

How do we know 3 bits must cross?
Might there be a clever encoding?

"Fooling argument"

Ex. Sup. we have following bisection:

x_0	y_0	$x_{n/2}$	$y_{n/2}$
x_1	y_1		
$x_{n/2-1}$	$y_{n/2-1}$	x_n	y_n

$S = n/2 \Rightarrow$ intuitively $n/2$ bits must cross (L to R)

Claim: $B(n) \cdot T(n) \geq n/2$.

Pf. (fooling arg.) Sup. $B(n)T(n) < n/2$

communication patterns on $B(n)$ wires (L to R) over time $T(n) = 2^{B(n)T(n)} < 2^{n/2}$

values for $x_0, \ldots, x_{n/2-1} = 2^{n/2}$.

$\therefore$ $\exists$ 2 distinct $x_0', \ldots, x_{n/2-1}'$ and $x_0'', \ldots, x_{n/2-1}''$ that produce identical comm patterns.

RHS of circuit can't distinguish $\Rightarrow$ produces same values for $y_{n/2}, \ldots, y_n$ for both $\Rightarrow$ must operate wrong for one. Contradiction. ☒

Thm. For any bisection of outputs, $B(n)T(n) \geq n/2$

Pf. Consider arb. bisection

Ex.

y_0 y_2 y_4	x_0 x_1 x_2 x_4 x_5
y_1 y_3 y_5	x_3

Make an $n \times n$ table:

s	x_0	x_1	x_2	x_3	x_4	x_5
0		X				X
1	X		X	X	X	
2		X				X
3	X		X	X	X	
4		X				X
5	X		X	X	X	

X if shift of s causes x_i to cross bisection

Every column contains $n/2$ X's.
∴ Average # X's per row = $n/2$.
⇒ some row contains $\geq n/2$ X's.
(some shift causes $n/2$ bits to cross). ☒

Network	$B(n)$	$T(n)$
Linear array	$\Theta(1)$	$\Omega(n)$
Tree	$\Theta(1)$	$\Omega(n)$
$\sqrt{n} \times \sqrt{n}$ mesh	$\Theta(\sqrt{n})$	$\Omega(\sqrt{n})$
n-input butterfly	$\Theta(n)$	$\Omega(1)$

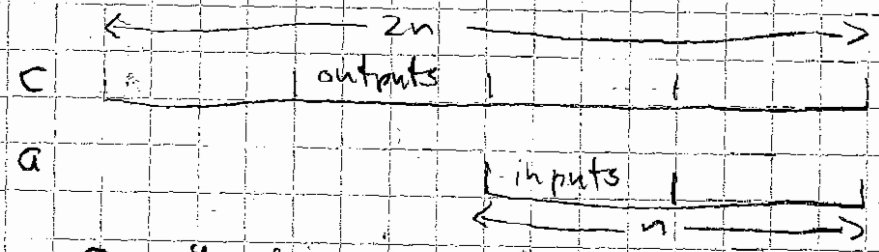
6.896
4/26/04
L19.4

Theorem Any circuit for shifting n bits has $AT^2 = \Omega(n^2)$

Pf. $AT^2 = \Omega(B^2T^2)$
$= \Omega(n^2)$ ☒

Theorem Any circuit for multiplying two n-bit #'s has $AT^2 = \Omega(n^2)$.

Pf. $c = ab$. Let b take on powers of 2.
Essentially corresp. to shift problem.



Consider bisection of outputs.
Build shift/input matrix as before: $n \times n/2$

	$a_{n/2}$	$a_{n/2+1}$	...	a_n
0		x	x	
1	x			x
s			x	
		x		x
n-1	x			

$n/4$ x's per column

$n^2/8$ x's in matrix
$\Rightarrow$ some row has $n/8$ x's.
$\therefore BT \geq \Omega(n)$ ☒

Also, FFT, convolution, sorting, routing, etc.

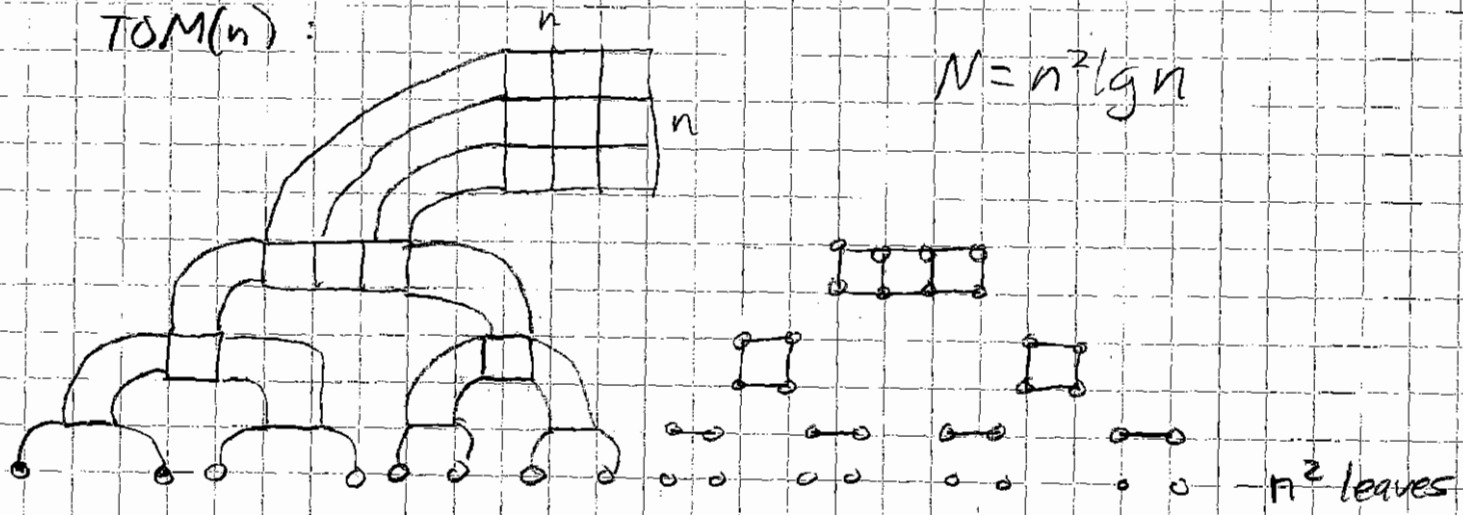
Exam: In class on May 12. Prob. session May 10.

6.896
4/28/04
L20.1

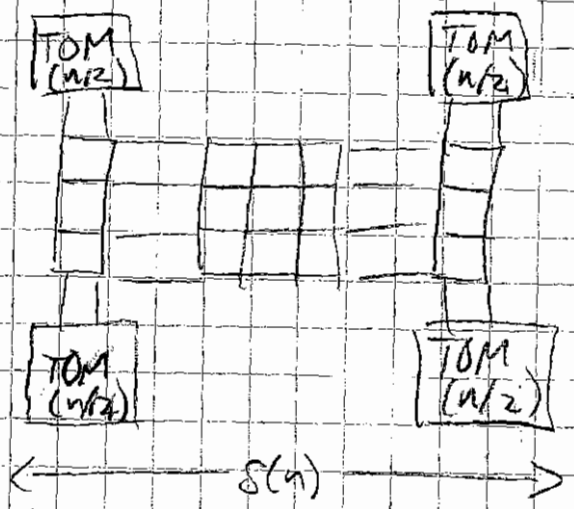
General Layout Strategy

Tree of meshes (not mesh of trees)

TOM(n):



Area:

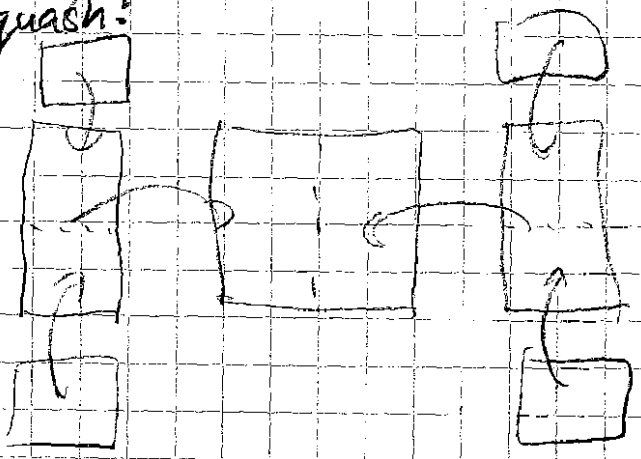


$$S(n) = 2S(n/2) + n$$
$$= \Theta(n \lg n)$$

$$A(n) = \Theta(n^2 \lg^2 n)$$

6.896
4/28/04
L20.2

Fold and squash:



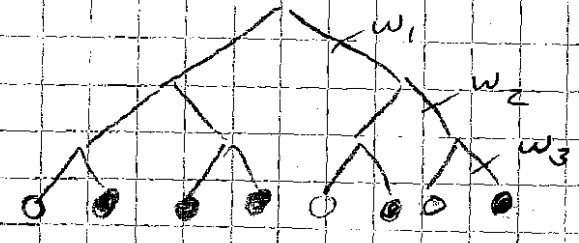
$n^2 \times \lg n$ layers $\xrightarrow{\text{squash}}$ $\Theta(n^2 \lg^2 n)$ area.

Truncated TOM: $TOM(n,k)$ – top k levels.
Area $= \Theta(n^2 k^2)$

Decomposition trees

T is a $\langle w_1, w_2, \ldots, w_r \rangle$ decomposition tree for $G=(V,E)$:

1. Vertices in V mapped to leaves of T.
2. Edges in E run through links of T.
3. # edges leaving subtree rooted at depth i is $\leq w_i$



For $1 < \alpha \leq 2$, G has a (w,α) decomp tree if it has a $\langle w, w/\alpha, w/\alpha^2, \ldots, O(1) \rangle$ decomp tree.

A decomp tree is balanced if all subgraphs at the same depth have same # vertices to within 1.

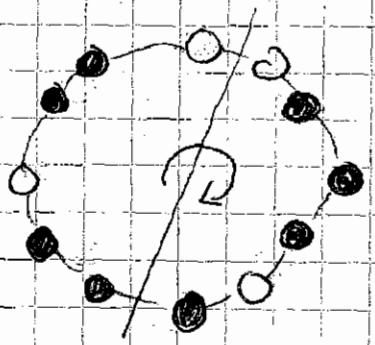
6.896
4/28/04
L20.3

Layout strategy

1. Start with $(w, \sqrt{2})$ decomp tree.
2. Balance the decomp tree
3. Embed the balanced tree in trunc TOM
4. Use trunc TOM layout to yield $O(w^2 \lg^2 n)$ area layout.

Balancing decomp trees

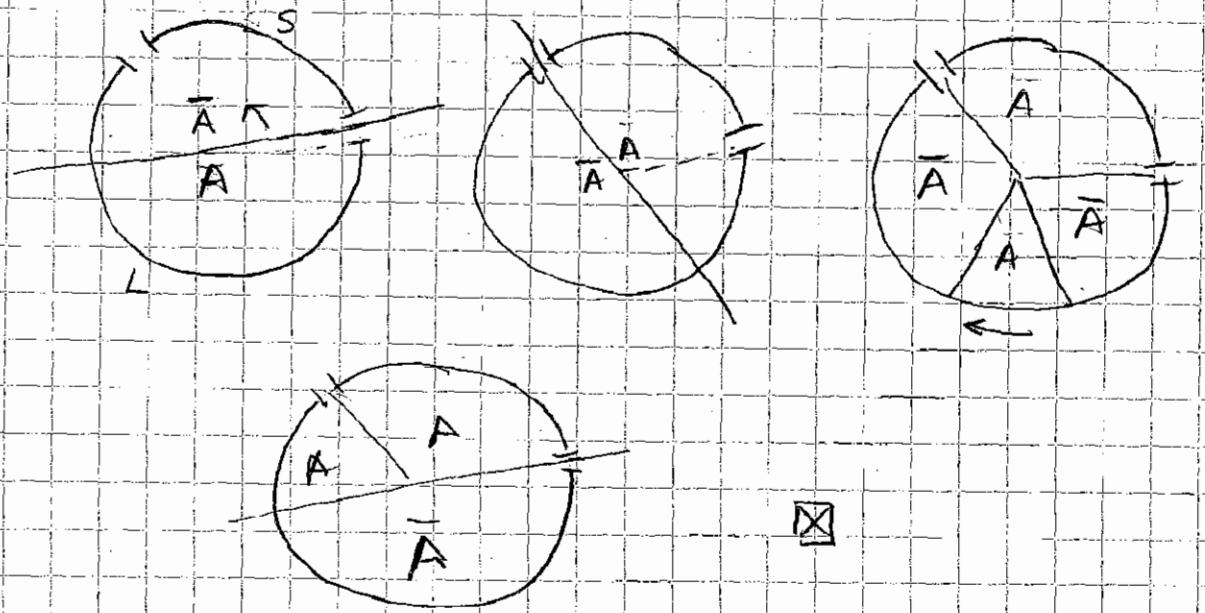
Warm-up: Necklace with black and white pearls.
How many cuts to divide into 2 sets, each with half the pearls of each color?



2 cuts suffice.
Continuity argument.

Lemma. Consider any 2 strings composed of an even # of black pearls and an even # of white pearls. By making at most 2 cuts, the pearls can be partitioned into 2 sets, each containing 2 strings, such that each set has 1/2 the pearls of each color.

Pf. (Continuity arg.)

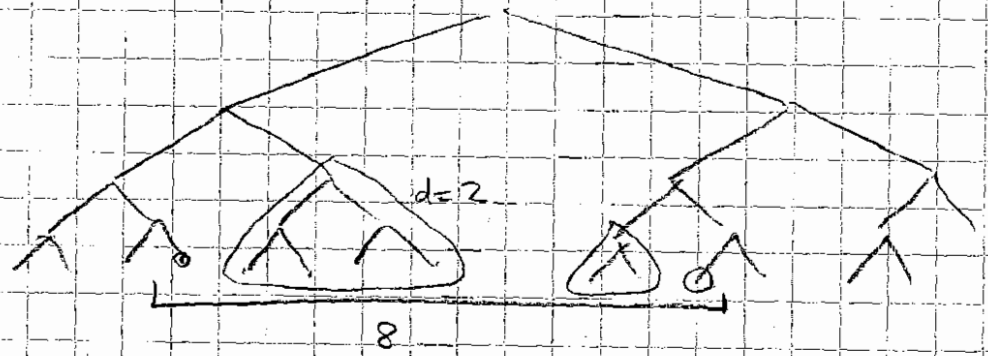


□

6.896
4/28/04
L20.4

Lemma. Let T be cbt drawn with n leaves on a straight line, and consider any set S of k consecutive leaves of T. Then, $\exists$ a forest F of complete binary subtrees of T s.t.

1. $S = \{\text{leaves of } F\}$
2. at most 2 trees of F have any given height.
3. depth of largest tree in F is $\le \lg k$

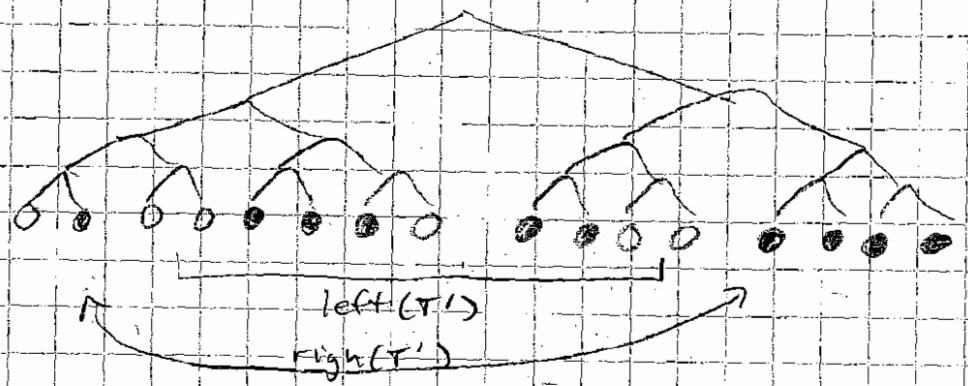


Pf. F be forest of maximal cbt's whose leaves lie only in S. (1) & (3) follow. Use induction to prove (2). ☒

Thm. Let G be a graph on n vertices that has a $\langle w_1, w_2, \ldots, w_r \rangle$ decomp tree T. Then, G has a $\langle w_1', w_2', \ldots, w_{\lceil r/4 \rceil}' \rangle$ balanced decomp tree T', where

$$w_i' = 4 \sum_{k=i}^{r} w_k .$$

Pf. Color leaves of T: 1 = node of G, 0 = empty.



Recursively split B & W leaves evenly. Each stage has ≤ 2 strings of consec. leaves from T, each of which has ≤ 2 cbt's of a given height.

6.896
4/28/04
L20.5

Total # wires leaving a string
≤ sum of wires leaving each of its cbt's.

$$w_i' \le 4 \sum_{k \ge i}^{r} w_k. \quad \boxtimes$$

Corollary A graph with a (w, α) decomp tree, α const, has an $(O(w), \alpha)$ balanced decomp tree.

Pf. Sum is geometric:

$$w_i' = 4 \sum_{k \ge i}^{r} w_k$$

$$\le 4 \sum_{k \ge i}^{r} \frac{w}{\alpha^{k-1}}$$

$$\le \frac{4w}{\alpha^{i-1}} \left(\frac{\alpha}{\alpha - 1} \right).$$

Graph has $(4w\alpha/(\alpha-1), \alpha)$ decomp tree. $\boxtimes$

Next week: Embed in trunc TOM $\Rightarrow$ layout.
Area-universal networks.

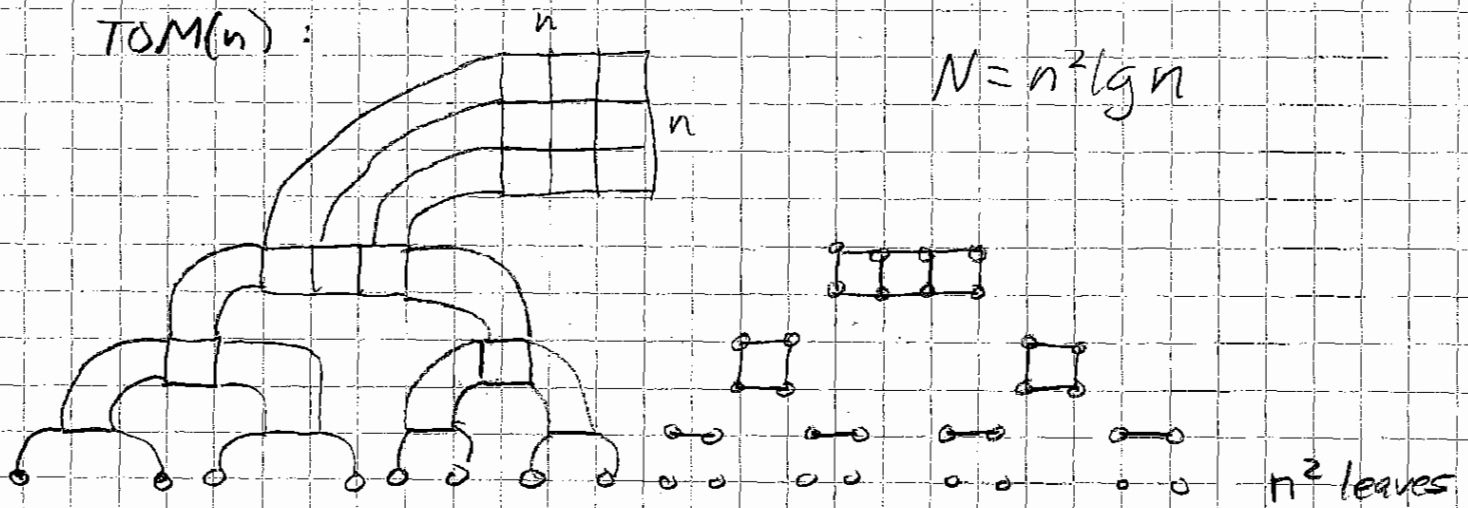
«Exam issues»

6.896
4/28/04
L20.1

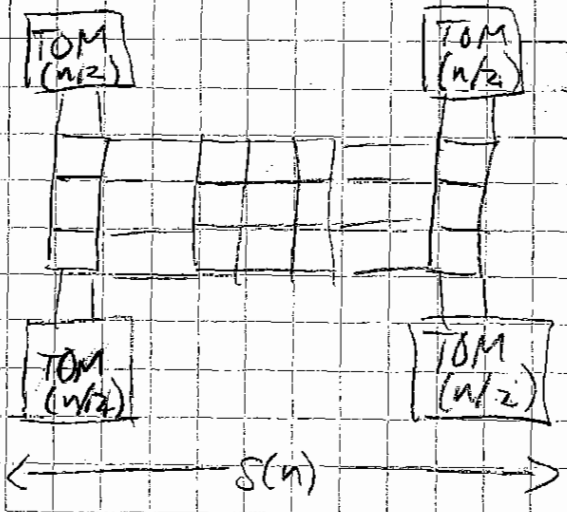
General Layout Strategy

Tree of meshes (not mesh of trees)

TOM(n):



Area:

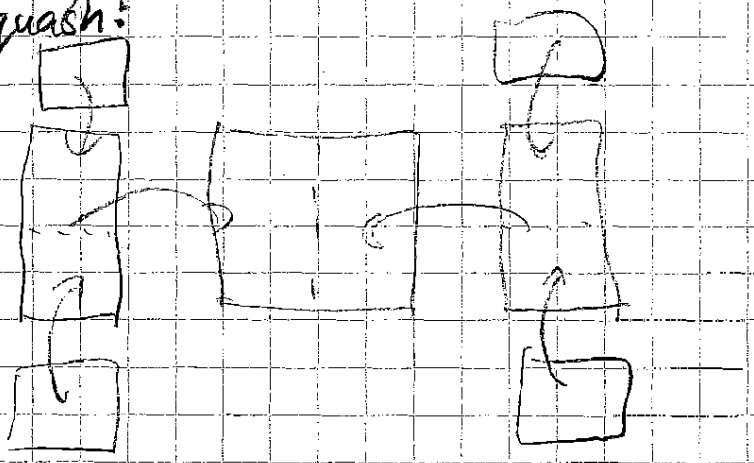


$$S(n) = 2S(n/2) + n$$
$$= \Theta(n \lg n)$$

$$A(n) = \Theta(n^2 \lg^2 n)$$

6.896
4/28/04
L 20.2

Fold and squash:



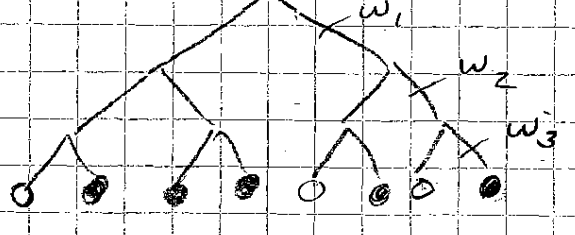
$n^2 \times \lg n$ layers $\xrightarrow{\text{squash}}$ $\Theta(n^2 \lg^2 n)$ area.

Truncated TOM: $TOM(n,k)$ – top k levels.
Area $= \Theta(n^2 k^2)$

Decomposition trees

T is a $\langle w_1, w_2, \ldots, w_r \rangle$ decomposition tree for $G=(V,E)$:

1. Vertices in V mapped to leaves of T.
2. Edges in E run through links of T.
3. # edges leaving subtree rooted at depth i is $\le w_i$



For $1 < \alpha \le 2$, G has a (w,α) decomp tree if it has a $\langle w, w/\alpha, w/\alpha^2, \ldots, O(1) \rangle$ decomp tree.

A decomp tree is balanced if all subgraphs at the same depth have same # vertices to within 1.

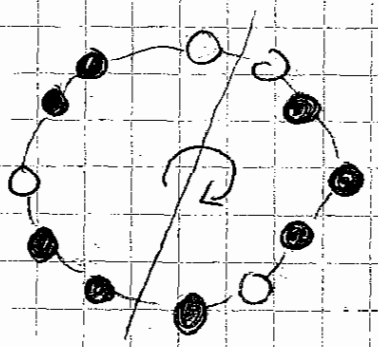
6.896
4/28/04
L20.3

Layout strategy

1. Start with $(w, \sqrt{2})$ decomp tree.
2. Balance the decomp tree
3. Embed the balanced tree in trunc TOM
4. Use trunc TOM layout to yield $O(w^2 \lg^2 n)$ area layout.

Balancing decomp trees

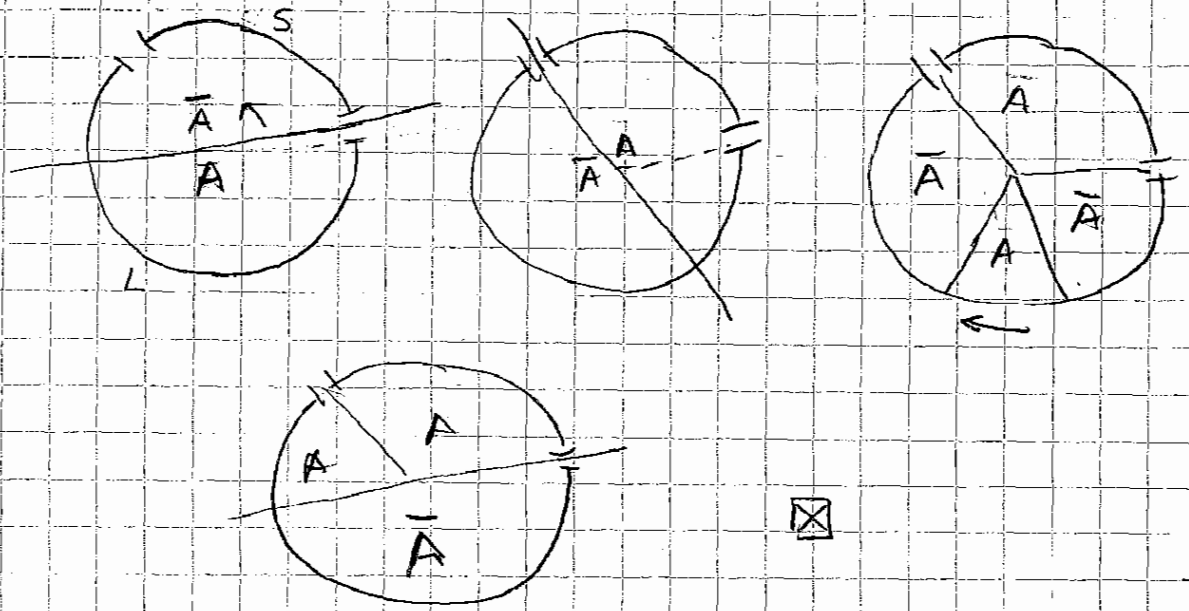
Warm-up: Necklace with black and white pearls. How many cuts to divide into 2 sets, each with half the pearls of each color?



2 cuts suffice. Continuity argument.

Lemma. Consider any 2 strings composed of an even # of black pearls and an even # of white pearls. By making at most 2 cuts, the pearls can be partitioned into 2 sets, each containing 2 strings, such that each set has 1/2 the pearls of each color.

Pf. (Continuity arg.)

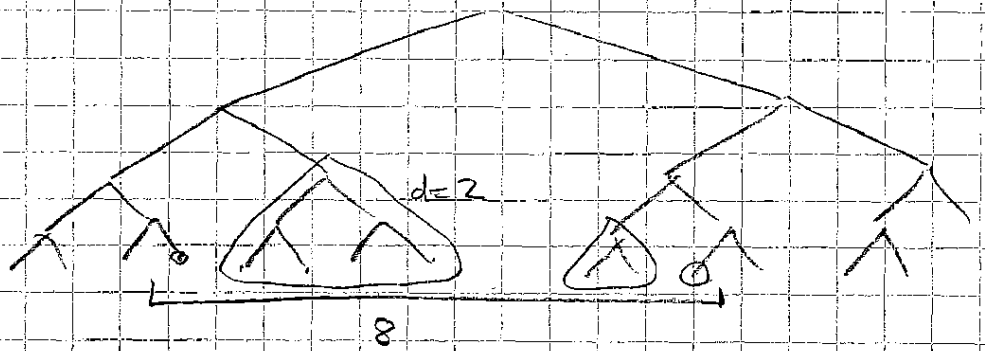


□

6.896
4/28/04
L20.4

Lemma. Let T be cbt drawn with n leaves on a straight line, and consider any set S of k consecutive leaves of T. Then, $\exists$ a forest F of complete binary subtrees of T s.t.

1. $S = \{\text{leaves of } F\}$
2. at most 2 trees of F have any given height.
3. depth of largest tree in F is $\le \lg k$.

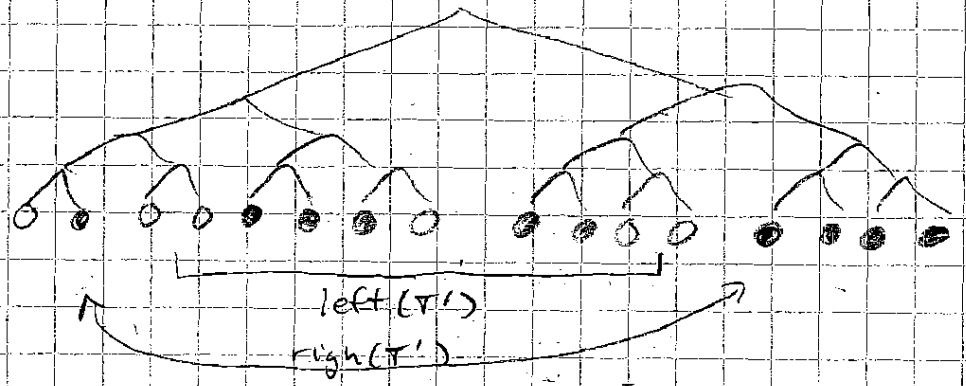


Pf. F be forest of maximal cbt's whose leaves lie only in S. (1) & (3) follow. Use induction to prove (2). ☒

Thm. Let G be a graph on n vertices that has a $\langle w_1, w_2, \ldots, w_r \rangle$ decomp tree T. Then, G has a $\langle w_1', w_2', \ldots, w_{\lceil \lg r \rceil}' \rangle$ balanced decomp tree T', where

$$w_i' = 4 \sum_{k=i}^{r} w_k .$$

Pf. Color leaves of T: 1 = node of G, 0 = empty.



Recursively split B & W leaves evenly. Each stage has ≤ 2 strings of consec. leaves from T, each of which has ≤ 2 cbt's of a given height.

6.896
4/28/04
L20.5

Total # wires leaving a string
≤ sum of wires leaving each of its cbt's.

$$w_i' \le 4 \sum_{k=i}^{r} w_k. \quad \boxtimes$$

Corollary A graph with a (w,α) decomp tree, α const, has an $(O(w),\alpha)$ balanced decomp tree.

Pf. Sum is geometric:

$$w_i' = 4 \sum_{k=i}^{r} w_k$$

$$\le 4 \sum_{k=i}^{r} \frac{w}{\alpha^{k-1}}$$

$$\le \frac{4w}{\alpha^{i-1}} \left(\frac{\alpha}{\alpha - 1} \right).$$

Graph has $(4w\alpha/(\alpha-1), \alpha)$ decomp tree. $\boxtimes$

Next week: Embed in $\sqrt{\text{vol}}$ TOM ⇒ layout.
Area-universal networks.

«Exam issues»

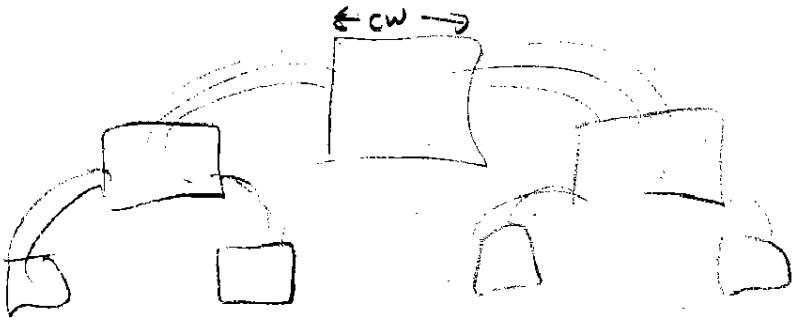
6.896
5/3/04
L21.1

Recall from last time:
- Truncated TOM(n, k) has area $O(n^2k^2)$.
- TOM(n) has area $O(n^2 \lg^2 n)$
- If G has a (w, α) decomp tree, it has a $(O(w), \alpha)$ balanced decomp tree.

Theorem Every N-node graph with a $(w, \sqrt{2})$ decomp tree can be laid out in $O(w^2 \lg^2(N/w))$ area.

Pf. Get an $(O(w), \sqrt{2})$ balanced decomp tree for G.

Embed G in $TOM(cw, 2\lg(N/w))$ for some const c:

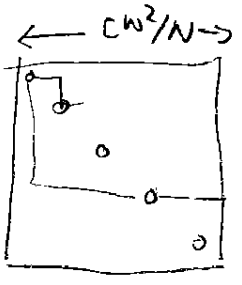


Leaf meshes: $\#\text{nodes} = \frac{(cw)^2}{2^{2\lg(N/w)}} = \frac{c^2w^2}{N^2/w^2} = \frac{c^2w^4}{N^2}$

$\text{side length} = \frac{cw^2}{N}$.

$\#\text{vertices in leaf meshes} = \frac{N}{2^{2\lg(N/w)}} = \frac{N}{N^2/w^2} = \frac{w^2}{N}$

$\#\text{edges leaving leaf mesh} = \frac{O(w)}{(\sqrt{2})^{2\lg(N/w)}} = \frac{O(w)}{N/w} = O\left(\frac{w^2}{N}\right)$



By adjusting c, can route edges within mesh + room on perimeter for wires to escape.

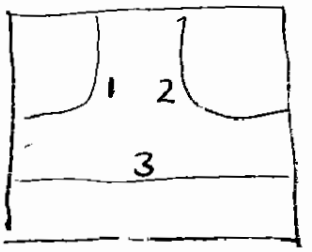
6.896
5/3/04
L21.2

At depth p, side length of mesh is $\geq \frac{cw}{2^{\lceil p/2 \rceil}}$.

edges leaving $= O(w)/(\sqrt{2})^{p-1} = O(w)/2^{p/2}$.

Adjust c for adequate capacity.

Routing internal-node meshes:



Type 1 & 2: 2 layers each.
Type 3: 3 layers

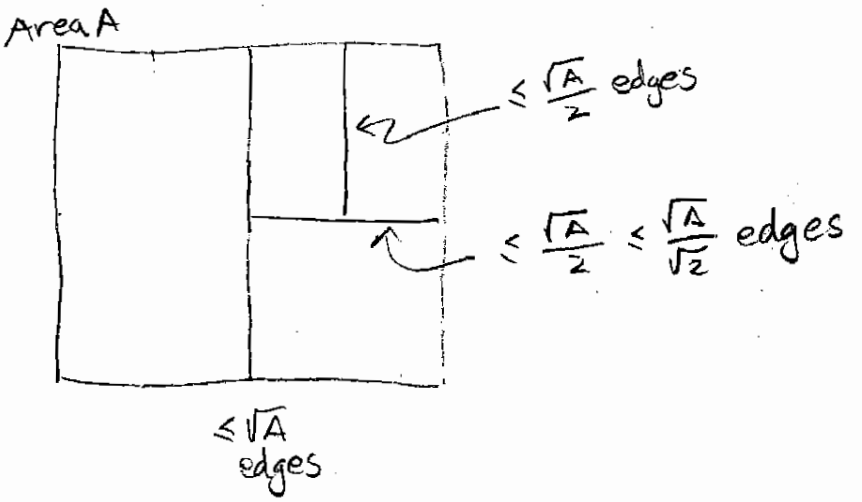
$\therefore$ 7 layers (squash to 2 if desired).

Area of TOM$(cw, 2\lg(N/w))$ is

$$O((cw)^2(2\lg(N/w))^2) = O(w^2\lg^2(N/w)) \quad \boxtimes$$

Corollary. Let w be smallest value for a $(w, \sqrt{2})$ decomp tree for N-node graph G. Let A be min area. Then, $w^2 \leq A \leq O(w^2\lg^2(N/w))$.

Pf.



edges leaving subgraph at depth p
$\leq \frac{\sqrt{A}}{\sqrt{2}^{p-1}}$

$\therefore$ G has $(\sqrt{A}, \sqrt{2})$ decomp tree $\Rightarrow w \leq \sqrt{A}$. $\boxtimes$

6.896
5/3/04
L21.3

Which network is best?

Network	Area	Routing Time	AT^2
Linear array	N	N	N^3
2D array	N	$\sqrt{N}$	N^2
3D array	$N^{4/3}$	$N^{1/3}$	N^2
CBT	N	N	N^3
hypercube	N^2	$\lg N$	$N^2 \lg^2 N$
butterfly	$N^2/\lg^2 N$	$\lg N$	N^2
2D MOT	$N \lg^2 N$	$\sqrt{N}$	$N^2 \lg^2 N$

Universality: An N-node butterfly can simulate any other N-node bounded-degree network with $O(\lg N)$ slowdown, just by routing messages.

Universal = expensive?

VLSI perspective: normalize to area, not #procs.

Area A network. Route A packets
- 2D array: $N = A$
  Route A packets in $\sqrt{A}$ time
- Butterfly: $N = \sqrt{A} \lg A$ ($\lg A \sim \lg N$)
  Route N packets in $\lg N$ time
  $\sqrt{A} \lg A$ $\lg A$

$\sqrt{A}/\lg A$ batches of $\sqrt{A} \lg A$ packets, each taking $\lg A$ time. Total time $= \sqrt{A}/\lg A \times \lg A = \sqrt{A}$.

Same! (Reason: basically since $AT^2 = N^2$ for both)

How can we compare? Ans. Simulate
- Can an area-A butterfly simulate any other area-A network efficiently? ($O(\lg A)$ slowdown)
  → Can't even do linear array.

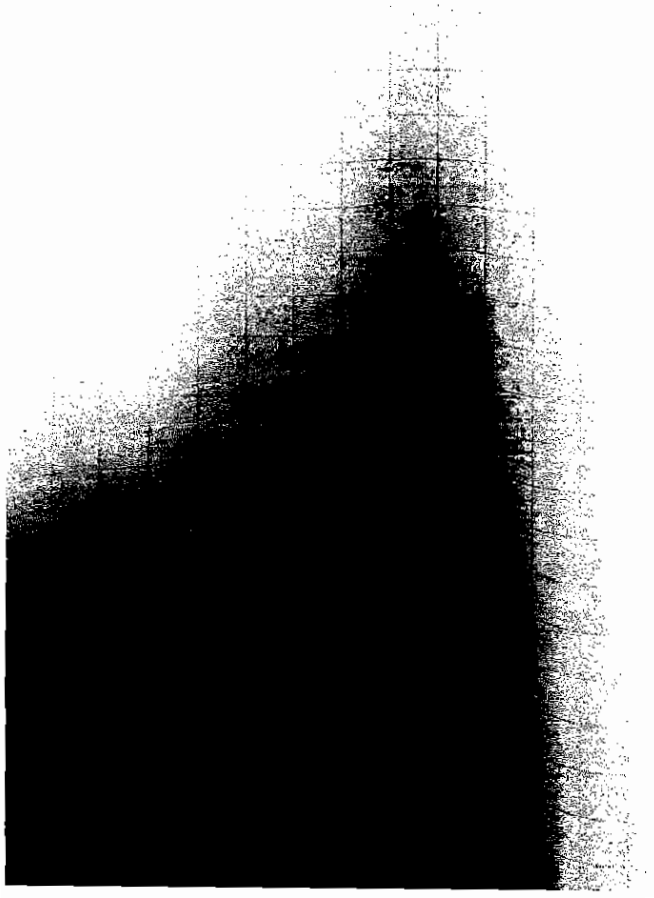
procs in butterfly $= \sqrt{A} \lg A$
procs in lin. array $= A$
Slowdown $= A / \sqrt{A} \lg A = \sqrt{A}/\lg A$

6.896
5/3/04
L21.4

1D array can't sim 2D array (diam)
2D array can't sim CBT (diam)
CBT can't sim 2D array (bis. width)
2D MOT can sim others with $lg^2 A$ slowdown.

Next time: "Area-universal" networks.
Idea: physical structure is TOM, but <u>low</u> diam.
"Fat-trees"

«Reminder: catch up on reading for final»

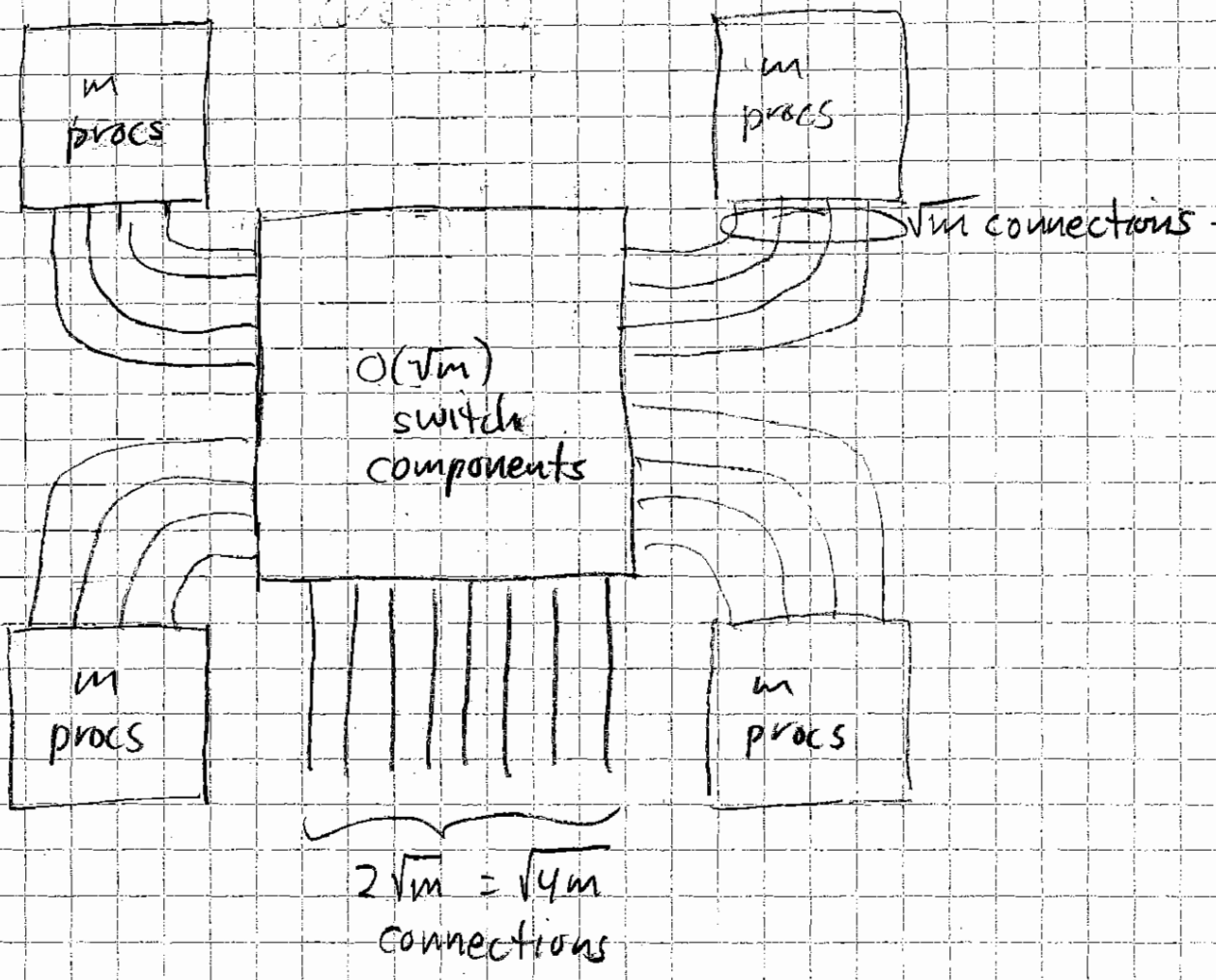


6.896
5/5/04
L22.1

Area-Universal Networks

Idea: Make #wires leaving region proportional to perimeter of region (like 2D mesh or TOM), but small diameter.

Fat-Tree (slide 2)



What is area $A(N)$ of N-leaf fat-tree?
Embed in $\overline{TOM}(\sqrt{N}) \Rightarrow A(N) = O(N \lg^2 N)$
or $S(N) = \sqrt{A(N)}$

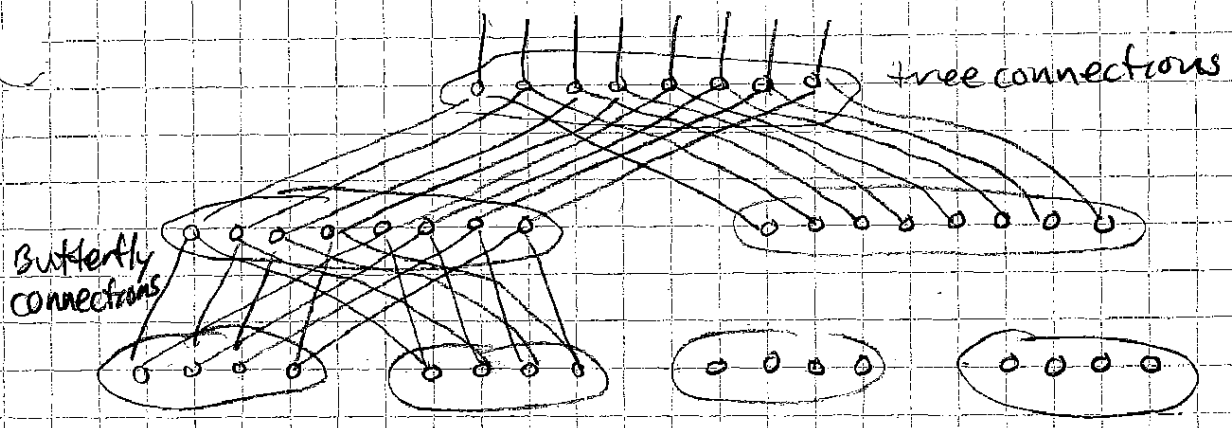
$$S(N) = 2S(N/4) + \Theta(\sqrt{N})$$
$$= \Theta(\sqrt{N} \lg N)$$
$$\therefore A(N) = \Theta(N \lg^2 N)$$

« Know master theorem for final exam! »

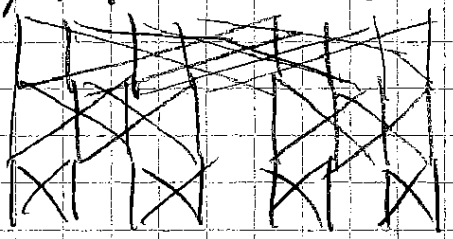
How many switches?
$$H(N) = 4H(N/4) + \Theta(\sqrt{N})$$
$$= \Theta(N)$$

6.896
5/5/04
L22.2

Butterfly fat-tree (Slide 3)

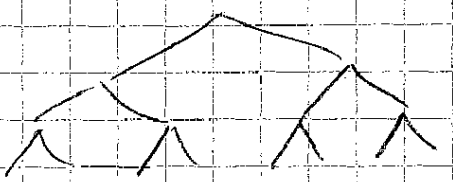


Only ⋈ connections



butterfly
(or Benes)

Only tree connections

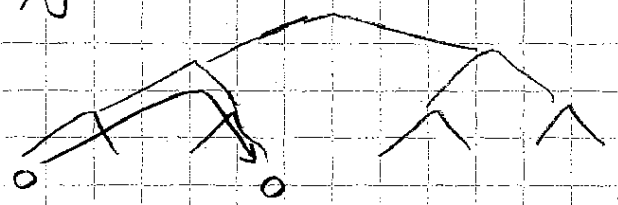


CBT

Alternate: area-universal network
Other ratios: variable growth of connections
- scalable to available technology
- no math law governs fatness.

Routing on fat-trees

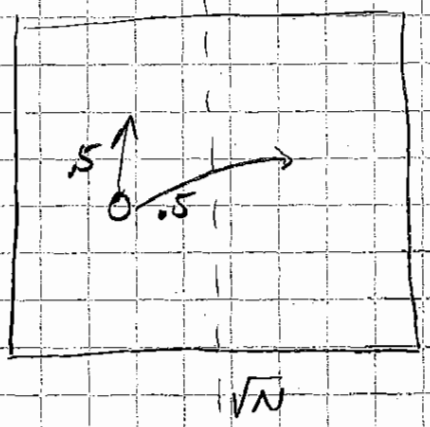
Message goes from source to dest via least common ancestor, just like tree:



Like phone network with exchanges.

6.896
5/5/04
L22.3

Issue: some routing problems harder than others.
Example: random communication is hard



$E[\#\text{crossing bisection}] = N/2$
Bandwidth of bisection $= \sqrt{N}$
Time $\geq \frac{N}{2} \div \sqrt{N} = \Omega(\sqrt{N})$

$$\text{Load factor } \lambda = \max_{\{\text{cuts}\}} \frac{\#\text{msgs crossing cut}}{\text{bandwidth of cut}}$$

Lemma. Any set of messages with load factor λ can be routed on an N-leaf fat-tree in expected time $O(\lambda + \lg N)$.

Pf. [Leighton-Maggs-Ranade-Rao] ☒

Theorem An N-leaf (N procs) fat-tree can simulate any area-N fat-tree in $O(\lg N)$ time.

Proof. (Slides 4-5) $\lambda = O(1)$ ☒

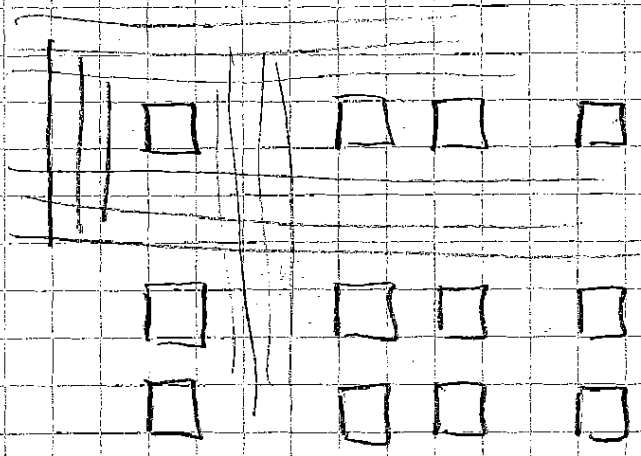
Tighter result: Area-$O(A)$ network that can simulate any other area-A network in $O(\lg A)$ time.
Idea: Area-A area-universal FT has $N = A/\lg^2 A$ procs. «sloppy asymptotics»
∴ Put $(\lg A) \times (\lg A)$ mesh at each leaf of area-universal FT with n leaves.



$N = n \lg^2 n$

6.896
5/5/04
L22.4

Area: 1 proc → $(\lg n) \times (\lg n)$ procs expands each dimension of layout by additive $\sqrt{n} \lg n$.



procs are aligned.

∴ Side length increases by const. factor.
$A = \Theta(n \lg^2 n)$. Procs are dense.

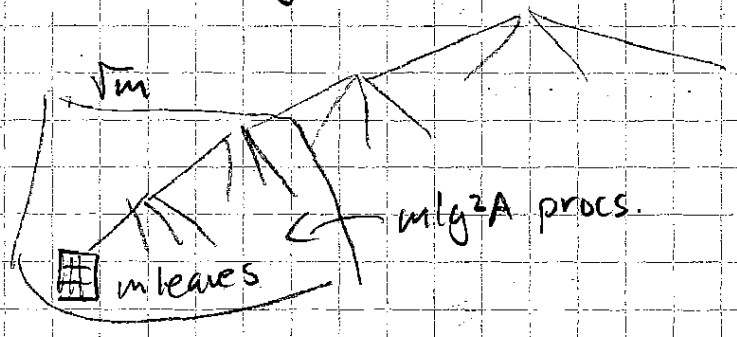
Simulation of network R

1. Divide R into $\sqrt{n} \times \sqrt{n}$ blocks of size $(\lg A) \times (\lg A)$

2. Simulate procs in (i,j) block of R by corresp. $(\lg A) \times (\lg A)$ leaf mesh of FT. Time $= O(\lg A)$

3. Communication among blocks:

- m-leaf subtree of FT has $\sqrt{m}$ external connections corresp. to $m \lg^2 A$-area region of R.



- Simulate wires of R with msgs.
How many msgs out of $m \lg^2 A$ area? $O(\sqrt{m} \lg A)$
$\lambda = \frac{\sqrt{m} \lg A}{\sqrt{m}} = \lg A \Rightarrow$ Routing time $= O(\lg A)$ ☒